



ChatGPT

RHR 8 Квест-курс «Чёрный пояс в ООП»

Модуль 1 - «Путь ученика» — Основы ООП в RHR 8

Урок 1 - Пробуждение переменных — магия инкапсуляции

Шаг 01:

Миссия 1

Молодой разработчик Никита написал класс User , но сделал все свойства публичными. Что он нарушил?

Подсказка: Инкапсуляция — это про защиту данных от прямого доступа.

Ответ:

- **Принцип инкапсуляции**
- Принцип наследования
- Принцип полиморфизма
- Принцип абстракции

Шаг 02:

Миссия 2

Программист Аня создала класс BankAccount и сделала свойство \$balance приватным. Какой способ доступа к балансу будет правильным?

Подсказка: Вспомните о геттерах и сеттерах.

Ответ:

- **Создать метод getBalance()**
- Объявить свойство публичным

- Использовать глобальную переменную
- Присваивать напрямую вне класса

🔮 Шаг 03:

Миссия 3

Петя создал класс Car и сделал метод drive() публичным. Почему именно публичный?

Подсказка: Подумайте, кто будет вызывать метод — сам класс или внешний код.

Ответ:

- **Метод должен быть доступен извне, когда объект используется**
- Метод нужен только для внутренних вычислений
- Метод скрывает детали реализации
- Метод не должен быть вызван

🔮 Шаг 04:

Миссия 4

Разработчик Костя сделал метод calculateTax() приватным. Зачем?

Подсказка: Такие методы обычно служат служебными, а не для внешнего вызова.

Ответ:

- **Метод используется только внутри класса**
- Метод нужен для вызова пользователем
- Метод должен быть защищён от наследников
- Метод всегда статический

🔮 Шаг 05:

Миссия 5

Саша сделал класс Profile, где данные пользователя закрыты, но доступны через методы. Как называется такой подход?

Подсказка: Вспомните основное свойство ООП — защита данных.

Ответ:

- **Инкапсуляция**
- Полиморфизм
- Наследование
- Абстракция

🔮 Шаг 06:

Миссия 6

Свяжите уровни доступа с их правильными объяснениями.

Подсказка: Подумайте, кто именно может видеть свойства и методы.

Ответ:

- **public** — Доступен отовсюду, включая внешний код
- **private** — Доступен только внутри класса
- **protected** — Доступен в классе и его наследниках

🔧 Шаг 07:

Миссия 7

Сопоставьте персонажей с их ошибками в инкапсуляции.

Подсказка: Вспомните, что публичные свойства опасны, а приватные — защищают.

Аня:

```
public class Car {  
    public String model;  
    public int speed;  
}
```

Кирилл:

```
public class User {  
    private String email;  
    private String password;  
    // нет геттеров и сеттеров  
}
```

Лена:

```
public class Engine {  
    public void calibrate() {  
        // служебный метод, не должен быть публичным  
    }  
}
```

Ответ:

- **Аня** — Сделала все свойства публичными
- **Кирилл** — Не добавил геттеры и сеттеры для приватных данных
- **Лена** — Сделала служебный метод публичным

🔧 Шаг 08:

Миссия 8

Соедините классы с тем, какие данные стоит скрывать.

Подсказка: Баланс счета явно не должен быть публичным.

Ответ:

- **BankAccount** — Баланс счета
- **User** — Пароль
- **Car** — Внутреннее состояние двигателя

🔧 Шаг 09:

Миссия 9

Сопоставьте примеры кода с правильным уровнем доступа.

Подсказка: Смотрите на ключевые слова перед свойством или методом.

Ответ:

- **public function run()** — Доступен всем
- **private \$password** — Доступен только внутри класса
- **protected function log()** — Доступен классу и наследникам

🔗 Шаг 10:

Миссия 10

Соедините описания методов с их назначением.

Подсказка: Определите, какие методы управляют внешними действиями, а какие — внутренними.

Ответ:

- **getName()** — Возвращает имя пользователя
- **setBalance()** — Изменяет баланс счета
- **calculateBonus()** — Внутренний расчет бонуса

🔗 Шаг 11:

Миссия 11

Соедините ключевые термины с их определениями.

Подсказка: Это основы ООП, без которых инкапсуляцию не понять.

Ответ:

- **Инкапсуляция** — Скрытие деталей реализации и управление доступом
- **Геттер** — Метод для получения значения
- **Сеттер** — Метод для изменения значения

🔗 Шаг 12:

Миссия 12

Расположите шаги при создании класса с инкапсуляцией в правильном порядке.

Подсказка: Сначала определяются свойства, потом доступ, затем методы.

Ответ:

- Определить свойства класса
- Выбрать уровень доступа
- Создать геттеры и сеттеры
- Использовать объект извне

🔗 Шаг 13:

Миссия 13

Упорядочьте действия при использовании приватного свойства `$salary` в классе `Employee`.

Подсказка: Вспомните про сеттер и геттер.

Ответ:

- Создать приватное свойство `$salary`
- Добавить сеттер `setSalary()`
- Добавить геттер `getSalary()`
- Использовать методы в объекте

🔗 Шаг 14:

Миссия 14

Расположите этапы исправления ошибки: свойства сделали публичными вместо приватных.

Подсказка: Нужно сначала изменить модификатор доступа, а потом добавить методы.

Ответ:

- Изменить свойства на private
- Создать геттеры
- Создать сеттеры
- Заменить прямой доступ на вызовы методов

 Шаг 15:

Миссия 15

Упорядочьте шаги работы с инкапсуляцией при проектировании класса Product .

Подсказка: Представьте процесс проектирования: свойства → доступ → методы.

Ответ:

- Определить, какие данные скрыть
- Выбрать модификаторы доступа
- Создать методы доступа
- Использовать класс в приложении

 Шаг 16:

Миссия 16

Расставьте действия программиста, который хочет защитить данные пользователя от прямого изменения.

Подсказка: Сначала скрываем данные, потом даём методы для доступа.

Ответ:

- Сделать свойство private
- Написать геттер
- Написать сеттер
- Протестировать вызовы

 Шаг 17:

Миссия 17

Упорядочьте шаги в истории программиста, который внедряет инкапсуляцию в старый код.

Подсказка: Подумайте, что сначала нужно спрятать данные, потом изменить доступ к ним.

Ответ:

- Найти публичные свойства
- Сделать их приватными
- Добавить методы доступа
- Заменить использование на вызовы методов

 Урок 2 - Тайное знание классов и объектов

 Шаг 01:

Миссия 1

Антон только что устроился стажёром в IT-компанию. Ему поручили создать первый класс в PHP. Что является правильным определением класса?

Подсказка: Вспомни, класс — это как чертёж для объектов.

Ответ:

- **Шаблон, описывающий свойства и методы объекта**
- Файл с настройками PHP
- Переменная для хранения строки
- Инструмент для компиляции кода

🔮 **Шаг 02:**

Миссия 2

Мария написала класс User и хочет создать объект. Как правильно это сделать?

Подсказка: Вспомни ключевое слово new .

Ответ:

- **`$u = new User();`**
- `$u = User;`
- `User($u);`
- `create User($u);`

🔮 **Шаг 03:**

Миссия 3

Вадим сделал класс Car , но забыл про конструктор. Какой метод является конструктором в PHP 8?

Подсказка: Вспомни магический метод с двумя подчёркиваниями.

Ответ:

- **`__construct`**
- `init`
- `create`
- `__start`

🔮 **Шаг 04:**

Миссия 4

Игорь добавил в класс BankAccount свойство \$balance . Какое ключевое слово лучше использовать для безопасности?

Подсказка: Думай о принципе инкапсуляции.

Ответ:

- **`private`**
- `public`
- `static`
- `global`

🔮 **Шаг 05:**

Миссия 5

Света хочет, чтобы её метод greet() был доступен для всех объектов класса. Какой модификатор доступа выбрать?

Подсказка: Вспомни открытый доступ.

Ответ:

- **public**
- private
- protected
- hidden

🔮 Шаг 06:

Миссия 6

Александр изучает ключевые слова для свойств и методов. Сопоставьте модификаторы доступа с их смыслом.

Подсказка: Думай, кто может получить доступ: весь мир, только класс или наследники.

Ответ:

- **public** — Доступен отовсюду
- **private** — Доступен только внутри текущего класса
- **protected** — Доступен внутри класса и его наследников
- **final** — Запрещает переопределение метода

🔮 Шаг 07:

Миссия 7

У Лены есть класс Book с методами. Сопоставь элементы класса с их описанием.

Подсказка: Вспомни разницу между свойствами и методами.

Ответ:

- **Свойство** — Переменная внутри класса
- **Метод** — Функция внутри класса
- **Конструктор** — Метод для инициализации объекта
- **Объект** — Экземпляр класса

🔮 Шаг 08:

Миссия 8

Разработчики спорят о терминах. Сопоставьте понятия ООП с их объяснениями.

Подсказка: Подумай о трёх китах ООП: инкапсуляция, наследование и полиморфизм.

Ответ:

- **Инкапсуляция** — Соккрытие деталей реализации внутри класса
- **Наследование** — Передача свойств и методов дочерним классам
- **Полиморфизм** — Один интерфейс — много реализаций
- **Абстракция** — Выделение главных характеристик объекта

🔮 Шаг 09:

Миссия 9

Степан хочет закрепить знание магических методов. Сопоставьте метод с его назначением.

Подсказка: Магические методы начинаются с двух подчёркиваний.

Ответ:

- **__construct** — Вызывается при создании объекта
- **__destruct** — Вызывается при удалении объекта

- **__toString** — Определяет строковое представление объекта
- **__call** — Вызывается при обращении к несуществующему методу

Шаг 10:

Миссия 10

Юлия создала класс Order . Сопоставьте ключевые действия с методами.

Подсказка: Смотри на название метода и подумай, что он делает.

Ответ:

- **addItem()** — Добавить товар в заказ
- **removeItem()** — Удалить товар из заказа
- **getTotal()** — Посчитать общую сумму
- **confirm()** — Подтвердить заказ

Шаг 11:

Миссия 11

Павел изучает объекты. Сопоставьте действие с его определением.

Подсказка: Объекты могут быть созданы, изменены и уничтожены.

Ответ:

- **Создание объекта** — Вызов конструктора через new
- **Изменение состояния** — Присвоение значений свойствам
- **Вызов метода** — Запуск функции объекта
- **Удаление объекта** — Вызов деструктора

Шаг 12:

Миссия 12

Команда обсуждает понятие объектов. Сопоставьте термины с их определениями.

Подсказка: Объект = данные + поведение.

Ответ:

- **Класс** — Шаблон для создания объектов
- **Объект** — Экземпляр класса
- **Свойство** — Переменная объекта
- **Метод** — Функция объекта

Шаг 13:

Миссия 13

Определи правильный порядок работы с классом.

Подсказка: Сначала создаём класс, потом объект, потом работаем с ним.

Ответ:

- Создать класс
- Создать объект с помощью new
- Вызвать методы объекта
- Изменить свойства объекта

Шаг 14:

Миссия 14

Расставь этапы создания интернет-магазина с использованием ООП.

Подсказка: Думай от проектирования до использования.

Ответ:

- Проектирование классов
- Реализация свойств и методов
- Создание объектов
- Вызов методов для обработки заказов

Шаг 15:

Миссия 15

Расставь порядок вызова магических методов при жизненном цикле объекта.

Подсказка: Вспомни: сначала конструктор, потом работа, потом деструктор.

Ответ:

- `__construct`
- Вызов методов объекта
- Изменение свойств
- `__destruct`

Шаг 16:

Миссия 16

Программист пишет код. Определи порядок шагов при создании класса с инкапсуляцией.

Подсказка: Сначала проектируем, потом пишем код.

Ответ:

- Определить свойства как `private`
- Добавить методы для работы со свойствами
- Создать конструктор
- Создать объект класса

Шаг 17:

Миссия 17

Расставь шаги при работе с классами и объектами в проекте.

Подсказка: Сначала создаём основу, потом используем.

Ответ:

- Написать класс
- Создать объект
- Вызвать метод
- Удалить объект

Урок 3 - Врата наследования

Шаг 01:

Миссия 1

Младший разработчик Костя создает свой первый класс Car в PHP. Что обязательно должно быть внутри определения класса?

Подсказка: Вспомните: класс — это как чертеж, а без деталей он бесполезен.

Ответ:

- **Свойства или методы**
- Комментарии
- Импорт CSS-стилей
- SQL-запросы напрямую

Шаг 02:

Миссия 2

Оля пишет класс User . Она хочет, чтобы у каждого пользователя было имя. Как лучше это описать?

Подсказка: Имя должно храниться внутри объекта и быть доступно при работе.

Ответ:

- **Добавить свойство \$name**
- Написать echo 'Имя'; в классе
- Создать HTML-тег Имя
- Сделать отдельный SQL-таблицу

Шаг 03:

Миссия 3

Разработчик Антон добавил метод drive() в класс Car . Что такое метод в ООП?

Подсказка: Метод — это действие, которое может выполнять объект.

Ответ:

- **Функция, принадлежащая классу**
- Переменная глобальной области
- Таблица в базе данных
- Комментарий для разработчиков

Шаг 04:

Миссия 4

Никита создал класс Programmer с методом writeCode() . Как вызвать этот метод у объекта?

Подсказка: Нужно сначала создать объект через new .

Ответ:

- **\$dev = new Programmer(); \$dev->writeCode();**
- Programmer::writeCode(); без объекта
- include 'Programmer.php';
- echo Programmer;

Шаг 05:

Миссия 5

Лена заметила, что при создании объекта удобно сразу задавать значения. Какой метод для этого используется?

Подсказка: Этот метод запускается автоматически при new .

Ответ:

- **__construct**
- **__init**
- **__create**
- **__start**

Шаг 06:

Миссия 6

Влад хочет сделать свойство `$salary` в классе `Employee` доступным только внутри класса. Какой модификатор он выберет?

Подсказка: Вспомните магию инкапсуляции — защита данных.

Ответ:

- **private**
- `public`
- `static`
- `global`

Шаг 07:

Миссия 7

Андрей использует наследование. Класс `Dog` расширяет класс `Animal`. Как правильно это записать?

Подсказка: Вспомните ключевое слово для расширения возможностей класса.

Ответ:

- **class Dog extends Animal**
- `class Dog implements Animal`
- `class Dog inherits Animal`
- `class Dog use Animal`

Шаг 08:

Миссия 8

Сопоставьте ключевые слова с их назначением в PHP-классах:

Подсказка: Вспомните уровни доступа и их ограничения.

Ответ:

- **public** — Доступно везде
- **protected** — Доступно в классе и его наследниках
- **private** — Доступно только внутри текущего класса
- **extends** — Используется для наследования

Шаг 09:

Миссия 9

Сопоставьте примеры кода с их описанием:

Подсказка: Обратите внимание на ключевые слова и синтаксис.

Ответ:

- **class Student { }** — Определение нового класса

- **new Student()** — Создание объекта
- **__construct()** — Конструктор класса
- **extends** — Ключевое слово для наследования

🔗 Шаг 10:

Миссия 10

Сопоставьте понятие с его примером:

Подсказка: Подумайте, что относится к свойствам, а что к методам.

Ответ:

- **Свойство** — \$balance
- **Метод** — deposit()
- **Конструктор** — __construct()
- **Объект** — new BankAccount()

🔗 Шаг 11:

Миссия 11

Сопоставьте персонажей с их классами:

Подсказка: Вспомните: класс — это шаблон, объект — его конкретный экземпляр.

Ответ:

- **Вася** — водитель — class Driver
- **Маша** — студентка — class Student
- **Петя** — разработчик — class Programmer
- **Кот Барсик** — class Cat

🔗 Шаг 12:

Миссия 12

Сопоставьте понятия ООП с их объяснением:

Подсказка: Инкапсуляция защищает данные, наследование передает свойства.

Ответ:

- **Инкапсуляция** — Скрытие деталей реализации
- **Наследование** — Передача свойств и методов родителя
- **Полиморфизм** — Разные реализации одного метода
- **Класс** — Шаблон для создания объектов

🔗 Шаг 13:

Миссия 13

Расставьте шаги создания класса в правильном порядке:

Подсказка: Сначала нужен чертеж, затем детали, потом объекты.

Ответ:

- Объявить класс
- Добавить свойства и методы
- Создать объект через new
- Вызвать методы объекта

Шаг 14:

Миссия 14

Упорядочьте действия программиста при наследовании классов:

Подсказка: Сначала создается базовый класс, потом наследник.

Ответ:

- Создать базовый класс Animal
- Добавить свойства и методы в Animal
- Создать класс Dog extends Animal
- Добавить уникальные методы в Dog

Шаг 15:

Миссия 15

Расставьте шаги работы конструктора:

Подсказка: Конструктор выполняется сразу при создании объекта.

Ответ:

- Вызов new ClassName()
- Запуск __construct()
- Присвоение свойств
- Готовый объект возвращается

Шаг 16:

Миссия 16

Определите порядок действий при вызове метода у объекта:

Подсказка: Нельзя вызвать метод, пока объект не создан.

Ответ:

- Определить метод в классе
- Создать объект через new
- Вызвать метод через ->
- Выполнить действия внутри метода

Шаг 17:

Миссия 17

Упорядочьте шаги при проектировании ООП-системы:

Подсказка: Сначала проектирование, потом код, затем тестирование.

Ответ:

- Определить классы и их роли
- Прописать свойства и методы
- Реализовать классы в коде
- Протестировать систему

Урок 4 - Полиморфизм Искусство подмены

Шаг 01:

Миссия 1

Андрей пишет программу для онлайн-магазина. Он хочет, чтобы класс Payment имел разные реализации: CreditCardPayment , PaypalPayment . Что это примером является?

Подсказка: Когда один интерфейс может использоваться с разными реализациями.

Ответ:

- **Полиморфизм**
- Инкапсуляция
- Абстракция
- Наследование

Шаг 02:

Миссия 2

Мария делает чат-бота. У неё есть метод sendMessage() , который ведёт себя по-разному у классов EmailBot и TelegramBot . Как называется эта особенность?

Подсказка: Один метод — разное поведение в зависимости от объекта.

Ответ:

- **Полиморфизм методов**
- Инкапсуляция данных
- Наследование свойств
- Статический анализ

Шаг 03:

Миссия 3

Вася пишет игру. У него есть интерфейс Weapon и реализации: Sword , Bow . Метод attack() у каждого оружия работает по-своему. Это пример чего?

Подсказка: Каждый класс реализует общий интерфейс, но поведение разное.

Ответ:

- **Полиморфизм через интерфейсы**
- Наследование классов
- Статическая типизация
- Абстракция данных

Шаг 04:

Миссия 4

Катя проектирует API. У неё есть метод calculate() , который принимает объект Shape . Класс Circle и Square реализуют этот метод по-разному. Что позволяет ей писать общий код для разных фигур?

Подсказка: Подстановка объектов разных типов без изменения логики вызова.

Ответ:

- **Поддержка полиморфизма**
- Статическое наследование
- Инкапсуляция методов
- Множественное наследование

Шаг 05:

Миссия 5

Разработчик Петя использует оператор `instanceof`, чтобы проверить тип объекта перед вызовом метода. Какой принцип ООП он тем самым нарушает?

Подсказка: Суть полиморфизма — избегать ручной проверки типа.

Ответ:

- **Принцип полиморфизма**
- Принцип DRY
- Принцип KISS
- Принцип наследования

🔧 Шаг 06:

Миссия 6

Сопоставьте понятия полиморфизма с их определениями.

Подсказка: Вспомните ключевые виды полиморфизма в PHP.

Ответ:

- **Полиморфизм через наследование** — Разные классы наследуют базовый и переопределяют его методы.
- **Полиморфизм через интерфейсы** — Разные классы реализуют один и тот же контракт.
- **Полиморфизм времени компиляции** — Методы с одинаковым именем, но разными сигнатурами (PHP поддерживает частично).
- **Полиморфизм времени выполнения** — Разное поведение метода в зависимости от конкретного объекта.

🔧 Шаг 07:

Миссия 7

Сопоставьте разработчиков и их примеры кода с типом полиморфизма.

Подсказка: Смотрите, кто использует интерфейсы, а кто наследование.

Ответ:

- **Иван: класс `Dog` наследует `Animal` и переопределяет `bark()`** — Полиморфизм через наследование
- **Ольга: интерфейс `Logger` с реализациями `FileLogger` и `DbLogger`** — Полиморфизм через интерфейсы
- **Сергей: метод `sum(int $a, int $b)` и `sum(float $a, float $b)`** — Полиморфизм времени компиляции
- **Мария: вызов метода `render()` для разных объектов `Page` и `Post`** — Полиморфизм времени выполнения

🔧 Шаг 08:

Миссия 8

Сопоставьте примеры с их определением.

Подсказка: Обратите внимание, как объекты подменяют поведение.

Ответ:

- **Класс `Car` и `Truck` наследуют `Vehicle` и реализуют `drive()`** — Разные реализации метода в наследниках
- **Класс `MysqlDb` и `PostgresDb` реализуют интерфейс `DbConnector`** — Единый контракт с разными реализациями
- **Класс `Shape` и наследники `Circle` и `Square`** — Общий метод `area()` с разным поведением

- **Класс `AudioPlayer` и `VideoPlayer` с методом `play()`** — Полиморфизм во время выполнения

🔗 Шаг 09:

Миссия 9

Сопоставьте принципы с правильными формулировками.

Подсказка: Вспомните SOLID и его связь с полиморфизмом.

Ответ:

- **Принцип подстановки Лисков** — Объекты дочерних классов должны заменять родительские без ошибок.
- **Принцип единственной ответственности** — Каждый класс отвечает за одну задачу.
- **Принцип открытости/закрытости** — Классы открыты для расширения, но закрыты для изменения.
- **Принцип разделения интерфейсов** — Лучше несколько мелких интерфейсов, чем один большой.

🔗 Шаг 10:

Миссия 10

Сопоставьте ситуации с соответствующим типом полиморфизма.

Подсказка: Подумайте, когда речь идёт об интерфейсах, а когда о наследовании.

Ответ:

- **Функция `processPayment` принимает любой объект, реализующий `PaymentInterface`** — Полиморфизм через интерфейсы
- **Класс `Cat` переопределяет метод `makeSound()` базового класса `Animal`** — Полиморфизм через наследование
- **Метод `calculate()` работает по-разному у `Circle` и `Rectangle`** — Полиморфизм времени выполнения
- **Метод `log()` в `Logger` вызывается для разных реализаций `FileLogger` и `ConsoleLogger`** — Единый контракт с разными реализациями

🔗 Шаг 11:

Миссия 11

Расположите шаги реализации полиморфизма в правильном порядке.

Подсказка: Сначала проектируем интерфейс, потом классы, затем используем.

Ответ:

- Определить интерфейс или базовый класс
- Реализовать методы в наследниках
- Создать объекты классов-наследников
- Использовать общий метод для разных объектов

🔗 Шаг 12:

Миссия 12

Расположите действия при добавлении нового типа оплаты в интернет-магазин.

Подсказка: Используйте интерфейс `PaymentInterface`.

Ответ:

- Создать новый класс, реализующий `PaymentInterface`

- Определить метод `pay()` в новом классе
- Зарегистрировать новый класс в фабрике оплат
- Использовать его в клиентском коде без изменения логики

Шаг 13:

Миссия 13

Расположите шаги проверки корректности полиморфизма.

Подсказка: Сначала пишем тесты, потом проверяем замену классов.

Ответ:

- Написать тесты для базового класса
- Заменить базовый класс на наследника
- Запустить тесты
- Убедиться, что поведение корректно

Шаг 14:

Миссия 14

Упорядочьте шаги проектирования системы с использованием полиморфизма.

Подсказка: Вспомните: анализ → проектирование → реализация → тесты.

Ответ:

- Анализ требований
- Проектирование интерфейсов
- Реализация классов
- Тестирование поведения

Шаг 15:

Миссия 15

Расположите шаги использования полиморфизма при разработке плеера.

Подсказка: Общий метод `play()` должен вызываться у разных медиа.

Ответ:

- Создать интерфейс `Media`
- Реализовать `Audio` и `Video` классы
- Написать метод `play()` для каждого
- Вызывать `play()` у объектов в общем списке

Шаг 16:

Миссия 16

Упорядочьте шаги добавления нового класса `Document` в систему редактора.

Подсказка: Используйте общий интерфейс `Renderable`.

Ответ:

- Создать класс `Document`
- Реализовать интерфейс `Renderable`
- Определить метод `render()`
- Использовать `Document` наряду с другими `Renderable`

Урок 5 - Абстракции против хаоса

Шаг 01:

Миссия 1

Программист Дима устал от хаоса в коде и решил применить абстракцию. Что это значит в контексте PHP 8?

Подсказка: Абстракция скрывает детали и показывает только суть.

Ответ:

- **Соккрытие деталей реализации и работа через интерфейсы/абстрактные классы**
- Добавление новых комментариев к каждой функции
- Копирование кода из одного класса в другой
- Создание переменных с длинными именами

Шаг 02:

Миссия 2

Анна пишет систему бронирования. Она хочет определить общий контракт для разных способов оплаты (карта, PayPal, криптовалюта). Что ей лучше использовать?

Подсказка: Контракт в ООП — это не бумага, а структура кода.

Ответ:

- **Интерфейс PaymentMethod**
- Абстрактный класс Booking
- Таблицу в базе данных
- Статическую функцию validatePayment()

Шаг 03:

Миссия 3

В команде проекта обсуждают: абстрактный класс и интерфейс — в чём разница?

Подсказка: Один задаёт форму, другой — и форму, и частичное содержание.

Ответ:

- **Интерфейс описывает только методы без реализации, абстрактный класс может содержать реализацию**
- **Интерфейс можно наследовать только один, абстрактный класс — несколько**
- **Интерфейс можно создавать через new, абстрактный класс — нельзя**
- **Абстрактный класс существует только в Java, а интерфейсы — в PHP**

Шаг 04:

Миссия 4

Сергей создаёт приложение для доставки еды. Ему нужен класс Dish, у которого есть обязательный метод cook(), но у каждого блюда он реализуется по-разному. Как это лучше оформить?

Подсказка: Подумай о базовом плане, который все блюда обязаны выполнять.

Ответ:

- **Создать абстрактный класс Dish с абстрактным методом cook()**
- Создать интерфейс Cookable с методом serve()
- Написать один общий метод cook() и использовать для всех блюд

- Использовать глобальную функцию `cookDish()`

🔧 Шаг 05:

Миссия 5

Маша проектирует библиотеку для работы с документами. Какой принцип абстракции она применяет, если создаёт интерфейс `Exportable` с методом `export()` ?

Подсказка: Подумай: интерфейс задаёт контракт, но не детали.

Ответ:

- **Программирование через интерфейсы**
- Полиморфизм через наследование
- Инкапсуляция данных
- Глобальные переменные для экспорта

🔧 Шаг 06:

Миссия 6

📖 Сопоставьте понятия абстракции с их описаниями.

Подсказка: Вспомни: интерфейс — форма, абстрактный класс — ещё и часть содержимого.

Ответ:

- **Интерфейс** — Определяет только набор методов без реализации
- **Абстрактный класс** — Может содержать как абстрактные, так и реализованные методы
- **Контракт** — Обязательства, которые берёт на себя класс при реализации
- **Абстракция** — Соккрытие деталей и выделение сути объекта

🔧 Шаг 07:

Миссия 7

📖 Соедини тип абстракции и пример из кода.

Подсказка: У каждого подхода свой стиль: интерфейсы для контракта, абстрактные классы для общих основ.

Ответ:

- **Интерфейс `Logger`** — Классы `FileLogger` и `DbLogger` реализуют метод `log()`
- **Абстрактный класс `Shape`** — Метод `area()` обязан быть реализован в подклассах
- **Интерфейс `Serializable`** — Любой класс, реализующий его, должен иметь метод `serialize()`
- **Абстрактный класс `Animal`** — Метод `makeSound()` реализуется по-разному

🔧 Шаг 08:

Миссия 8

📖 Определи, что ближе к интерфейсу, а что к абстрактному классу.

Подсказка: Смотри, где есть реализация, а где только контракт.

Ответ:

- **Класс `ReportGenerator`** — Наследует абстрактный класс и реализует часть логики
- **Интерфейс `JsonExportable`** — Заставляет реализовать метод `exportToJson()`
- **Класс `AbstractCache`** — Содержит абстрактный метод `store()` и готовый метод `clear()`
- **Интерфейс `Notifiable`** — Требуется метод `notify()`

Шаг 09:

Миссия 9

 Найди соответствие между понятием и примером абстракции.

Подсказка: Программисты часто используют эти абстракции для гибкости и поддержки.

Ответ:

- **Интерфейс Repository** — Любой класс, реализующий его, управляет доступом к данным
- **Абстрактный класс Transport** — Метод move() должен быть реализован в наследниках
- **Интерфейс Authenticatable** — Класс обязан иметь метод authenticate()
- **Абстрактный класс Controller** — Может содержать базовые методы для обработки запросов

Шаг 10:

Миссия 10

 Соедини принцип и его пользу в проекте.

Подсказка: Абстракции спасают проект от хаоса.

Ответ:

- **Программирование через интерфейсы** — Код становится гибким и легко заменяемым
- **Абстрактные классы** — Можно задавать общую основу и переопределять детали
- **Инкапсуляция** — Скрывает внутреннюю реализацию объекта
- **Абстракция** — Позволяет выделить главное и игнорировать второстепенное

Шаг 11:

Миссия 11

Расположи шаги создания абстрактного класса.

Подсказка: Сначала проект, потом методы, потом реализация.

Ответ:

- Определить общую сущность
- Выбрать методы, которые должны быть абстрактными
- Добавить методы с готовой логикой
- Создать классы-наследники и реализовать абстрактные методы

Шаг 12:

Миссия 12

Упорядочи шаги применения интерфейса в проекте.

Подсказка: Контракт сначала, реализация позже.

Ответ:

- Определить интерфейс с методами
- Создать класс, реализующий интерфейс
- Написать код, работающий с интерфейсом
- Тестировать разные реализации

Шаг 13:

Миссия 13

Упорядочи действия программиста при добавлении нового вида транспорта (например, электросамокат) в систему с абстрактным классом Transport.

Подсказка: Сначала проект, потом код, потом тесты.

Ответ:

- Создать класс Scooter, наследующий Transport
- Реализовать метод move() для Scooter
- Зарегистрировать новый класс в системе
- Протестировать работу нового транспорта

Шаг 14:

Миссия 14

Расположи шаги использования абстракции при работе с оплатой.

Подсказка: Контракт для всех способов оплаты задаётся один раз.

Ответ:

- Создать интерфейс PaymentMethod
- Реализовать его в классах CardPayment и CryptoPayment
- Написать сервис, принимающий PaymentMethod
- Добавить новые способы оплаты при необходимости

Шаг 15:

Миссия 15

Расположи этапы проектирования через абстрактные классы.

Подсказка: Общая основа важнее деталей.

Ответ:

- Определить базовый абстрактный класс
- Добавить обязательные абстрактные методы
- Реализовать подклассы с конкретной логикой
- Проверить работу через тесты

Шаг 16:

Миссия 16

Упорядочи шаги использования интерфейсов для уведомлений.

Подсказка: Интерфейс определяет контракт для всех видов уведомлений.

Ответ:

- Создать интерфейс Notifiable
- Реализовать его в EmailNotifier и SmsNotifier
- Написать сервис, который принимает Notifiable
- Протестировать с разными реализациями

Урок 6 - Интерфейсы Печать договорённости

Шаг 01:

Миссия 1

Ты работаешь в команде над CRM-системой. Коллега предлагает использовать интерфейсы, чтобы упростить замену реализации отправки уведомлений. Какое ключевое преимущество интерфейсов в PHP 8?

Подсказка: Подумай о гибкости и возможностях подмены классов без изменения остального кода.

Ответ:

- **Они задают контракт, который могут реализовать разные классы**
- Они автоматически ускоряют работу кода
- Они хранят данные в базах MySQL без SQL-запросов
- Они заменяют использование абстрактных классов

Шаг 02:

Миссия 2

Разработчик хочет объявить интерфейс для класса оплаты. Как правильно это сделать в PHP 8?

Подсказка: Вспомни ключевое слово для объявления интерфейса.

Ответ:

- **interface PaymentProcessor { public function pay(float \$amount); }**
- class PaymentProcessor { public function pay(float \$amount); }
- trait PaymentProcessor { public function pay(float \$amount); }
- abstract PaymentProcessor { public function pay(float \$amount); }

Шаг 03:

Миссия 3

Ты пишешь модуль отправки сообщений. В проекте 3 разных способа отправки: Email, SMS и Push. Как интерфейс поможет тебе?

Подсказка: Вспомни про возможность использовать общий тип в коде.

Ответ:

- **Можно определить общий метод send(), а классы реализуют его по-своему**
- Можно подключить один класс, который решит все задачи
- Интерфейсы автоматически пишут код за программиста
- Они заменяют тестирование

Шаг 04:

Миссия 4

Коллега говорит: «Интерфейсы помогают соблюдать принцип единственной ответственности (SRP)». Почему это так?

Подсказка: Вспомни, что интерфейс концентрируется только на поведении.

Ответ:

- **Интерфейс описывает только набор методов, не смешивая логику и данные**
- Интерфейс всегда содержит реализацию
- Интерфейс уменьшает количество кода
- Интерфейс позволяет наследовать несколько классов сразу

Шаг 05:

Миссия 5

Ты внедряешь оплату через разные платёжные системы. Как использовать интерфейс для удобного тестирования?

Подсказка: Подумай о возможности подмены классов в тестах.

Ответ:

- **Реализовать интерфейс MockPayment для тестов и подставлять его вместо реальных**
- Отключить интерфейс во время тестов
- Интерфейс генерирует тесты автоматически
- Интерфейсы не используются в тестировании

🔗 Шаг 06:

Миссия 6

Сопоставь интерфейсы с их ролью в проекте.

Подсказка: Подумай, как разные интерфейсы решают конкретные задачи.

Ответ:

- **LoggerInterface** — Определяет метод log() для записи сообщений
- **CacheInterface** — Содержит методы get() и set() для хранения данных
- **PaymentInterface** — Определяет метод pay() для проведения оплаты
- **NotifierInterface** — Задаёт метод send() для уведомлений

🔗 Шаг 07:

Миссия 7

Ты подключаешь библиотеку. Сопоставь интерфейс с примером реализации.

Подсказка: Обрати внимание, что реализация должна логично соответствовать контракту.

Ответ:

- **LoggerInterface** — FileLogger
- **CacheInterface** — RedisCache
- **PaymentInterface** — StripePayment
- **NotifierInterface** — EmailNotifier

🔗 Шаг 08:

Миссия 8

Сопоставь интерфейс с принципом SOLID, который он помогает реализовать.

Подсказка: Вспомни, какой принцип связан с заменой реализаций без изменения кода.

Ответ:

- **PaymentInterface** — Open/Closed Principle (OCP)
- **LoggerInterface** — Dependency Inversion Principle (DIP)
- **NotifierInterface** — Interface Segregation Principle (ISP)
- **CacheInterface** — Single Responsibility Principle (SRP)

🔗 Шаг 09:

Миссия 9

Программист создал интерфейс и реализовал его в нескольких классах. Сопоставь действие с его результатом.

Подсказка: Думай о том, что меняется, а что остаётся одинаковым.

Ответ:

- **Реализация интерфейса** — Класс обязан реализовать все методы
- **Использование интерфейса в коде** — Можно подставить любую реализацию
- **Добавление нового класса** — Не требует изменения остального кода
- **Удаление реализации** — Не ломает интерфейс

Шаг 10:

Миссия 10

Сопоставь интерфейсы с областью их применения в типичном веб-проекте.

Подсказка: Вспомни, где обычно используются логирование, кэширование и уведомления.

Ответ:

- **LoggerInterface** — Запись ошибок в файл или консоль
- **CacheInterface** — Хранение временных данных
- **NotifierInterface** — Отправка сообщений пользователям
- **PaymentInterface** — Интеграция с платёжными сервисами

Шаг 11:

Миссия 11

Сопоставь интерфейс с преимуществом, которое он даёт проекту.

Подсказка: Интерфейсы повышают гибкость и тестируемость.

Ответ:

- **LoggerInterface** — Легко заменить FileLogger на ConsoleLogger
- **CacheInterface** — Можно быстро сменить Redis на Memcached
- **PaymentInterface** — Можно переключиться с PayPal на Stripe
- **NotifierInterface** — Поддержка разных способов уведомлений

Шаг 12:

Миссия 12

Расположи шаги создания интерфейса в правильном порядке.

Подсказка: Вспомни: сначала контракт, потом реализация, потом использование.

Ответ:

- Объявить интерфейс с методами
- Реализовать интерфейс в классе
- Подключить класс через интерфейс
- Заменить реализацию без изменения кода

Шаг 13:

Миссия 13

Расположи этапы использования интерфейсов для тестирования.

Подсказка: Сначала нужен контракт, затем реализация и только потом подмена в тестах.

Ответ:

- Создать интерфейс
- Реализовать реальный класс
- Создать мок-реализацию

- Подключить мок в тестах

Шаг 14:

Миссия 14

Расположи преимущества интерфейсов от самого очевидного к более продвинутому.

Подсказка: Думай от базовых преимуществ к более стратегическим.

Ответ:

- Повышение читаемости кода
- Гибкость при замене реализаций
- Упрощение тестирования
- Следование принципам SOLID

Шаг 15:

Миссия 15

Расположи действия при интеграции новой платёжной системы через интерфейс.

Подсказка: Подумай о порядке: от создания контракта до использования.

Ответ:

- Реализовать PaymentInterface
- Подключить новый класс в код
- Провести тестирование
- Запустить в продакшн

Шаг 16:

Миссия 16

Расположи шаги, которые выполняет команда при рефакторинге с использованием интерфейсов.

Подсказка: Важно сохранить функциональность, но улучшить архитектуру.

Ответ:

- Выделить общий контракт
- Создать интерфейс
- Реализовать интерфейс в классах
- Заменить зависимости в коде

Шаг 17:

Миссия 17

Расположи шаги работы программиста с интерфейсами при написании нового модуля.

Подсказка: Сначала проектирование, потом реализация, а в конце интеграция.

Ответ:

- Определить контракт интерфейса
- Создать реализацию
- Интегрировать в систему
- Написать тесты для проверки

Урок 7 - Конструкторы и разрушители миров

Шаг 01:

Миссия 1

Анна пишет класс Car и хочет задать цвет при создании. Какой метод в PHP отвечает за инициализацию объекта?

Подсказка: Этот метод запускается автоматически при вызове new .

Ответ:

- **__construct()**
- __destruct()
- init()
- start()

Шаг 02:

Миссия 2

Сергей создаёт класс FileLogger . Ему нужно освободить ресурсы (закрыть файл), когда объект перестаёт существовать. Какой метод использовать?

Подсказка: Этот метод вызывается автоматически при уничтожении объекта.

Ответ:

- __construct()
- **__destruct()**
- finalize()
- close()

Шаг 03:

Миссия 3

Игорь пишет класс User . В конструктор нужно передать имя и email. Как правильно объявить конструктор в PHP 8?

Подсказка: Используй ключевое слово public и двойное подчеркивание перед именем метода.

Ответ:

- **public function __construct(\$name, \$email) { ... }**
- function construct(\$name, \$email) { ... }
- public function init(\$name, \$email) { ... }
- __construct(\$name, \$email);

Шаг 04:

Миссия 4

Екатерина пишет класс Database . Ей нужно подключиться к базе при создании объекта. Где лучше разместить код подключения?

Подсказка: Вспомни метод, который выполняется при new .

Ответ:

- **В методе __construct()**
- В методе __destruct()
- В отдельной функции вне класса
- В глобальном пространстве

🔧 Шаг 05:

Миссия 5

У Ольги есть класс `Session`. Она хочет, чтобы при старте объект автоматически получал ID с помощью конструктора. Как правильно это реализовать?

Подсказка: В конструктор можно передавать параметры при создании объекта.

Ответ:

- Передавать ID в `__construct()` при создании объекта
- Создавать метод `setId()` и вызывать после конструктора
- Задавать ID через глобальную переменную
- ID всегда фиксированный, не нужен конструктор

🔧 Шаг 06:

Миссия 6

Сопоставьте методы с их назначением в ООП:

Подсказка: Один метод создаёт, другой завершает.

Ответ:

- `__construct()` — Вызывается при создании объекта
- `__destruct()` — Вызывается при уничтожении объекта
- `initData()` — Инициализирует дополнительные данные
- `saveLog()` — Сохраняет данные в журнал

🔧 Шаг 07:

Миссия 7

Артем работает с классом `Image`. Сопоставьте ситуацию с методом:

Подсказка: Подумай, когда нужно загружать файл, а когда удалять временные данные.

Ответ:

- Создание объекта с путем к файлу — `__construct()`
- Удаление временных файлов при завершении работы — `__destruct()`
- Изменение размера изображения — `resize()`
- Сохранение копии изображения — `save()`

🔧 Шаг 08:

Миссия 8

Дмитрий пишет систему логирования. Сопоставьте задачи с методами:

Подсказка: Логика жизни объекта начинается и заканчивается в разных местах.

Ответ:

- Создание подключения к файлу — `__construct()`
- Заккрытие ресурса — `__destruct()`
- Запись строки лога — `write()`
- Очистка файла лога — `clear()`

🔧 Шаг 09:

Миссия 9

Сопоставьте примеры с правильным описанием:

Подсказка: Помни, конструктор отвечает за старт, деструктор — за финал.

Ответ:

- **public function __construct(\$db)** — Подключение к базе данных при создании
- **public function __destruct()** — Закрытие соединения
- **function render()** — Вывод данных на экран
- **function update()** — Обновление информации

Шаг 10:

Миссия 10

Сопоставьте персонажей и их действия с методами:

Подсказка: Каждый персонаж действует в стиле конструктора или деструктора.

Ответ:

- **Программист открывает соединение с API** — __construct()
- **Программист завершает сессию** — __destruct()
- **Программист добавляет новые данные** — insert()
- **Программист проверяет статус** — checkStatus()

Шаг 11:

Миссия 11

Сопоставьте методы класса Game с их назначением:

Подсказка: Конструктор начинает игру, деструктор её завершает.

Ответ:

- **__construct()** — Запуск новой игры
- **__destruct()** — Вывод финального счёта
- **play()** — Игровой процесс
- **pause()** — Пауза игры

Шаг 12:

Миссия 12

Расставьте действия в правильном порядке при работе с классом FileHandler :

Подсказка: Начало — создание, конец — освобождение ресурсов.

Ответ:

- Создание объекта и открытие файла (__construct())
- Запись данных в файл (write())
- Закрытие файла (__destruct())

Шаг 13:

Миссия 13

Расположите этапы работы с базой данных:

Подсказка: Начинаем с подключения, завершаем освобождением ресурсов.

Ответ:

- Подключение к базе (__construct())
- Выполнение запросов
- Закрытие соединения (__destruct())

Шаг 14:

Миссия 14

В классе Game игрок выполняет разные действия. Расположите их в правильной последовательности:

Подсказка: Игра должна начинаться с конструктора, а завершаться деструктором.

Ответ:

- Создание игры (__construct())
- Игровой процесс (play())
- Финал игры (__destruct())

Шаг 15:

Миссия 15

Расположите шаги работы программы с сессией:

Подсказка: Сначала старт, потом действия, затем завершение.

Ответ:

- Запуск сессии (__construct())
- Запись данных в сессию
- Закрытие сессии (__destruct())

Шаг 16:

Миссия 16

Расположите этапы работы класса Cache :

Подсказка: Кэш должен стартовать, выполнять задачи и завершаться корректно.

Ответ:

- Создание объекта и загрузка данных (__construct())
- Использование кэша (get/set)
- Очистка и завершение (__destruct())

Модуль 2 - «Оружие мастера» — Инструменты продвинутого ООП

Урок 1 - Пространства имён — Мудрость разделения

Шаг 01:

Миссия 1

Аня пишет проект и решила подключить библиотеку через composer. Она заметила, что в коде появляется namespace . Для чего оно нужно?

Подсказка: Подумай: если два разных разработчика создадут класс User , что произойдет без разделения пространств?

Ответ:

- Чтобы избежать конфликтов имён классов
- Чтобы ускорить работу PHP
- Чтобы автоматически создавать таблицы в БД
- Чтобы уменьшить размер кода

Шаг 02:

Миссия 2

Вадим подключает библиотеку для работы с PDF, а у него уже есть свой класс Document . Как избежать конфликта имён?

Подсказка: Вспомни ключевое слово, которое помогает явно указать область принадлежности класса.

Ответ:

- **Использовать namespace и импортировать класс через use**
- Переименовать свой класс Document в Doc1
- Скопировать библиотеку в другой каталог
- Запустить PHP без пространств имён

Шаг 03:

Миссия 3

Саша пишет большой проект и хочет, чтобы структура директорий совпадала с пространствами имён. Какой стандарт используется?

Подсказка: Он принят в PHP-сообществе как соглашение для автозагрузки.

Ответ:

- **PSR-4**
- MVC
- SOLID
- PSR-12

Шаг 04:

Миссия 4

Марк случайно объявил два класса с одинаковым именем Report в одном пространстве имён. Что произойдёт?

Подсказка: Вспомни: PHP не любит дублирующиеся классы.

Ответ:

- **Ошибка: класс уже объявлен**
- Оба класса будут работать параллельно
- PHP выберет последний класс
- Код скомпилируется, но методы перепутаются

Шаг 05:

Миссия 5

Соотнеси ключевые слова с их назначением.

Подсказка: Вспомни: одно слово задаёт область, другое — импортирует, третье — задаёт псевдоним.

Ответ:

- **namespace** — Объявляет пространство имён в начале файла
- **use** — Импортирует класс или пространство для удобного обращения
- **as** — Создаёт псевдоним для импортируемого класса или пространства

Шаг 06:

Миссия 6

Соотнеси примеры классов с правильными пространствами имён.

Подсказка: Подумай о структуре: модели обычно в Models, сервисы в Services.

Ответ:

- **App\Models\User** — Класс пользователя в слое модели
- **App\Services\Mailer** — Класс для отправки писем
- **App\Controllers\HomeController** — Класс контроллера главной страницы

Шаг 07:

Миссия 7

Сопоставь ситуацию с правильным решением.

Подсказка: Используй разные техники: полный путь, use, псевдоним.

Ответ:

- **Два класса с именем User в разных библиотеках** — Использовать псевдоним через as
- **Нужно вызвать класс только один раз** — Указать полный путь к классу
- **Класс используется много раз** — Импортировать через use

Шаг 08:

Миссия 8

Соотнеси стандарт с его назначением.

Подсказка: Подумай про автозагрузку, форматирование и имена.

Ответ:

- **PSR-4** — Стандарт автозагрузки по пространствам имён
- **PSR-1** — Базовые правила кодирования
- **PSR-12** — Подробные правила стиля кода

Шаг 09:

Миссия 9

Сопоставь пространство имён с типичной ролью.

Подсказка: Controllers управляют логикой, Models — данными, Services — процессами.

Ответ:

- **App\Controllers** — Обработка HTTP-запросов
- **App\Models** — Работа с данными
- **App\Services** — Служебная бизнес-логика

Шаг 10:

Миссия 10

Сопоставь ошибку с её причиной.

Подсказка: Иногда проблема в конфликте имён, иногда — в неправильной структуре директорий.

Ответ:

- **Class not found** — Файл не соответствует PSR-4 структуре

- **Cannot redeclare class** — Два класса с одинаковым именем в пространстве
- **Unexpected namespace** — Неправильное объявление пространства в файле

🔧 Шаг 11:

Миссия 11

Расположи шаги подключения библиотеки с пространствами имён.

Подсказка: Сначала composer, затем autoload, потом использование.

Ответ:

- Установить библиотеку через composer
- Подключить autoload.php
- Импортировать класс через use
- Создать объект класса

🔧 Шаг 12:

Миссия 12

Расположи в порядке действий программиста, столкнувшегося с конфликтом имён.

Подсказка: Сначала анализ, потом импорт, затем псевдоним.

Ответ:

- Определить, какие классы конфликтуют
- Импортировать первый класс
- Импортировать второй класс с псевдонимом
- Использовать оба класса в коде

🔧 Шаг 13:

Миссия 13

Кирилл подключает несколько внешних библиотек через Composer. Он заметил, что автозагрузка работает медленно. Что можно сделать для ускорения?

Подсказка: Подумай про оптимизированную карту классов.

Ответ:

- **Запустить `composer dump-autoload -o`**
- Удалить vendor
- Использовать require для всех файлов
- Отключить PSR-4

🔧 Шаг 14:

Миссия 14

Сопоставьте тип автозагрузки Composer с его особенностью.

Подсказка: Разные режимы решают разные задачи.

Ответ:

- **PSR-4** — Пространства имён соответствуют папкам, автоматическое подключение классов.
- **classmap** — Создает полную карту соответствия классов и файлов.
- **files** — Подключает указанные файлы напрямую без пространства имён.
- **exclude-from-classmap** — Исключает выбранные директории из classmap.

Шаг 15:

Миссия 15

Расположите шаги подключения автозагрузчика в собственном фреймворке в правильном порядке.

Подсказка: Сначала пишем функцию, потом регистрируем её, потом используем классы.

Ответ:

- Написать функцию автозагрузки
- Регистрировать функцию через `spl_autoload_register`
- Создать необходимые директории и файлы классов
- Использовать классы в проекте

Шаг 16:

Миссия 16

Настройка автозагрузки через PSR-4 в Composer: расположите шаги в правильном порядке.

Подсказка: Сначала `composer.json`, потом команда, потом подключение файла.

Ответ:

- Редактировать `composer.json`, добавив секцию `autoload PSR-4`
- Запустить команду `composer dump-autoload`
- Подключить `vendor/autoload.php`
- Использовать классы в коде

Шаг 17:

Миссия 17

Виктор хочет, чтобы старый проект постепенно переходил на автозагрузку. Расположите шаги внедрения.

Подсказка: Начни с ключевых классов, потом подключай новые.

Ответ:

- Выбрать самые часто используемые классы и подключить их через автозагрузчик
- Добавить остальные классы по мере необходимости
- Удалять старые `include/require`
- Проверять работу проекта после каждого изменения

Урок 2 - Автозагрузка — Призыв классов из пустоты

Шаг 01:

Миссия 1

Игорь пишет проект на PHP 8 и устал подключать десятки файлов вручную. Он слышал про автозагрузку классов. Что делает автозагрузка?

Подсказка: Подумай: это механизм, который избавляет от постоянного `include/require`.

Ответ:

- **Автоматически подключает файл с классом при его первом использовании**
- Создает копии классов в памяти
- Удаляет неиспользуемые классы
- Заменяет функции на классы

Шаг 02:

Миссия 2

Марина работает над большим проектом и хочет включить автозагрузку через composer. Какой файл отвечает за настройку автозагрузки?

Подсказка: Этот файл хранит конфигурацию зависимостей и секцию autoload .

Ответ:

- **composer.json**
- autoload.php
- config.php
- settings.ini

Шаг 03:

Миссия 3

Сергей случайно написал класс UserProfile , но файл назвал profile.php . Автозагрузка по PSR-4 не сработала. Почему?

Подсказка: Вспомни: PSR-4 требует строгого соответствия имени класса и пути к файлу.

Ответ:

- **Имя файла должно соответствовать имени класса**
- PHP не поддерживает класс UserProfile
- Автозагрузка работает только с функциями
- Нужно вручную подключить файл

Шаг 04:

Миссия 4

Артем подключил автозагрузку Composer и написал require 'vendor/autoload.php'; . Что произойдет?

Подсказка: Этот файл создается Composer и активирует загрузку классов.

Ответ:

- **Подключится автозагрузчик Composer**
- Система сгенерирует новые классы
- Все классы будут закешированы в памяти
- Проект не запустится без PSR-0

Шаг 05:

Миссия 5

Вася использует spl_autoload_register . Что именно делает эта функция?

Подсказка: Она управляет функциями-обработчиками для автозагрузки.

Ответ:

- **Регистрирует пользовательскую функцию автозагрузки**
- Создает новый класс из строки
- Удаляет старые классы
- Запускает garbage collector

🔧 Шаг 06:

Миссия 6

Катя добавила секцию `autoload` в `composer.json`, но автозагрузка не сработала. Какое действие она забыла выполнить?

Подсказка: После изменений в `composer.json` нужно обновить автозагрузчик.

Ответ:

- **Запустить команду `composer dump-autoload`**
- Перезапустить Apache
- Удалить кэш браузера
- Заменить `vendor/autoload.php` вручную

🔧 Шаг 07:

Миссия 7

Оля читает документацию и хочет понять различия между стандартами автозагрузки.

Подсказка: У разных стандартов разные правила для путей и имён файлов.

Ответ:

- **PSR-0** — Требуется соответствия пространств имён и директорий с подчёркиваниями.
- **PSR-4** — Современный стандарт, где пространство имён напрямую соответствует папке.
- **`spl_autoload_register`** — Функция для ручной регистрации автозагрузчиков.
- **Composer autoload** — Автоматически управляет подключением классов через `composer.json`.

🔧 Шаг 08:

Миссия 8

Антон разбирается, как Composer подключает классы. Сопоставьте механизм с его описанием.

Подсказка: Composer поддерживает разные режимы: `psr-4`, `classmap`, `files`.

Ответ:

- **`psr-4`** — Задаёт пространство имён и соответствующую директорию.
- **`classmap`** — Создает карту соответствия имён классов и файлов.
- **`files`** — Просто подключает указанные файлы.
- **`exclude-from-classmap`** — Исключает папки из карты классов.

🔧 Шаг 09:

Миссия 9

Света добавила в проект несколько автозагрузчиков. Что произойдет?

Подсказка: `spl_autoload_register` позволяет вызывать несколько функций.

Ответ:

- **Первый автозагрузчик** — Вызывается первым, если класс не найден — передаёт управление дальше.
- **Второй автозагрузчик** — Срабатывает, если первый не смог подключить класс.
- **Все автозагрузчики** — Вызываются по очереди до тех пор, пока класс не найден.
- **Ни один не сработал** — PHP выдаёт ошибку `Class not found`.

Шаг 10:

Миссия 10

Егор экспериментирует с Composer и заметил разные файлы в `vendor/composer`. Сопоставь файл с его назначением.

Подсказка: Вспомни: там есть `autoload_real.php`, `autoload_static.php` и др.

Ответ:

- **`autoload.php`** — Главный файл, подключаемый в проекте.
- **`autoload_real.php`** — Файл, который запускает логику подключения автозагрузчика.
- **`autoload_static.php`** — Хранит статическую карту классов для ускорения.
- **`ClassLoader.php`** — Реализует сам класс автозагрузчика Composer.

Шаг 11:

Миссия 11

Тимур подключает старый код и хочет понять разницу между `require` и автозагрузкой.

Подсказка: Вспомни: `require` подключает всё сразу, автозагрузка — по требованию.

Ответ:

- **`require`** — Подключает файл сразу, даже если класс не используется.
- **Автозагрузка** — Подключает файл только при первом обращении к классу.
- **Недостаток `require`** — Подключает лишние файлы и замедляет старт.
- **Преимущество автозагрузки** — Экономит ресурсы и ускоряет выполнение.

Шаг 12:

Миссия 12

Алексей использует namespaces и хочет понять, как они связаны с автозагрузкой.

Подсказка: В PSR-4 пространство имён соответствует папке.

Ответ:

- **Пространство имён `App\Controllers`** — Соответствует папке `src/Controllers`.
- **Пространство имён `App\Models`** — Соответствует папке `src/Models`.
- **Пространство имён `App\Services`** — Соответствует папке `src/Services`.
- **Пространство имён `App\Helpers`** — Соответствует папке `src/Helpers`.

Шаг 13:

Миссия 13

Дима хочет вручную реализовать простой автозагрузчик. Расположи шаги в правильном порядке.

Подсказка: Сначала пишем функцию, потом регистрируем её.

Ответ:

- Написать функцию, которая принимает имя класса
- Сформировать путь к файлу на основе имени класса
- Подключить файл через `require_once`
- Зарегистрировать функцию через `spl_autoload_register`

Шаг 14:

Миссия 14

Ирина настроила PSR-4. Расположи шаги настройки в правильном порядке.

Подсказка: Сначала редактируется composer.json, потом запускается команда.

Ответ:

- Открыть composer.json
- Добавить секцию autoload с psr-4
- Запустить composer dump-autoload
- Подключить vendor/autoload.php в проект

Шаг 15:

Миссия 15

Федя экспериментирует с несколькими файлами автозагрузки. Расположи события по порядку.

Подсказка: Вспомни: PHP вызывает автозагрузчики последовательно.

Ответ:

- PHP ищет класс в первом автозагрузчике
- **Если не найден** — передает управление второму
- **Если ни один не сработал** — ошибка Class not found
- При успешной загрузке выполнение продолжается

Шаг 16:

Миссия 16

Лена хочет понять, как работает оптимизированный автозагрузчик Composer. Расположи шаги.

Подсказка: Оптимизация использует статическую карту классов.

Ответ:

- Composer генерирует autoload_static.php
- Создается статическая карта классов
- autoload_real.php подключает статическую карту
- Классы подключаются напрямую из карты

Урок 3 - Трейты — Наследие без наследования

Шаг 01:

Миссия 1

Аня создает класс, который должен использовать методы двух разных классов. Как лучше всего объединить функциональность без множественного наследования?

Подсказка: В PHP 8 есть способ использовать готовые куски кода.

Ответ:

- **Использовать трейты**
- Множественное наследование
- Создать интерфейс
- Копировать методы вручную

Шаг 02:

Миссия 2

Иван использует трейт Logger в нескольких классах. Как правильно подключить трейт?

Подсказка: Подключение происходит внутри класса, не в глобальной области.

Ответ:

- **Через ключевое слово use внутри класса**
- Через include вне класса
- Через extends
- Через require_once внутри метода

Шаг 03:

Миссия 3

Мария использует два трейта с одинаковым методом. Как PHP позволит решить конфликт?

Подсказка: Конфликт можно разрешить прямо в классе.

Ответ:

- **Использовать insteadof и as для разрешения конфликта**
- Переименовать класс
- Удалить один из трейтов
- PHP выдаст фатальную ошибку и исправить нельзя

Шаг 04:

Миссия 4

 Задача: Сопоставьте трейт с его особенностью.

Подсказка: Обратите внимание на то, как трейт влияет на класс.

Ответ:

- **Трейт** — Повторно используемый набор методов для разных классов
- **Метод insteadof** — Позволяет выбрать, какой метод использовать при конфликте
- **Метод as** — Позволяет переименовать метод трейта в классе
- **Интерфейс** — Определяет набор методов, которые класс обязан реализовать
- **Множественное наследование** — В PHP невозможно использовать, заменяется трейтами

Шаг 05:

Миссия 5

Сергей хочет, чтобы трейт использовался только в одном конкретном классе. Как ограничить использование трейта?

Подсказка: В PHP есть возможность проверять контекст использования.

Ответ:

- **Использовать метод self::class внутри трейта для проверки класса**
- Трейт нельзя ограничить, он всегда глобальный
- Создать отдельный namespace для трейта
- Подключать трейт через require_once

Шаг 06:

Миссия 6

Наталья использует трейт с приватным методом. Как класс может использовать этот метод?

Подсказка: Приватные методы трейта видны только внутри класса.

Ответ:

- **Метод доступен внутри класса, использующего трейт**
- Метод доступен во всех классах с трейтом
- Метод становится публичным автоматически
- Приватные методы нельзя использовать в трейтах

 **Шаг 07:**

Миссия 7

Алексей хочет использовать два трейта с одинаковым методом greet. Как он может сохранить оба метода?

Подсказка: Можно переименовать один метод при подключении.

Ответ:

- **Использовать as для переименования одного метода**
- Удалить один из трейтов
- Использовать множественное наследование
- PHP автоматически объединяет методы

 **Шаг 08:**

Миссия 8

Сопоставьте понятие трейта с его возможностью.

Подсказка: Подумайте про гибкость использования кода.

Ответ:

- **Трейт** — Можно использовать в нескольких классах без наследования
- **Метод private** — Доступен только внутри класса
- **Метод public** — Доступен во всех местах, где виден объект
- **Метод protected** — Доступен в классе и наследниках
- **insteadof** — Выбирает метод при конфликте трейтов

 **Шаг 09:**

Миссия 9

Юлия подключила трейт, но метод не работает как ожидалось. Что нужно проверить в первую очередь?

Подсказка: Трейты подключаются внутри класса, порядок имеет значение.

Ответ:

- **Проверить, подключен ли трейт через use внутри класса**
- Проверить namespace файла трейта
- Перезапустить сервер
- Сделать метод статическим

 **Шаг 10:**

Миссия 10

Сопоставьте методы трейтов с их уровнем доступа.

Подсказка: Доступ влияет на то, кто может использовать метод.

Ответ:

- **public метод** — Доступен везде
- **protected метод** — Доступен в классе и наследниках
- **private метод** — Доступен только внутри класса
- **insteadof** — Выбирает один метод при конфликте трейтов
- **as** — Переименовывает метод трейта

 Шаг 11:

Миссия 11

Расположите шаги правильного подключения трейта с конфликтующими методами.

Подсказка: Сначала подключаем трейты, потом разрешаем конфликты.

Ответ:

- Подключить оба трейта через use
- Использовать instanceof для выбора метода
- Использовать as для переименования метода при необходимости
- Проверить работу класса с методами

 Шаг 12:

Миссия 12

Расположите шаги внедрения трейта в новый класс проекта, чтобы минимизировать конфликты методов.

Подсказка: Сначала подключаем, потом разрешаем конфликты, затем тестируем.

Ответ:

- Создать класс, куда будет внедряться трейт
- Подключить трейт через use
- Разрешить конфликты с помощью instanceof или as
- Проверить работу класса с новыми методами

 Шаг 13:

Миссия 13

Расположите этапы создания универсального трейта для логирования действий в разных классах.

Подсказка: Подумайте о структуре и доступности методов.

Ответ:

- Определить методы логирования
- Сделать методы public для всех классов
- Создать трейт и подключить методы
- Использовать трейт в нужных классах

 Шаг 14:

Миссия 14

Расположите шаги для безопасного использования нескольких трейтов с одинаковыми методами в одном классе.

Подсказка: Сначала подключаем трейты, затем решаем конфликты, тестируем работу методов.

Ответ:

- Подключить все трейты через use
- Использовать insteadof для выбора метода
- Использовать as для переименования методов
- Проверить корректность работы класса

🔮 **Шаг 15:**

Миссия 15

Расположите шаги рефакторинга класса, чтобы вынести повторяющийся код в трейт.

Подсказка: Сначала выявите повторяющиеся методы, потом создайте трейт и подключите его.

Ответ:

- Проанализировать класс на повторяющиеся методы
- Создать трейт с повторяющимся функционалом
- Подключить трейт к классу через use
- Удалить дублирующиеся методы из класса и протестировать

Урок 4 - Магические методы Тёмное искусство

🔮 **Шаг 01:**

Миссия 1

Какой магический метод автоматически вызывается при создании объекта класса?

Подсказка: Подумайте, какой метод инициализирует объект при new.

Ответ:

- **__construct()**
- __destruct()
- __call()
- __invoke()

🔮 **Шаг 02:**

Миссия 2

Какой метод вызывается при попытке вывести объект через echo или print?

Подсказка: Этот метод преобразует объект в строку.

Ответ:

- **__toString()**
- __get()
- __call()
- __isset()

🔮 **Шаг 03:**

Миссия 3

Какой магический метод позволяет перехватывать вызовы несуществующих методов?

Подсказка: Он позволяет динамически обрабатывать любые вызовы.

Ответ:

- **__call()**

- `__invoke()`
- `__construct()`
- `__clone()`

🔮 Шаг 04:

Миссия 4

Какой метод вызывается при попытке вызвать объект как функцию?

Подсказка: Подумайте, что нужно реализовать, чтобы объект вел себя как callable.

Ответ:

- **`__invoke()`**
- `__call()`
- `__toString()`
- `__construct()`

🔮 Шаг 05:

Миссия 5

Какой магический метод вызывается при клонировании объекта через `clone`?

Подсказка: Этот метод позволяет настроить процесс копирования объекта.

Ответ:

- **`__clone()`**
- `__construct()`
- `__call()`
- `__destruct()`

🔮 Шаг 06:

Миссия 6

Сопоставьте магический метод с его назначением.

Ответ:

- `__construct()`
- `__destruct()`
- `__toString()`
- `__call()`
- `__invoke()`

1. Вызывается при удалении объекта
2. Вызывается при создании объекта
3. Преобразует объект в строку
4. Перехватывает вызовы несуществующих методов
5. Позволяет вызывать объект как функцию

Подсказка: Подумайте о жизненном цикле объекта и его поведении.

- **`__construct()`** — Вызывается при создании объекта
- **`__destruct()`** — Вызывается при удалении объекта
- **`__toString()`** — Преобразует объект в строку
- **`__call()`** — Перехватывает вызовы несуществующих методов
- **`__invoke()`** — Позволяет вызывать объект как функцию

Шаг 07:

Миссия 7

Сопоставьте магический метод с ситуацией его использования.

Ответ:

- `__clone()`
 - `__get()`
 - `__set()`
 - `__isset()`
 - `__unset()`
1. Перехватывает попытку доступа к несуществующему свойству
 2. Перехватывает присвоение значения несуществующему свойству
 3. Позволяет проверить существование свойства через `isset()`
 4. Позволяет удалить свойство через `unset()`
 5. Вызывается при клонировании объекта

Подсказка: Подумайте, как объект реагирует на работу с его свойствами.

- **`__clone()`** — Вызывается при клонировании объекта
- **`__get()`** — Перехватывает попытку доступа к несуществующему свойству
- **`__set()`** — Перехватывает присвоение значения несуществующему свойству
- **`__isset()`** — Позволяет проверить существование свойства через `isset()`
- **`__unset()`** — Позволяет удалить свойство через `unset()`

Шаг 08:

Миссия 8

Сопоставьте магический метод с его особенностью.

Ответ:

- `__sleep()`
 - `__wakeup()`
 - `__serialize()`
 - `__unserialize()`
 - `__debugInfo()`
1. Вызывается перед сериализацией объекта
 2. Вызывается при десериализации объекта
 3. Позволяет задать собственный формат сериализации
 4. Позволяет задать, как объект восстанавливается из сериализованной строки
 5. Позволяет задать информацию для `var_dump()`

Подсказка: Подумайте о сохранении состояния объекта и отладке.

- **`__sleep()`** — Вызывается перед сериализацией объекта
- **`__wakeup()`** — Вызывается при десериализации объекта
- **`__serialize()`** — Позволяет задать собственный формат сериализации
- **`__unserialize()`** — Позволяет задать, как объект восстанавливается из сериализованной строки
- **`__debugInfo()`** — Позволяет задать информацию для `var_dump()`

Шаг 09:

Миссия 9

Расположите действия при клонировании объекта с магическим методом `__clone`.

1. Создание нового объекта копией исходного
2. Вызов метода `__clone`, если он определен
3. Копирование свойств объекта в новый объект
4. Возврат нового объекта

Подсказка: Подумайте о последовательности: копирование → кастомизация → возвращение объекта.

Ответ:

- Создание нового объекта копией исходного
- Копирование свойств объекта в новый объект
- Вызов метода `__clone`, если он определен
- Возврат нового объекта

Шаг 10:

Миссия 10

Расположите шаги работы магических методов `__get` и `__set` при обращении к несуществующему свойству.

1. Проверка существования свойства
2. Вызов `__get()` при чтении или `__set()` при записи
3. Выполнение логики внутри метода
4. Возврат значения (для `__get`) или сохранение значения (для `__set`)

Подсказка: Сначала проверка, потом перехват, затем действие.

Ответ:

- Проверка существования свойства
- Вызов `__get()` при чтении или `__set()` при записи
- Выполнение логики внутри метода
- Возврат значения (для `__get`) или сохранение значения (для `__set`)

Шаг 11:

Миссия 11

Расположите последовательность вызовов при сериализации объекта с `__sleep` и `__wakeup`.

1. Вызывается `__sleep()` перед сериализацией
2. Сериализация выбранных свойств
3. Сохранение сериализованной строки
4. Десериализация строки
5. Вызов `__wakeup()` после десериализации

Подсказка: Подумайте, что делается до и после хранения объекта.

Ответ:

- Вызывается `__sleep()` перед сериализацией
- Сериализация выбранных свойств
- Сохранение сериализованной строки

- Десериализация строки
- Вызов `__wakeup()` после десериализации

🔮 Шаг 12:

Миссия 12

Расположите шаги применения магического метода `__invoke` для объекта-функции.

1. Создание объекта с методом `__invoke`
2. Вызов объекта как функции
3. Выполнение логики внутри `__invoke`
4. Возврат результата

Подсказка: Подумайте о том, как объект превращается в callable.

Ответ:

- Создание объекта с методом `__invoke`
- Вызов объекта как функции
- Выполнение логики внутри `__invoke`
- Возврат результата

🔮 Шаг 13:

Миссия 13

Расположите шаги работы `__debugInfo` для отладки объекта.

1. Вызов `var_dump` или `print_r` на объекте
2. Автоматический вызов `__debugInfo()`, если он определен
3. Получение массива с информацией для отладки
4. Вывод информации на экран

Подсказка: Подумайте о последовательности: запрос информации → предоставление данных → отображение.

Ответ:

- Вызов `var_dump` или `print_r` на объекте
- Автоматический вызов `__debugInfo()`, если он определен
- Получение массива с информацией для отладки
- Вывод информации на экран

🔮 Шаг 14:

Миссия 14

Расположите шаги последовательности работы методов `__serialize` и `__unserialize`.

1. Вызов `__serialize()` перед сериализацией
2. Сериализация возвращенного массива
3. Сохранение сериализованной строки
4. Десериализация строки
5. Вызов `__unserialize()` с восстановленным массивом

Подсказка: Подумайте о парах методов: сериализация → сохранение → десериализация → восстановление.

Ответ:

- Вызов `__serialize()` перед сериализацией
- Сериализация возвращенного массива
- Сохранение сериализованной строки
- Десериализация строки
- Вызов `__unserialize()` с восстановленным массивом

 Шаг 15:

Миссия 15

Расположите действия при клонировании объекта с `__clone` и `__construct`.

1. Вызов `__construct()` при создании исходного объекта
2. Использование `clone` для копирования объекта
3. Автоматический вызов `__clone()` для кастомизации
4. Создание нового объекта-копии с нужными свойствами

Подсказка: Сначала объект создается, потом клонируется с дополнительной логикой.

Ответ:

- Вызов `__construct()` при создании исходного объекта
- Использование `clone` для копирования объекта
- Автоматический вызов `__clone()` для кастомизации
- Создание нового объекта-копии с нужными свойствами

 Шаг 16:

Миссия 16

Расположите шаги работы `__invoke` вместе с `__call` для динамического поведения объекта.

1. Попытка вызвать объект как функцию
2. Вызов `__invoke()`, если метод существует
3. Если метода нет, срабатывает `__call()` для перехвата
4. Выполнение логики и возврат результата

Подсказка: Подумайте о приоритете: сначала объект вызывается как функция, потом динамический перехват методов.

Ответ:

- Попытка вызвать объект как функцию
- Вызов `__invoke()`, если метод существует
- Если метода нет, срабатывает `__call()` для перехвата
- Выполнение логики и возврат результата

 Урок 5 - Статические заклинания и их ловушки

 Шаг 01:

Миссия 1

Что происходит при вызове статического метода класса без создания объекта?

Подсказка: Статические методы не требуют экземпляра класса.

Ответ:

- **Метод выполняется напрямую через класс**

- Создается новый объект автоматически
- Метод не может быть вызван без объекта
- Вызывается конструктор класса

Шаг 02:

Миссия 2

Какой из вариантов правильно определяет статическое свойство в классе PHP?

Подсказка: Обратите внимание на ключевое слово для свойства, доступного без экземпляра.

Ответ:

- **public static \$counter = 0;**
- public \$counter = 0;
- static counter = 0;
- counter = 0;

Шаг 03:

Миссия 3

Что важно помнить при использовании self:: внутри статического метода?

Подсказка: self:: ссылается на текущий класс, а не на экземпляр.

Ответ:

- **Используется для доступа к статическим свойствам/методам текущего класса**
- Используется для вызова методов экземпляра
- Всегда создает новый объект класса
- Заменяет \$this внутри метода

Шаг 04:

Миссия 4

 Сопоставьте статические элементы с их особенностями.

Элементы:

Ответ:

- Статический метод
- Статическое свойство
- self::
- static::

Определения:

1. Метод, который можно вызвать без создания объекта класса
2. Свойство, общее для всех экземпляров класса
3. Ссылка на текущий класс при вызове внутри метода
4. Позволяет позднее статическое связывание

Подсказка: Обратите внимание на различия между self и static в наследовании.

- **Статический метод** — Метод, который можно вызвать без создания объекта класса
- **Статическое свойство** — Свойство, общее для всех экземпляров класса
- **self::** — Ссылка на текущий класс при вызове внутри метода
- **static::** — Позволяет позднее статическое связывание

🔦 Шаг 05:

Миссия 5

📖 Сопоставьте примеры с описанием их поведения.

Элементы:

Ответ:

- `self::$counter`
- `static::$counter`
- Обращение через объект -> `метод()`
- Обращение через класс `::метод()`

Определения:

1. Обращается к свойству текущего класса без учёта наследников
2. Позволяет учитывать наследуемые классы
3. Вызывает нестатический метод экземпляра объекта
4. Вызывает статический метод класса

Подсказка: Вспомните разницу между `self` и `static` и доступ к методам через объект/класс.

- **`self::$counter`** — Обращается к свойству текущего класса без учёта наследников
- **`static::$counter`** — Позволяет учитывать наследуемые классы
- **Обращение через объект -> `метод()`** — Вызывает нестатический метод экземпляра объекта
- **Обращение через класс `::метод()`** — Вызывает статический метод класса

🔦 Шаг 06:

Миссия 6

📖 Сопоставьте возможные ловушки статических методов.

Элементы:

Ответ:

- Статическое свойство меняется в наследниках
- Попытка использовать `$this` внутри статического метода
- Неправильный вызов через объект
- Конфликты при позднем статическом связывании

Определения:

1. `static::$property` учитывает наследование, `self::$property` нет
2. `$this` недоступен, ошибка выполнения
3. Можно вызвать метод, но это не соответствует правилам PHP
4. Методы с `static::` могут вести себя иначе, чем `self::` в наследниках

Подсказка: Статические методы и свойства имеют свои особенности при наследовании.

- **Статическое свойство меняется в наследниках** — `static::$property` учитывает наследование, `self::$property` нет
- **Попытка использовать `$this` внутри статического метода** — `$this` недоступен, ошибка выполнения
- **Неправильный вызов через объект** — Можно вызвать метод, но это не соответствует правилам PHP

- **Конфликты при позднем статическом связывании** — Методы с `static::` могут вести себя иначе, чем `self::` в наследниках

Шаг 07:

Миссия 7

Расположите шаги работы статического метода, который изменяет счетчик вызовов:

1. Метод вызывается через `ClassName::method()`
2. Доступ к `static::$counter`
3. Увеличение счетчика на 1
4. Возврат текущего значения счетчика

Подсказка: Подумайте, что происходит сначала при вызове метода, а что — в конце.

Ответ:

- Метод вызывается через `ClassName::method()`
- Доступ к `static::$counter`
- Увеличение счетчика на 1
- Возврат текущего значения счетчика

Шаг 08:

Миссия 8

 Сопоставьте ошибки при статическом доступе с их последствиями.

Элементы:

Ответ:

- Использование `$this` внутри статического метода
- Обращение к нестатическому свойству через `ClassName::`
- Неправильное наследование статического метода
- Переопределение статического свойства без `static`

Определения:

1. Ошибка времени выполнения, `$this` недоступен
2. Ошибка, свойство недоступно без объекта
3. Метод не учитывает позднее статическое связывание
4. Свойство не наследуется корректно в дочерних классах

Подсказка: Статические методы имеют свои ограничения при наследовании и обращении к нестатическим элементам.

- **Использование `$this` внутри статического метода** — Ошибка времени выполнения, `$this` недоступен
- **Обращение к нестатическому свойству через `ClassName::`** — Ошибка, свойство недоступно без объекта
- **Неправильное наследование статического метода** — Метод не учитывает позднее статическое связывание
- **Переопределение статического свойства без `static`** — Свойство не наследуется корректно в дочерних классах

Шаг 09:

Миссия 9

 Сопоставьте ситуации с правильной практикой использования статических методов.

Элементы:

Ответ:

- Утилитарный метод без состояния
- Счетчик вызовов всех экземпляров
- Фабричный метод для создания объектов
- Метод с зависимостью от `$this`

Определения:

1. Можно сделать статическим, не требуя объекта
2. Используем `static::$counter` для корректного наследования
3. Статический метод подходит для создания экземпляров без состояния объекта
4. Нельзя делать статическим, так как нужен объект

Подсказка: Подумайте, где можно использовать метод без привязки к конкретному объекту.

- **Утилитарный метод без состояния** — Можно сделать статическим, не требуя объекта
- **Счетчик вызовов всех экземпляров** — Используем `static::$counter` для корректного наследования
- **Фабричный метод для создания объектов** — Статический метод подходит для создания экземпляров без состояния объекта
- **Метод с зависимостью от `$this`** — Нельзя делать статическим, так как нужен объект

Шаг 10:

Миссия 10

 Сопоставьте ключевые слова с их назначением в статическом контексте.

Элементы:

Ответ:

- `static`
- `self`
- `public static`
- `private static`

Определения:

1. Позволяет позднее статическое связывание
2. Ссылка на текущий класс
3. Статическое свойство или метод доступно из любого места
4. Статическое свойство или метод доступно только внутри класса

Подсказка: Разница между `self` и `static` важна для наследования.

- **`static`** — Позволяет позднее статическое связывание
- **`self`** — Ссылка на текущий класс
- **`public static`** — Статическое свойство или метод доступно из любого места
- **`private static`** — Статическое свойство или метод доступно только внутри класса

Шаг 11:

Миссия 11

Расположите шаги использования статического метода-счетчика:

1. Определение свойства static \$counter
2. Вызов метода ClassName::method()
3. Увеличение значения счетчика
4. Возврат текущего значения

Подсказка: Подумайте, что нужно сделать перед использованием метода и после изменения счетчика.

Ответ:

- Определение свойства static \$counter
- Вызов метода ClassName::method()
- Увеличение значения счетчика
- Возврат текущего значения

Шаг 12:

Миссия 12

Расположите шаги вызова статического метода для фабрики объектов:

1. Определение статического метода create()
2. Вызов ClassName::create()
3. Создание нового экземпляра объекта внутри метода
4. Возврат объекта

Подсказка: Сначала создаем метод, потом объект через него.

Ответ:

- Определение статического метода create()
- Вызов ClassName::create()
- Создание нового экземпляра объекта внутри метода
- Возврат объекта

Шаг 13:

Миссия 13

Расположите этапы при наследовании статических свойств:

1. Определение свойства в родительском классе
2. Использование self::\$property в родителе
3. Создание дочернего класса
4. Обращение через static::\$property для учета наследников

Подсказка: Статические свойства ведут себя по-разному при self и static.

Ответ:

- Определение свойства в родительском классе
- Использование self::\$property в родителе
- Создание дочернего класса
- Обращение через static::\$property для учета наследников

Шаг 14:

Миссия 14

Расположите шаги безопасного изменения статического свойства:

1. Определить `public static $config`
2. Проверка, что свойство не `null`
3. Изменение значения через класс `ClassName::$config`
4. Использование свойства в методах класса

Подсказка: Следите за корректным доступом и изменением свойства.

Ответ:

- Определить `public static $config`
- Проверка, что свойство не `null`
- Изменение значения через класс `ClassName::$config`
- Использование свойства в методах класса

Шаг 15:

Миссия 15

Расположите шаги вызова статического метода с наследованием:

1. Определение метода в родителе
2. Создание дочернего класса
3. Вызов метода через `static::method()` в дочернем классе
4. Получение результата с учетом наследника

Подсказка: `static::` учитывает позднее статическое связывание.

Ответ:

- Определение метода в родителе
- Создание дочернего класса
- Вызов метода через `static::method()` в дочернем классе
- Получение результата с учетом наследника

Шаг 16:

Миссия 16

Расположите действия при использовании статических методов для логирования:

1. Создание класса `Logger` с `public static log()`
2. Вызов `Logger::log('message')`
3. Добавление сообщения в массив сообщений
4. Использование массива сообщений для отчета

Подсказка: Статический метод позволяет вести учет сообщений без создания объектов.

Ответ:

- Создание класса `Logger` с `public static log()`
- Вызов `Logger::log('message')`
- Добавление сообщения в массив сообщений
- Использование массива сообщений для отчета

Урок 6 - Финальные классы и методы — Последняя черта

Шаг 01:

Миссия 1

Что происходит, если попытаться наследовать от класса, объявленного как final?

Подсказка: Подумайте о том, зачем используют final для классов.

Ответ:

- **Возникает ошибка времени выполнения**
- Класс наследуется как обычно
- Класс становится абстрактным
- PHP игнорирует final и наследует класс

Шаг 02:

Миссия 2

Что происходит при попытке переопределить final метод в дочернем классе?

Подсказка: Final методы защищены от изменения поведения в наследниках.

Ответ:

- **Возникает ошибка времени выполнения**
- Метод успешно переопределяется
- Метод становится статическим
- Метод автоматически удаляется из дочернего класса

Шаг 03:

Миссия 3

Зачем разработчики используют final методы?

Подсказка: Подумайте о контроле поведения методов в сложных системах.

Ответ:

- **Чтобы запретить изменение метода в наследниках**
- Чтобы метод стал абстрактным
- Чтобы метод нельзя было вызывать из объекта
- Чтобы метод автоматически стал статическим

Шаг 04:

Миссия 4

 Сопоставьте тип ограничения с его эффектом.

Элементы:

Ответ:

- final класс
- final метод
- abstract класс
- abstract метод

Определения:

1. Нельзя наследовать этот класс

2. Нельзя переопределять этот метод в наследниках
3. Нельзя создавать экземпляр напрямую
4. Метод должен быть реализован в наследнике

Подсказка: Final и abstract — противоположные ограничения.

- **final класс** — Нельзя наследовать этот класс
- **final метод** — Нельзя переопределять этот метод в наследниках
- **abstract класс** — Нельзя создавать экземпляр напрямую
- **abstract метод** — Метод должен быть реализован в наследнике

Шаг 05:

Миссия 5

 Сопоставьте ситуацию с правильной практикой использования final методов и классов.

Элементы:

Ответ:

- Библиотечный класс для утилит
- Метод, обеспечивающий критическую бизнес-логику
- Шаблонный класс, который должны расширять
- Метод, который должен быть реализован дочерними классами

Определения:

1. Можно объявить final, чтобы запретить наследование
2. Можно объявить final, чтобы запретить переопределение
3. Не делаем final, чтобы можно было расширять
4. Не делаем final, оставляем abstract

Подсказка: Final предотвращает изменение, а abstract требует реализации.

- **Библиотечный класс для утилит** — Можно объявить final, чтобы запретить наследование
- **Метод, обеспечивающий критическую бизнес-логику** — Можно объявить final, чтобы запретить переопределение
- **Шаблонный класс, который должны расширять** — Не делаем final, чтобы можно было расширять
- **Метод, который должен быть реализован дочерними классами** — Не делаем final, оставляем abstract

Шаг 06:

Миссия 6

Расположите шаги правильного использования final класса:

1. Определить класс как final
2. Добавить методы и свойства
3. Использовать класс без наследования
4. Вызвать методы через объект класса

Подсказка: Final предотвращает наследование, но позволяет использование объектов.

Ответ:

- Определить класс как final
- Добавить методы и свойства

- Использовать класс без наследования
- Вызвать методы через объект класса

Шаг 07:

Миссия 7

Расположите шаги правильного использования final метода:

1. Определить метод как final
2. Реализовать логику метода
3. Вызвать метод в дочернем классе (без переопределения)
4. Получить результат выполнения

Подсказка: Final метод нельзя переопределять, но можно вызывать.

Ответ:

- Определить метод как final
- Реализовать логику метода
- Вызвать метод в дочернем классе (без переопределения)
- Получить результат выполнения

Шаг 08:

Миссия 8

Расположите шаги при работе с final и abstract вместе:

1. Создать abstract класс
2. Добавить final методы, которые не должны переопределяться
3. Добавить abstract методы, которые должны быть реализованы
4. Создать наследника и реализовать abstract методы
5. Использовать final методы без изменений

Подсказка: Final методы защищают логику, abstract требует реализации.

Ответ:

- Создать abstract класс
- Добавить final методы, которые не должны переопределяться
- Добавить abstract методы, которые должны быть реализованы
- Создать наследника и реализовать abstract методы
- Использовать final методы без изменений

Шаг 09:

Миссия 9

Расположите шаги использования final класса в библиотеке:

1. Объявить класс final
2. Добавить публичные методы и свойства
3. Использовать класс в других частях проекта без наследования
4. Вызывать методы и получать результат

Подсказка: Final класс защищает целостность, но позволяет использовать объекты.

Ответ:

- Объявить класс final

- Добавить публичные методы и свойства
- Использовать класс в других частях проекта без наследования
- Вызывать методы и получать результат

Шаг 10:

Миссия 10

Расположите шаги правильного использования final метода с бизнес-логикой:

1. Определить метод как final
2. Добавить критическую логику
3. Вызвать метод из объекта дочернего класса (без переопределения)
4. Получить корректный результат

Подсказка: Final метод нельзя переопределять, но можно безопасно использовать.

Ответ:

- Определить метод как final
- Добавить критическую логику
- Вызвать метод из объекта дочернего класса (без переопределения)
- Получить корректный результат

Шаг 11:

Миссия 11

Расположите шаги при работе с final и static методами:

1. Объявить метод static и final
2. Реализовать его логику
3. Вызывать метод через имя класса
4. Получить результат выполнения

Подсказка: Статический final метод нельзя переопределять и не требует объекта.

Ответ:

- Объявить метод static и final
- Реализовать его логику
- Вызывать метод через имя класса
- Получить результат выполнения

Шаг 12:

Миссия 12

 Сопоставьте описание ситуации с правильным применением final:

Ответ:

- Класс утилит
- Метод вычисления налога
- Шаблонный класс для расширения
- Метод, требующий реализации в наследнике

1. Класс объявляется final
2. Метод объявляется final
3. Не объявляется final

4. Не объявляется final, оставляется abstract

Подсказка: Final защищает, abstract требует реализации.

- **Класс утилит** — Класс объявляется final
- **Метод вычисления налога** — Метод объявляется final
- **Шаблонный класс для расширения** — Не объявляется final
- **Метод, требующий реализации в наследнике** — Не объявляется final, оставляется abstract

Шаг 13:

Миссия 13

 Сопоставьте ограничения с их эффектами:

Ответ:

- final класс
 - final метод
 - abstract класс
 - abstract метод
1. Нельзя наследовать
 2. Нельзя переопределять
 3. Нельзя создать экземпляр напрямую
 4. Обязательная реализация в наследнике

Подсказка: Помните разницу между защитой от изменения и требованием реализации.

- **final класс** — Нельзя наследовать
- **final метод** — Нельзя переопределять
- **abstract класс** — Нельзя создать экземпляр напрямую
- **abstract метод** — Обязательная реализация в наследнике

Шаг 14:

Миссия 14

Расположите шаги при объединении final и abstract методов:

1. Создать abstract класс
2. Добавить final метод с фиксированной логикой
3. Добавить abstract метод для реализации в наследниках
4. Создать наследника
5. Реализовать abstract метод в наследнике
6. Вызвать final метод

Подсказка: Final метод защищает, abstract требует реализации.

Ответ:

- Создать abstract класс
- Добавить final метод с фиксированной логикой
- Добавить abstract метод для реализации в наследниках
- Создать наследника
- Реализовать abstract метод в наследнике
- Вызвать final метод

Шаг 15:

Миссия 15

Расположите шаги безопасного использования final методов в проекте:

1. Объявить метод final
2. Реализовать логику
3. Вызвать метод через объект
4. Убедиться, что метод нельзя переопределить в наследниках

Подсказка: Final защищает поведение метода и предотвращает изменения в наследниках.

Ответ:

- Объявить метод final
- Реализовать логику
- Вызвать метод через объект
- Убедиться, что метод нельзя переопределить в наследниках

Шаг 16:

Миссия 16

Расположите шаги работы с final классом и методами в реальном проекте:

1. Создать final класс
2. Добавить final методы с критической логикой
3. Использовать объект класса в разных модулях
4. Вызвать методы и проверить результат
5. Убедиться, что никто не изменил методы через наследование

Подсказка: Final защищает как целый класс, так и его ключевые методы.

Ответ:

- Создать final класс
- Добавить final методы с критической логикой
- Использовать объект класса в разных модулях
- Вызвать методы и проверить результат
- Убедиться, что никто не изменил методы через наследование

Модуль 3 - «Испытание верности» — Архитектура и шаблоны

Урок 1 - Паттерн «Стратегия» Путь выбора

Шаг 01:

Миссия 1

Алиса разрабатывает платежный модуль и хочет менять стратегию расчета скидок без изменения кода самого модуля. Что ей следует использовать?

Подсказка: Подумайте о паттернах, которые позволяют менять алгоритмы на лету.

Ответ:

- Паттерн «Стратегия»
- Паттерн «Одиночка»
- Паттерн «Фабрика»

- Паттерн «Декоратор»

Шаг 02:

Миссия 2

Боб создает игру и хочет менять поведение персонажей без изменения их классов. Какой подход наиболее подходящий?

Подсказка: Вспомните паттерны для инкапсуляции алгоритмов.

Ответ:

- **Использовать паттерн «Стратегия»**
- Использовать наследование
- Создать глобальные функции для поведения
- Использовать шаблон «Фасад»

Шаг 03:

Миссия 3

 Задача: Сопоставьте примеры использования паттерна «Стратегия» с правильным описанием.

Примеры:

Ответ:

- Калькулятор налогов для разных стран
- Фильтры изображений (градации, размытие, контраст)
- Поведение NPC в игре
- Сортировка данных в таблице

Определения:

1. Изменение алгоритма сортировки без изменения класса таблицы
2. Выбор разных правил налогообложения для каждой страны
3. Применение разных фильтров к изображению
4. Динамическая смена поведения игрового персонажа

Подсказка:

Подумайте, где алгоритм может меняться независимо от объекта, который его использует.

- **Калькулятор налогов для разных стран** — Выбор разных правил налогообложения для каждой страны
- **Фильтры изображений (градации, размытие, контраст)** — Применение разных фильтров к изображению
- **Поведение NPC в игре** — Динамическая смена поведения игрового персонажа
- **Сортировка данных в таблице** — Изменение алгоритма сортировки без изменения класса таблицы

Шаг 04:

Миссия 4

 Сопоставьте стратегии расчета скидки с их назначением:

Стратегии:

Ответ:

- Скидка за объем заказа
- Сезонная скидка

- Скидка для постоянного клиента
- Скидка по промокоду

Назначения:

1. Поощрение повторных покупок
2. Стимулирование покупок в определенный сезон
3. Стимулирование крупных заказов
4. Привлечение новых клиентов через промо-акции

Подсказка:

Подумайте, какую цель преследует каждая скидка и кто ее получает.

- **Скидка за объем заказа** — Стимулирование крупных заказов
- **Сезонная скидка** — Стимулирование покупок в определенный сезон
- **Скидка для постоянного клиента** — Поощрение повторных покупок
- **Скидка по промокоду** — Привлечение новых клиентов через промо-акции

Шаг 05:

Миссия 5

 Сопоставьте принципы SOLID с их описанием:

Принципы:

Ответ:

- SRP
- OCP
- LSP
- DIP

Описание:

1. Класс должен быть открыт для расширения, но закрыт для модификации
2. Класс должен иметь только одну причину для изменения
3. Подклассы должны быть заменяемы родительским классом без изменения поведения
4. Зависимости должны быть на абстракции, а не на конкретных классах

Подсказка:

Обратите внимание, какой принцип прямо помогает реализовать «Стратегию» без изменения кода.

- **SRP** — Класс должен иметь только одну причину для изменения
- **OCP** — Класс должен быть открыт для расширения, но закрыт для модификации
- **LSP** — Подклассы должны быть заменяемы родительским классом без изменения поведения
- **DIP** — Зависимости должны быть на абстракции, а не на конкретных классах

Шаг 06:

Миссия 6

 Сопоставьте типы поведения персонажей с их стратегиями:

Поведение:

Ответ:

- Оборонительное

- Агрессивное
- Случайное
- Скоординированное с командой

Стратегии:

1. Уклонение и защита
2. Мгновенная атака
3. Движение без логики
4. Следование командным указаниям

🔗 Подсказка:

Подумайте, как алгоритмы поведения можно инкапсулировать и менять.

- **Оборонительное** — Уклонение и защита
- **Агрессивное** — Мгновенная атака
- **Случайное** — Движение без логики
- **Скоординированное с командой** — Следование командным указаниям

🔗 Шаг 07:

Миссия 7

Даниил хочет изменить стратегию логирования в приложении без изменения основного кода. Как лучше реализовать это?

Подсказка: Паттерн должен позволять выбирать алгоритм динамически.

Ответ:

- **Реализовать паттерн «Стратегия» для логирования**
- Жёстко закодировать функции логирования
- Создать глобальные переменные для выбора логирования
- Использовать наследование каждого типа логирования

🔗 Шаг 08:

Миссия 8

Мария хочет использовать стратегию сортировки данных в разных частях приложения без дублирования кода. Что использовать?

Подсказка: Подумайте о паттерне, который позволяет подставлять алгоритмы.

Ответ:

- **Паттерн «Стратегия»**
- Прямое наследование сортировщика
- Фиксированный метод сортировки в классе
- Дублирование кода сортировки везде

🔗 Шаг 09:

Миссия 9

Расположите шаги внедрения паттерна «Стратегия» в правильном порядке:

1. Определение интерфейса стратегии
2. Создание конкретных реализаций стратегии
3. Замена условных операторов в клиентском коде на стратегию
4. Использование стратегии в объекте клиента

Подсказка:

Подумайте о порядке: сначала интерфейс, потом реализации, потом подключение к клиенту.

Ответ:

- Определение интерфейса стратегии
- Создание конкретных реализаций стратегии
- Замена условных операторов в клиентском коде на стратегию
- Использование стратегии в объекте клиента

Шаг 10:

Миссия 10

Расположите шаги применения стратегии оплаты в интернет-магазине:

1. Создание интерфейса для оплаты
2. Реализация разных способов оплаты
3. Выбор способа оплаты пользователем
4. Вызов метода оплаты через выбранную стратегию

Подсказка:

Подумайте, где начинается интерфейс и где происходит вызов конкретной реализации.

Ответ:

- Создание интерфейса для оплаты
- Реализация разных способов оплаты
- Выбор способа оплаты пользователем
- Вызов метода оплаты через выбранную стратегию

Шаг 11:

Миссия 11

Расположите этапы внедрения стратегии фильтрации данных:

1. Определение интерфейса фильтра
2. Создание нескольких фильтров
3. Подключение выбранного фильтра к клиенту
4. Применение фильтрации к данным

Подсказка:

Подумайте, где нужно определить абстракцию и где применять конкретные стратегии.

Ответ:

- Определение интерфейса фильтра
- Создание нескольких фильтров
- Подключение выбранного фильтра к клиенту
- Применение фильтрации к данным

Шаг 12:

Миссия 12

Расположите шаги динамического выбора стратегии доставки:

1. Определение интерфейса доставки
2. Создание стратегий: курьер, почта, дрон

3. Выбор стратегии на основе условий
4. Вызов метода доставки через выбранную стратегию

 Подсказка:

Сначала создаём абстракцию, потом конкретные реализации, далее выбираем стратегию и используем её.

Ответ:

- Определение интерфейса доставки
- Создание стратегий: курьер, почта, дрон
- Выбор стратегии на основе условий
- Вызов метода доставки через выбранную стратегию

 Шаг 13:

Миссия 13

Расположите этапы интеграции стратегии расчета бонусов в системе:

1. Создание интерфейса бонусов
2. Реализация конкретных алгоритмов бонусов
3. Подключение выбранной стратегии в бизнес-логику
4. Тестирование работы с разными стратегиями

 Подсказка:

Подумайте о правильной последовательности: абстракция → реализация → использование → проверка.

Ответ:

- Создание интерфейса бонусов
- Реализация конкретных алгоритмов бонусов
- Подключение выбранной стратегии в бизнес-логику
- Тестирование работы с разными стратегиями

 Шаг 14:

Миссия 14

Расположите шаги внедрения стратегии оплаты в мобильном приложении:

1. Создание интерфейса оплаты
2. Реализация конкретных методов оплаты (карта, PayPal, Apple Pay)
3. Выбор метода оплаты пользователем
4. Вызов метода оплаты через выбранную стратегию

 Подсказка:

Сначала создаём абстракцию, потом реализации, потом выбор, и только затем вызов стратегии.

Ответ:

- Создание интерфейса оплаты
- Реализация конкретных методов оплаты (карта, PayPal, Apple Pay)
- Выбор метода оплаты пользователем
- Вызов метода оплаты через выбранную стратегию

 Шаг 15:

Миссия 15

Расположите шаги внедрения стратегии фильтрации новостей в приложении:

1. Определение интерфейса фильтрации
2. Создание конкретных фильтров (по дате, по популярности, по теме)
3. Выбор фильтра пользователем
4. Применение выбранного фильтра к ленте новостей

 Подсказка:

Сначала интерфейс, затем реализации, потом выбор стратегии и её использование.

Ответ:

- Определение интерфейса фильтрации
- Создание конкретных фильтров (по дате, по популярности, по теме)
- Выбор фильтра пользователем
- Применение выбранного фильтра к ленте новостей

 Шаг 16:

Миссия 16

Расположите этапы выбора стратегии компрессии изображений:

1. Определение интерфейса компрессии
2. Создание стратегий: JPEG, PNG, WebP
3. Выбор стратегии на основе формата и качества
4. Вызов метода компрессии через выбранную стратегию

 Подсказка:

Сначала абстракция, потом реализации, затем выбор и применение.

Ответ:

- Определение интерфейса компрессии
- Создание стратегий: JPEG, PNG, WebP
- Выбор стратегии на основе формата и качества
- Вызов метода компрессии через выбранную стратегию

 Шаг 17:

Миссия 17

Расположите этапы применения стратегии кэширования данных:

1. Определение интерфейса кэша
2. Создание стратегий: Memcached, Redis, файловый кэш
3. Выбор стратегии на основе нагрузки и размера данных
4. Применение стратегии к данным

 Подсказка:

Сначала абстракция, потом реализации, потом выбор и использование.

Ответ:

- Определение интерфейса кэша
- Создание стратегий: Memcached, Redis, файловый кэш
- Выбор стратегии на основе нагрузки и размера данных
- Применение стратегии к данным

Шаг 18:

Миссия 18

Расположите шаги внедрения стратегии шифрования данных:

1. Создание интерфейса шифрования
2. Реализация алгоритмов: AES, RSA, Blowfish
3. Выбор алгоритма в зависимости от типа данных
4. Шифрование данных через выбранную стратегию

Подсказка:

Сначала интерфейс, потом реализации, затем выбор стратегии и её применение.

Ответ:

- Создание интерфейса шифрования
- Реализация алгоритмов: AES, RSA, Blowfish
- Выбор алгоритма в зависимости от типа данных
- Шифрование данных через выбранную стратегию

Шаг 19:

Миссия 19

Расположите этапы внедрения стратегии уведомлений:

1. Определение интерфейса уведомлений
2. Создание стратегий: Email, SMS, Push
3. Выбор стратегии в зависимости от настроек пользователя
4. Отправка уведомления через выбранную стратегию

Подсказка:

Абстракция, конкретные реализации, выбор, вызов метода.

Ответ:

- Определение интерфейса уведомлений
- Создание стратегий: Email, SMS, Push
- Выбор стратегии в зависимости от настроек пользователя
- Отправка уведомления через выбранную стратегию

Урок 2 - Фабрика Создание по чертежу

Шаг 01:

Миссия 1

Вы разрабатываете простую игру и хотите создать разных персонажей с уникальными способностями. Какой паттерн поможет вам легко создавать новые объекты?

Подсказка: Подумайте о шаблоне, который позволяет создавать объекты по чертежу.

Ответ:

- **Фабрика (Factory)**
- Одиночка (Singleton)
- Наблюдатель (Observer)
- Стратегия (Strategy)

Шаг 02:

Миссия 2

Вы создаёте веб-приложение, которое поддерживает разные виды уведомлений (email, SMS, push). Что поможет вам создавать их без изменения существующего кода?

Подсказка: Подумайте о паттерне, который позволяет создавать объекты различных классов через единый интерфейс.

Ответ:

- **Фабрика (Factory)**
- Декоратор (Decorator)
- Мост (Bridge)
- Компоновщик (Composite)

Шаг 03:

Миссия 3

Ваша команда хочет добавить новые типы отчетов без изменения существующего кода генерации. Какой подход будет оптимальным?

Подсказка: Подумайте о том, как централизовать создание объектов разных типов.

Ответ:

- **Использовать фабричный метод**
- Прямо создавать объекты через new
- Копировать существующие объекты и менять поля
- Использовать глобальные переменные

Шаг 04:

Миссия 4

Вы хотите, чтобы создание объектов зависело от внешней конфигурации, а не жестко закодированных условий. Как лучше всего поступить?

Подсказка: Подумайте о паттерне, который скрывает детали создания объекта от клиента.

Ответ:

- **Применить фабрику**
- Создавать объекты прямо в контроллере
- Использовать if-else для каждого типа объекта
- Хранить объекты в глобальном массиве

Шаг 05:

Миссия 5

 Сопоставьте термин с его определением.

Подсказка: Обратите внимание, какой паттерн скрывает детали создания объектов.

Понятия:

Ответ:

- Фабрика
- Клиент
- Продукт
- Абстрактная фабрика

Определения:

1. Объект, который использует фабрику для получения нужного продукта.
 2. Класс, отвечающий за создание семейства связанных объектов.
 3. Интерфейс или класс, который создается через фабрику.
 4. Паттерн, предоставляющий интерфейс для создания объектов без указания конкретных классов.
- **Фабрика** — Паттерн, предоставляющий интерфейс для создания объектов без указания конкретных классов.
 - **Клиент** — Объект, который использует фабрику для получения нужного продукта.
 - **Продукт** — Интерфейс или класс, который создается через фабрику.
 - **Абстрактная фабрика** — Класс, отвечающий за создание семейства связанных объектов.

Шаг 06:

Миссия 6

 Сопоставьте пример с его ролью в паттерне "Фабрика".

Подсказка: Подумайте, кто создает, кто использует, а что является конечным продуктом.

Понятия:

Ответ:

- EmailNotificationFactory
- SMSNotification
- NotificationService
- PushNotificationFactory

Определения:

1. Создает объекты уведомлений для push
 2. Использует уведомления для отправки
 3. Создает email-уведомления
 4. Конкретный объект уведомления
- **EmailNotificationFactory** — Создает email-уведомления
 - **SMSNotification** — Конкретный объект уведомления
 - **NotificationService** — Использует уведомления для отправки
 - **PushNotificationFactory** — Создает объекты уведомлений для push

Шаг 07:

Миссия 7

 Сопоставьте описание с правильным типом фабрики.

Подсказка: Есть фабрика для одного продукта и фабрика для семейства продуктов.

Понятия:

Ответ:

- Фабричный метод
- Абстрактная фабрика
- Конкретная фабрика
- Продукт

Определения:

1. Создает один объект через метод
 2. Создает семейство взаимосвязанных объектов
 3. Реализует интерфейс фабрики для создания конкретного продукта
 4. Объект, который создается фабрикой
- **Фабричный метод** — Создает один объект через метод
 - **Абстрактная фабрика** — Создает семейство взаимосвязанных объектов
 - **Конкретная фабрика** — Реализует интерфейс фабрики для создания конкретного продукта
 - **Продукт** — Объект, который создается фабрикой

Шаг 08:

Миссия 8

 Сопоставьте класс с его назначением в фабричном паттерне.

Подсказка: Кто отвечает за создание, а кто — за использование?

Понятия:

Ответ:

- CarFactory
- Car
- Driver
- LuxuryCarFactory

Определения:

1. Создает стандартные автомобили
 2. Создает люксовые автомобили
 3. Использует объекты автомобилей
 4. Конкретный автомобиль
- **CarFactory** — Создает стандартные автомобили
 - **Car** — Конкретный автомобиль
 - **Driver** — Использует объекты автомобилей
 - **LuxuryCarFactory** — Создает люксовые автомобили

Шаг 09:

Миссия 9

Расположите шаги использования фабрики в правильном порядке:

1. Клиент запрашивает объект у фабрики
2. Фабрика создаёт объект
3. Объект возвращается клиенту
4. Клиент использует объект

Подсказка: Сначала клиент просит объект, потом фабрика его создает, затем возвращает, и только после этого клиент использует.

Ответ:

- Клиент запрашивает объект у фабрики
- Фабрика создаёт объект
- Объект возвращается клиенту

- Клиент использует объект

Шаг 10:

Миссия 10

Расположите шаги внедрения нового типа уведомлений через фабрику:

1. Создать класс нового уведомления
2. Добавить метод создания уведомления в фабрику
3. Использовать уведомление в сервисе
4. Протестировать корректность работы

Подсказка: Сначала создаем класс, потом интегрируем в фабрику, затем используем и тестируем.

Ответ:

- Создать класс нового уведомления
- Добавить метод создания уведомления в фабрику
- Использовать уведомление в сервисе
- Протестировать корректность работы

Шаг 11:

Миссия 11

Расположите шаги при добавлении нового продукта в абстрактную фабрику:

1. Определить интерфейс продукта
2. Создать конкретный класс продукта
3. Добавить создание продукта в конкретную фабрику
4. Использовать продукт в клиентском коде

Подсказка: Логика похожа на обычную фабрику, только с семейством продуктов.

Ответ:

- Определить интерфейс продукта
- Создать конкретный класс продукта
- Добавить создание продукта в конкретную фабрику
- Использовать продукт в клиентском коде

Шаг 12:

Миссия 12

Расположите этапы тестирования фабрики:

1. Проверить правильность создания объектов
2. Проверить поведение объектов
3. Проверить интеграцию с клиентом
4. Проверить обработку ошибок при неправильных входных данных

Подсказка: Сначала проверяем отдельные объекты, потом их использование и обработку ошибок.

Ответ:

- Проверить правильность создания объектов
- Проверить поведение объектов

- Проверить интеграцию с клиентом
- Проверить обработку ошибок при неправильных входных данных

Шаг 13:

Миссия 13

Расположите шаги при миграции существующего кода на фабрику:

1. Определить повторяющийся код для создания объектов
2. Создать фабрику и методы создания
3. Заменить прямое создание объектов на вызовы фабрики
4. Протестировать корректность работы системы

Подсказка: Сначала выделяем повторяющийся код, потом делаем фабрику и заменяем, затем тестируем.

Ответ:

- Определить повторяющийся код для создания объектов
- Создать фабрику и методы создания
- Заменить прямое создание объектов на вызовы фабрики
- Протестировать корректность работы системы

Шаг 14:

Миссия 14

Расположите шаги при расширении фабрики для нового семейства продуктов:

1. Определить новые интерфейсы продуктов
2. Создать конкретные классы для каждого продукта
3. Реализовать новые фабрики для семейства продуктов
4. Интегрировать новые фабрики в клиентский код

Подсказка: Логика расширения похожа на создание обычной фабрики, но для целого семейства продуктов.

Ответ:

- Определить новые интерфейсы продуктов
- Создать конкретные классы для каждого продукта
- Реализовать новые фабрики для семейства продуктов
- Интегрировать новые фабрики в клиентский код

Шаг 15:

Миссия 15

Расположите шаги при миграции существующей системы на использование фабрик:

1. Проанализировать существующий код и выделить объекты для фабрики
2. Создать фабрики и методы создания объектов
3. Заменить прямое создание объектов на вызовы фабрики
4. Протестировать корректность работы системы после изменений

Подсказка: Сначала анализируем код, затем создаём фабрики и заменяем объекты, в конце тестируем.

Ответ:

- Проанализировать существующий код и выделить объекты для фабрики

- Создать фабрики и методы создания объектов
- Заменить прямое создание объектов на вызовы фабрики
- Протестировать корректность работы системы после изменений

Шаг 16:

Миссия 16

Расположите шаги при написании юнит-тестов для фабрики:

1. Создать тесты для проверки создания каждого конкретного объекта
2. Проверить, что возвращаемые объекты соответствуют ожидаемым интерфейсам
3. Проверить работу фабрики с разными параметрами
4. Обеспечить обработку ошибок при некорректных входных данных

Подсказка: Тестирование начинается с конкретных объектов, затем интерфейсов и параметров, а завершается проверкой ошибок.

Ответ:

- Создать тесты для проверки создания каждого конкретного объекта
- Проверить, что возвращаемые объекты соответствуют ожидаемым интерфейсам
- Проверить работу фабрики с разными параметрами
- Обеспечить обработку ошибок при некорректных входных данных

Шаг 17:

Миссия 17

Расположите правильные шаги для полного внедрения фабричного паттерна в крупный проект:

1. Проанализировать текущие участки кода для потенциальных фабрик
2. Разработать фабрики и интерфейсы для объектов
3. Заменить все прямые вызовы конструктора на фабрики
4. Проверить интеграцию фабрик со всеми модулями проекта
5. Провести финальное тестирование всей системы

Подсказка: Внедрение фабрики требует тщательного анализа, проектирования, замены кода и тестирования на всех уровнях.

Ответ:

- Проанализировать текущие участки кода для потенциальных фабрик
- Разработать фабрики и интерфейсы для объектов
- Заменить все прямые вызовы конструктора на фабрики
- Проверить интеграцию фабрик со всеми модулями проекта
- Провести финальное тестирование всей системы

Урок 3 - Синглтон — Один, чтобы править

Шаг 01:

Миссия 1

Вы разрабатываете логгер для крупного веб-приложения, чтобы все части системы писали события в один и тот же файл. Какой паттерн здесь подходит лучше всего?

Подсказка: Подумайте о паттерне, который гарантирует один экземпляр объекта в программе.

Ответ:

- **Синглтон (Singleton)**

- Фабрика (Factory)
- Декоратор (Decorator)
- Компоновщик (Composite)

🔮 Шаг 02:

Миссия 2

Ваша команда хочет, чтобы доступ к базе данных был через один объект соединения. Что следует использовать?

Подсказка: Подумайте о глобальном объекте, который должен быть один на весь проект.

Ответ:

- **Синглтон (Singleton)**
- Стратегия (Strategy)
- Прототип (Prototype)
- Мост (Bridge)

🔮 Шаг 03:

Миссия 3

Вы разрабатываете приложение с кэшем, который должен быть общий для всех модулей. Как реализовать это корректно?

Подсказка: Идеально подходит паттерн, который гарантирует единственный экземпляр объекта.

Ответ:

- **Синглтон (Singleton)**
- Фабрика (Factory)
- Компоновщик (Composite)
- Декоратор (Decorator)

🔮 Шаг 04:

Миссия 4

Вы хотите, чтобы конфигурация приложения читалась только один раз при старте и была доступна всем модулям. Какой паттерн использовать?

Подсказка: Подумайте о глобальном объекте с единственным экземпляром.

Ответ:

- **Синглтон (Singleton)**
- Стратегия (Strategy)
- Абстрактная фабрика (Abstract Factory)
- Прототип (Prototype)

🔮 Шаг 05:

Миссия 5

📖 Задача: Сопоставьте термин Синглтона с правильным описанием.

Понятия:

Ответ:

- Синглтон
- Глобальная переменная

- Преимущество Singleton
- Опасность Singleton

Определения:

1. Обеспечивает единственный экземпляр объекта на весь проект
2. Доступен в любом месте, но не контролирует создание экземпляров
3. Позволяет централизованно управлять ресурсами
4. Может создавать скрытые зависимости и затруднять тестирование

Подсказка: Сравните Singleton с обычными глобальными переменными, обратите внимание на плюсы и минусы.

- **Синглтон** — Обеспечивает единственный экземпляр объекта на весь проект
- **Глобальная переменная** — Доступен в любом месте, но не контролирует создание экземпляров
- **Преимущество Singleton** — Позволяет централизованно управлять ресурсами
- **Опасность Singleton** — Может создавать скрытые зависимости и затруднять тестирование

Шаг 06:

Миссия 6

 Задача: Сопоставьте термин с примером из жизни программиста.

Ответ:

- Синглтон
 - Логгер
 - Конфигурация
 - Класс с глобальным состоянием
1. Создание единственного объекта для записи логов в файл
 2. Объект, который хранит настройки приложения
 3. Объект, который существует в единственном экземпляре в проекте
 4. Может быть изменен из любого места без контроля

Подсказка: Подумайте, что обычно делают с настройками, логами и глобальными данными.

- **Синглтон** — Объект, который существует в единственном экземпляре в проекте
- **Логгер** — Создание единственного объекта для записи логов в файл
- **Конфигурация** — Объект, который хранит настройки приложения
- **Класс с глобальным состоянием** — Может быть изменен из любого места без контроля

Шаг 07:

Миссия 7

 Сопоставьте паттерн Singleton с его особенностями реализации.

Ответ:

- Приватный конструктор
 - Статический метод getInstance()
 - Контроль количества экземпляров
 - Использование в многопоточности
1. Не позволяет создавать объект напрямую
 2. Возвращает единственный экземпляр объекта

3. Гарантирует, что будет только один объект
4. Нужен для безопасного доступа из разных потоков

Подсказка: Вспомните стандартную реализацию Singleton в PHP и основные элементы.

- **Приватный конструктор** — Не позволяет создавать объект напрямую
- **Статический метод getInstance()** — Возвращает единственный экземпляр объекта
- **Контроль количества экземпляров** — Гарантирует, что будет только один объект
- **Использование в многопоточности** — Нужен для безопасного доступа из разных потоков

Шаг 08:

Миссия 8

 Сопоставьте примеры использования Singleton в реальных проектах.

Ответ:

- Кэш
 - Менеджер сессий
 - Соединение с базой данных
 - Конфигурация приложения
1. Хранение данных между запросами в памяти
 2. Хранение данных о текущих пользователях
 3. Единый объект для запросов к БД
 4. Централизованное хранение настроек

Подсказка: Подумайте, какие объекты должны быть уникальными для всего приложения.

- **Кэш** — Хранение данных между запросами в памяти
- **Менеджер сессий** — Хранение данных о текущих пользователях
- **Соединение с базой данных** — Единый объект для запросов к БД
- **Конфигурация приложения** — Централизованное хранение настроек

Шаг 09:

Миссия 9

Расположите шаги создания Singleton для логгера:

1. Создать приватный конструктор
2. Создать статический метод getInstance()
3. Создать объект внутри getInstance() при первом вызове
4. Использовать экземпляр через getInstance()

Подсказка: Логика начинается с ограничения конструктора, затем создаём и используем объект.

Ответ:

- Создать приватный конструктор
- Создать статический метод getInstance()
- Создать объект внутри getInstance() при первом вызове
- Использовать экземпляр через getInstance()

Шаг 10:

Миссия 10

Расположите шаги внедрения Singleton для конфигурации приложения:

1. Определить параметры конфигурации
2. Создать класс Singleton с приватным конструктором
3. Реализовать метод getInstance()
4. Использовать Singleton во всех модулях приложения

Подсказка: Сначала определяем данные, затем создаём Singleton, после чего используем его.

Ответ:

- Определить параметры конфигурации
- Создать класс Singleton с приватным конструктором
- Реализовать метод getInstance()
- Использовать Singleton во всех модулях приложения

Шаг 11:

Миссия 11

Расположите шаги защиты Singleton от многократного создания через клонирование:

1. Сделать приватным метод __clone()
2. Сделать приватным метод __wakeup()
3. Контролировать создание экземпляра через getInstance()
4. Использовать объект Singleton

Подсказка: Подумайте о защите от клонирования и сериализации.

Ответ:

- Сделать приватным метод __clone()
- Сделать приватным метод __wakeup()
- Контролировать создание экземпляра через getInstance()
- Использовать объект Singleton

Шаг 12:

Миссия 12

Расположите шаги тестирования Singleton для сессий:

1. Создать тестовый класс
2. Получить несколько экземпляров через getInstance()
3. Проверить, что все экземпляры равны
4. Проверить сохранение данных между вызовами

Подсказка: Тестирование начинается с получения объектов и проверки их уникальности и состояния.

Ответ:

- Создать тестовый класс
- Получить несколько экземпляров через getInstance()
- Проверить, что все экземпляры равны
- Проверить сохранение данных между вызовами

Шаг 13:

Миссия 13

Расположите шаги внедрения Singleton для кэша:

1. Создать класс CacheSingleton
2. Сделать конструктор приватным
3. Добавить метод getInstance()
4. Создать методы set и get для хранения данных
5. Использовать SingletonCache в проекте

Подсказка: Логика схожа с обычным Singleton, добавляем лишь методы работы с кэшем.

Ответ:

- Создать класс CacheSingleton
- Сделать конструктор приватным
- Добавить метод getInstance()
- Создать методы set и get для хранения данных
- Использовать SingletonCache в проекте

Шаг 14:

Миссия 14

Расположите шаги внедрения Singleton для глобальной очереди задач:

1. Создать класс TaskQueueSingleton
2. Сделать конструктор приватным
3. Создать метод getInstance()
4. Добавить методы addTask и getTask
5. Использовать объект Singleton во всех модулях

Подсказка: Логика аналогична другим Singleton, добавляем лишь методы работы с очередью.

Ответ:

- Создать класс TaskQueueSingleton
- Сделать конструктор приватным
- Создать метод getInstance()
- Добавить методы addTask и getTask
- Использовать объект Singleton во всех модулях

Шаг 15:

Миссия 15

Расположите шаги внедрения Singleton для менеджера соединений с API:

1. Создать класс ApiConnectionSingleton
2. Сделать конструктор приватным
3. Реализовать метод getInstance()
4. Добавить методы sendRequest и getResponse
5. Использовать Singleton в проекте

Подсказка: Начните с класса, затем метод getInstance, после добавляйте функциональность и используйте объект.

Ответ:

- Создать класс ApiConnectionSingleton
- Сделать конструктор приватным
- Реализовать метод getInstance()

- Добавить методы `sendRequest` и `getResponse`
- Использовать Singleton в проекте

Шаг 16:

Миссия 16

Расположите шаги внедрения Singleton для менеджера уведомлений:

1. Создать класс `NotificationManagerSingleton`
2. Сделать конструктор приватным
3. Создать метод `getInstance()`
4. Добавить методы `sendNotification` и `queueNotification`
5. Использовать объект Singleton во всех модулях

Подсказка: Подумайте о последовательности: класс, конструктор, `getInstance`, методы, использование.

Ответ:

- Создать класс `NotificationManagerSingleton`
- Сделать конструктор приватным
- Создать метод `getInstance()`
- Добавить методы `sendNotification` и `queueNotification`
- Использовать объект Singleton во всех модулях

Урок 4 - Адаптер — Переводчик миров

Шаг 01:

Миссия 1

Программист Артём подключает к своей системе старую библиотеку перевода текстов, у которой другой интерфейс. Какой шаблон проектирования поможет ему сделать это без изменения исходного кода библиотеки?

Подсказка: Подумайте о шаблоне, который превращает один интерфейс в другой.

Ответ:

- Адаптер
- Стратегия
- Фабричный метод
- Декоратор

Шаг 02:

Миссия 2

Катя пишет код, который должен работать с новой системой платежей, но интерфейс сильно отличается от старого. Она решила применить адаптер. Какую роль выполняет адаптер в такой ситуации?

Подсказка: Вспомните: адаптер выступает как переводчик между системами.

Ответ:

- Преобразует интерфейс одной системы в интерфейс, понятный другой
- Хранит конфигурации для разных платежных систем
- Создает новые классы для каждой операции
- Оптимизирует скорость работы программы

Шаг 03:

Миссия 3

В команде Пети решили подключить внешнюю систему логирования. Но её методы называются не так, как в проекте. Что проще всего сделать?

Подсказка: Важно не переписывать сторонний код, а создать прослойку.

Ответ:

- Скопировать код библиотеки и переписать названия методов
- **Создать адаптер, который преобразует вызовы методов**
- Удалить использование логирования совсем
- Заменить библиотеку другой случайной

Шаг 04:

Миссия 4

Илья использует класс OldPrinter, но хочет, чтобы его проект принимал только интерфейс Printable. Как правильно применить адаптер?

Подсказка: Вспомните принцип «обернуть и перевести методы».

Ответ:

- **Создать класс Adapter, реализующий Printable и использующий внутри OldPrinter**
- Изменить код OldPrinter, добавив методы Printable
- Удалить Printable и работать напрямую с OldPrinter
- Наследоваться от OldPrinter и игнорировать интерфейс Printable

Шаг 05:

Миссия 5

В проекте Даши требуется подключить старый API погоды, но методы возвращают данные в неудобном формате. Как адаптер может помочь?

Подсказка: Он преобразует не только имена методов, но и данные.

Ответ:

- Удалить ненужные поля вручную при каждом вызове
- **Реализовать адаптер, который преобразует формат данных в ожидаемый**
- Скопировать API и переписать его заново
- Хранить все данные в JSON без изменений

Шаг 06:

Миссия 6

Алексей разрабатывает чат, куда нужно подключить библиотеку переводов. Но библиотека возвращает только коды языков, а чат требует полные названия. Как лучше поступить?

Подсказка: Можно добавить «переводчик» между библиотекой и системой.

Ответ:

- **Создать адаптер, который превращает коды языков в названия**
- Хранить отдельный справочник и вручную заменять коды
- Переписать библиотеку перевода
- Игнорировать проблему и показывать коды

Шаг 07:

Миссия 7

Виктор написал адаптер для старого API платежей. Но он заметил, что адаптер стал слишком большим и содержит много преобразований. Какой риск это создаёт?

Подсказка: Вспомните про принцип единственной ответственности.

Ответ:

- **Адаптер становится слишком сложным и нарушает SRP**
- Адаптер будет работать быстрее
- Адаптер перестанет компилироваться
- Адаптер всегда приводит к утечке памяти

Шаг 08:

Миссия 8

Марина работает с двумя разными API карт. Одно ожидает координаты в градусах, другое — в радианах. Как адаптер может помочь?

Подсказка: Адаптер может изменять формат входных данных.

Ответ:

- Привязать оба API напрямую друг к другу
- **Создать адаптер, который конвертирует градусы в радианы и обратно**
- Удалить одно из API
- Вывести ошибку при несовпадении форматов

Шаг 09:

Миссия 9

Сопоставьте персонажей и их задачи по использованию адаптера.

Подсказка: Думайте о том, какой именно «перевод» требуется каждому.

Ответ:

- **Артём** — Соединяет старую библиотеку перевода с новым интерфейсом
- **Катя** — Подключает систему платежей с другим интерфейсом
- **Илья** — Использует OldPrinter через Printable
- **Марина** — Конвертирует градусы в радианы для карт

Шаг 10:

Миссия 10

Сопоставьте элементы шаблона «Адаптер» с их назначением.

Подсказка: Вспомните: клиент, адаптер и адаптируемый объект.

Ответ:

- **Клиент** — Работает только с ожидаемым интерфейсом
- **Адаптер** — Преобразует интерфейс адаптируемого класса
- **Адаптируемый класс** — Имеет несовместимый интерфейс
- **Интерфейс** — Задаёт контракт, который понимает клиент

Шаг 11:

Миссия 11

Сопоставьте примеры из жизни с паттерном «Адаптер».

Подсказка: Подумайте о «переходниках».

Ответ:

- **Зарядное устройство с переходником** — Преобразует один стандарт вилки в другой
- **Программистский адаптер** — Преобразует методы одного интерфейса в методы другого
- **Переводчик между людьми** — Преобразует язык общения
- **Адаптер HDMI-VGA** — Превращает один стандарт передачи данных в другой

Шаг 12:

Миссия 12

Сопоставьте типы адаптеров с их особенностями.

Подсказка: Классовый адаптер использует наследование, объектный — композицию.

Ответ:

- **Классовый адаптер** — Наследуется от адаптируемого класса
- **Объектный адаптер** — Содержит ссылку на адаптируемый объект
- **Универсальный адаптер** — Может работать с несколькими классами
- **Специализированный адаптер** — Рассчитан только на один конкретный класс

Шаг 13:

Миссия 13

Сопоставьте программистов с их ошибками при использовании адаптера.

Подсказка: Ошибки часто связаны с нарушением SRP или избыточностью.

Ответ:

- **Виктор** — Создал слишком громоздкий адаптер
- **Катя** — Изменила исходный код библиотеки вместо адаптера
- **Илья** — Наследовался от старого класса вместо использования интерфейса
- **Марина** — Сделала адаптер, который меняет слишком много форматов сразу

Шаг 14:

Миссия 14

Расположите шаги применения адаптера в правильном порядке.

Подсказка: Сначала создаем интерфейс, затем адаптер, потом используем.

Ответ:

- Определить интерфейс, который нужен клиенту
- Создать адаптер, реализующий этот интерфейс
- В адаптере использовать адаптируемый класс
- Клиент вызывает методы через адаптер

Шаг 15:

Миссия 15

Расставьте этапы подключения стороннего API через адаптер.

Подсказка: Вспомните: анализ → проектирование → код → проверка.

Ответ:

- Определить несовместимости интерфейсов

- Спроектировать адаптер
- Реализовать адаптер
- Протестировать работу клиента с адаптером

Урок 5 - Наблюдатель — Око, следящее за изменениями

Шаг 01:

Миссия 1

Антон пишет систему уведомлений: при изменении профиля пользователя все подписчики должны мгновенно узнавать об этом. Какой паттерн поможет реализовать такую механику?

Подсказка: Подумайте о паттерне, где одни объекты следят за состоянием другого.

Ответ:

- **Наблюдатель**
- Стратегия
- Адаптер
- Одиночка

Шаг 02:

Миссия 2

Марина разрабатывает чат. Когда пользователь пишет сообщение, оно должно появляться у всех участников. Какую роль выполняет наблюдатель в этой схеме?

Подсказка: Подумайте: есть один источник события и несколько слушателей.

Ответ:

- **Следит за изменениями и реагирует на них**
- Создаёт новые объекты для каждой копии сообщения
- Оптимизирует алгоритм поиска сообщений
- Хранит историю сообщений

Шаг 03:

Миссия 3

Илья внедряет систему мониторинга сервера. Несколько модулей должны реагировать на изменение нагрузки. Как называется объект, который рассылает уведомления наблюдателям?

Подсказка: Он управляет списком подписчиков.

Ответ:

- **Субъект (Subject)**
- Интерфейс
- Клиент
- Фабрика

Шаг 04:

Миссия 4

В проекте Кати есть класс WeatherStation, который сообщает температуру. Несколько дисплеев должны автоматически обновляться при изменении данных. Что в этой схеме являются наблюдателями?

Подсказка: Это те объекты, которые реагируют на обновления.

Ответ:

- **Дисплеи**
- WeatherStation
- Клиентский код
- Сервер БД

Шаг 05:

Миссия 5

Дима пишет торговую систему: при изменении курса валют уведомления должны приходить трейдерам, банкам и аналитикам. В чём преимущество использования наблюдателя?

Подсказка: Вспомните, что код субъекта не должен зависеть от числа подписчиков.

Ответ:

- **Лёгкое подключение новых подписчиков без изменения субъекта**
- Ускорение вычислений
- Гарантия отсутствия ошибок
- Снижение нагрузки на сервер

Шаг 06:

Миссия 6

Алина пишет игру. Когда здоровье персонажа меняется, интерфейс, журнал событий и музыка должны реагировать. Что произойдёт, если один наблюдатель даст сбой?

Подсказка: Подумайте: субъект уведомляет всех, независимо от состояния конкретного подписчика.

Ответ:

- **Остальные наблюдатели всё равно получают уведомления**
- Субъект перестанет уведомлять всех
- Все наблюдатели остановятся
- Программа завершится аварийно

Шаг 07:

Миссия 7

Разработчик Никита использует наблюдателя в системе оповещений. Но подписчики стали слишком сильно зависеть от конкретного субъекта. Какую ошибку он допустил?

Подсказка: Вспомните принцип программирования «через интерфейсы».

Ответ:

- **Подписчики завязаны на конкретную реализацию, а не на интерфейс**
- Подписчики слишком часто обновляются
- Субъект не уведомляет подписчиков
- Все уведомления отправляются вручную

Шаг 08:

Миссия 8

Егор создал систему обновления новостей. Иногда наблюдатели получают слишком много данных, а им нужно только часть. Как это решить?

Подсказка: Субъект может отправлять только необходимую информацию.

Ответ:

- **Передавать наблюдателям только релевантные данные**
- Отключить половину наблюдателей
- Отменить использование наблюдателя
- Дублировать все данные для каждого наблюдателя

🔗 Шаг 09:

Миссия 9

Сопоставьте элементы паттерна «Наблюдатель» с их ролями.

Подсказка: Вспомните: субъект уведомляет, наблюдатель реагирует.

Ответ:

- **Субъект** — Хранит список наблюдателей и рассылает уведомления
- **Наблюдатель** — Реагирует на изменения состояния
- **Интерфейс Observer** — Определяет метод обновления
- **Интерфейс Subject** — Определяет методы подписки и отписки

🔗 Шаг 10:

Миссия 10

Сопоставьте примеры из жизни с паттерном «Наблюдатель».

Подсказка: Подумайте о ситуациях, когда одно событие вызывает реакцию многих.

Ответ:

- **Подписка на YouTube** — Новый ролик уведомляет всех подписчиков
- **Система сигнализации** — Оповещает охрану и полицию при тревоге
- **Биржевые приложения** — Следят за изменением курса акций
- **Мессенджер** — Рассылает сообщения всем в чате

🔗 Шаг 11:

Миссия 11

Сопоставьте разработчиков и их сценарии с использованием наблюдателя.

Подсказка: У каждого есть свой «субъект» и «подписчики».

Ответ:

- **Антон** — Профиль пользователя уведомляет подписчиков
- **Марина** — Чат рассылает сообщения всем участникам
- **Илья** — Мониторинг сервера сообщает модулям
- **Катя** — WeatherStation обновляет дисплей

🔗 Шаг 12:

Миссия 12

Сопоставьте проблемы с правильными решениями при использовании наблюдателя.

Подсказка: Ошибки часто связаны с избыточностью или зависимостью от реализации.

Ответ:

- **Слишком много уведомлений** — Передавать только релевантные данные
- **Подписчики зависят от реализации субъекта** — Использовать интерфейсы
- **Один подписчик ломает остальных** — Обрабатывать ошибки индивидуально

- **Трудно добавить новых подписчиков** — Использовать централизованный список

🔧 Шаг 13:

Миссия 13

Сопоставьте типы реализации наблюдателя с их особенностями.

Подсказка: Вспомните про pull и push-модель.

Ответ:

- **Push-модель** — Субъект отправляет все данные наблюдателям
- **Pull-модель** — Наблюдатель сам запрашивает нужные данные
- **Синхронная модель** — Все уведомления обрабатываются сразу
- **Асинхронная модель** — Уведомления могут приходить с задержкой

🔧 Шаг 14:

Миссия 14

Расположите шаги реализации наблюдателя в правильном порядке.

Подсказка: Сначала интерфейсы, потом субъект, затем подписчики.

Ответ:

- Создать интерфейс Observer
- Создать интерфейс Subject
- Реализовать субъект с методами подписки
- Добавить наблюдателей
- Изменить состояние и уведомить

🔧 Шаг 15:

Миссия 15

Расставьте действия системы оповещения при изменении состояния.

Подсказка: Сначала субъект меняет состояние, потом уведомляет.

Ответ:

- Субъект меняет состояние
- Субъект вызывает метод уведомления
- Каждый наблюдатель получает сигнал
- Наблюдатели обновляют данные

🔧 Шаг 16:

Миссия 16

Расположите преимущества наблюдателя в порядке их проявления.

Подсказка: Сначала гибкость, потом расширяемость, затем слабая связанность.

Ответ:

- Гибкость при добавлении подписчиков
- Простота расширения системы
- Снижение связанности компонентов

Урок 6 - Декоратор — Расширение без вторжения

Шаг 01:

Миссия 1

Молодой разработчик Андрей пишет систему для кофейни. Он хочет добавить новые функции (например, скидки и бонусные баллы), но не изменять основной класс Coffee. Какой шаблон проектирования лучше всего подходит?

Подсказка: Нужно расширить поведение объекта, не трогая его исходный код.

Ответ:

- **Декоратор**
- Адаптер
- Стратегия
- Одиночка

Шаг 02:

Миссия 2

Разработчица Катя делает проект для интернет-магазина. Она хочет динамически добавлять к объекту Product функционал «Подарочная упаковка». Что она выберет?

Подсказка: Подумай о шаблоне, который оборачивает базовый класс, добавляя новые свойства.

Ответ:

- Фасад
- **Декоратор**
- Посредник
- Команда

Шаг 03:

Миссия 3

В команде спор: чем отличается «Декоратор» от «Наследования»?

Подсказка: Один способ жёстко фиксирует структуру, другой более гибок и динамичен.

Ответ:

- **Декоратор позволяет изменять поведение объектов во время выполнения, а наследование — только при компиляции.**
- Декоратор всегда быстрее, чем наследование.
- **Наследование используется только для интерфейсов, а декоратор — для классов.**
- Между ними нет различий.

Шаг 04:

Миссия 4

Программист Дима хочет добавить к объекту «Сообщение» функцию шифрования и логирования. Как лучше реализовать?

Подсказка: Нужно оборачивать объект дополнительными слоями.

Ответ:

- **Использовать несколько декораторов (шифрование, затем логирование)**
- Изменить класс «Сообщение», добавив туда всё сразу
- **Сделать цепочку обёрток вокруг исходного объекта**

- Скопировать код шифрования в каждый метод

Шаг 05:

Миссия 5

Что является ключевым признаком паттерна «Декоратор»?

Подсказка: Важно помнить про одинаковый интерфейс и возможность оборачивать объекты друг в друга.

Ответ:

- Добавление новых конструкторов
- **Обертка реализует тот же интерфейс, что и основной объект**
- Обязательное наследование от абстрактного класса
- Использование глобальных переменных

Шаг 06:

Миссия 6

Сопоставьте примеры использования декоратора с их описанием.

Подсказка: Подумай, где нужна гибкость без изменения исходного класса.

Ответ:

- **Добавление логирования к запросу** — Позволяет фиксировать действия без изменения кода запроса
- **Шифрование данных перед отправкой** — Позволяет оборачивать сообщение в защитный слой
- **Добавление кеширования к сервису** — Сохраняет результат выполнения и уменьшает нагрузку
- **Декоратор скидки в корзине** — Расширяет поведение товара динамическим образом

Шаг 07:

Миссия 7

Сопоставьте шаблон проектирования и его ключевую особенность.

Подсказка: Вспомни: декоратор — гибкость, адаптер — совместимость, фасад — упрощение.

Ответ:

- **Декоратор** — Позволяет динамически расширять функционал
- **Адаптер** — Приводит несовместимые интерфейсы к одному виду
- **Фасад** — Упрощает доступ к сложной системе
- **Стратегия** — Позволяет выбирать алгоритм во время выполнения

Шаг 08:

Миссия 8

Программист создает систему для ресторана. Нужно сопоставить декораторы с их эффектом.

Подсказка: Вспомни, что каждый декоратор — это дополнительный «слой» возможностей.

Ответ:

- **Decorator «Соус»** — Добавляет соус к блюду
- **Decorator «Сыр»** — Добавляет сыр поверх основы
- **Decorator «Скидка»** — Снижает цену заказа
- **Decorator «Доставка»** — Добавляет плату и функцию доставки

🔮 Шаг 09:

Миссия 9

Сопоставьте понятия из шаблона «Декоратор».

Подсказка: Есть базовый интерфейс, конкретные реализации и обертки.

Ответ:

- **Component** — Общий интерфейс для объектов
- **ConcreteComponent** — Основная реализация объекта
- **Decorator** — Абстрактная обертка, реализующая интерфейс
- **ConcreteDecorator** — Реальная добавка, расширяющая поведение

🔮 Шаг 10:

Миссия 10

Разработчик хочет понять, где уместен декоратор, а где лучше другой паттерн. Сопоставь ситуацию и правильный выбор.

Подсказка: Декоратор — это про расширение без изменения.

Ответ:

- **Нужно динамически добавлять функционал** — Использовать Декоратор
- **Нужно скрыть сложность подсистемы** — Использовать Фасад
- **Нужно объединить разные интерфейсы** — Использовать Адаптер
- **Нужно выбирать алгоритм на лету** — Использовать Стратегию

🔮 Шаг 11:

Миссия 11

В проекте по обработке файлов программист хочет добавлять новые фильтры. Сопоставь фильтр с его действием.

Подсказка: Каждый фильтр — это отдельный слой вокруг файла.

Ответ:

- **Декоратор «Сжатие»** — Уменьшает размер файла
- **Декоратор «Шифрование»** — Делает данные недоступными без ключа
- **Декоратор «Логирование»** — Фиксирует операции с файлом
- **Декоратор «Кэширование»** — Сохраняет результат для ускорения повторного доступа

🔮 Шаг 12:

Миссия 12

Сопоставь декоратор в чате с его эффектом.

Подсказка: Представь, что каждый декоратор добавляет «спецэффект» к сообщению.

Ответ:

- **Decorator «Emoji»** — Добавляет смайлики к тексту
- **Decorator «Цвет»** — Меняет оформление сообщений
- **Decorator «Вложения»** — Позволяет прикреплять файлы
- **Decorator «Уведомления»** — Отправляет пуш-оповещение при новом сообщении

🔮 Шаг 13:

Миссия 13

Расставьте шаги создания декоратора в правильном порядке.

Подсказка: Сначала общий интерфейс, потом реализация, затем обертка и расширение.

Ответ:

- Создать общий интерфейс (Component)
- Создать конкретный класс (ConcreteComponent)
- Создать абстрактный класс-декоратор (Decorator)
- Создать конкретный декоратор (ConcreteDecorator)

Шаг 14:

Миссия 14

Программист применяет декораторы для оплаты: базовый платеж, затем добавляются комиссии. Расположите шаги.

Подсказка: Начинаем с основного объекта, потом «оборачиваем» его комиссиями.

Ответ:

- Создание базового объекта Payment
- Добавление декоратора TaxDecorator
- Добавление декоратора CommissionDecorator
- Вызов метода pay()

Шаг 15:

Миссия 15

Определи порядок действий при добавлении нового декоратора в систему уведомлений.

Подсказка: Сначала базовый функционал, потом дополнительный слой.

Ответ:

- Создать класс BaseNotifier
- Создать декоратор SMSDecorator
- Создать декоратор PushDecorator
- Отправить уведомление notify()

Шаг 16:

Миссия 16

Упорядочите шаги применения декоратора для класса FileReader.

Подсказка: Каждый шаг накладывает дополнительный слой обработки.

Ответ:

- Создание объекта FileReader
- Оборачивание в EncryptionDecorator
- Оборачивание в CompressionDecorator
- Вызов метода read()

Урок 7 - Dependency Injection — Контроль силы

Шаг 01:

Миссия 1

Ты пишешь сервис для работы с платежами. Как лучше всего внедрить объект Logger в PaymentService, чтобы не привязываться жестко к конкретной реализации?

Подсказка: Вспомни принцип инверсии зависимостей: класс не должен сам создавать свои зависимости.

Ответ:

- **Передать Logger через конструктор PaymentService**
- Создать new Logger() внутри метода pay()
- Сделать Logger статическим классом и обращаться к нему напрямую
- Хранить Logger в глобальной переменной

Шаг 02:

Миссия 2

Разработчик Дима пишет тесты. Ему нужно подменить базу данных на фейковую. Как Dependency Injection может помочь?

Подсказка: Подумай про интерфейсы и возможность подставить разные реализации.

Ответ:

- **Передавать в сервис абстракцию базы и в тестах подставлять фейковую реализацию**
- Жестко привязать код к MySQL через new PDO()
- Создать глобальную переменную с тестовой базой
- Использовать include с мок-реализацией в тестах

Шаг 03:

Миссия 3

Тебе нужно внедрить EmailSender в класс UserNotifier. Какой способ внедрения зависимости считается наиболее гибким?

Подсказка: Вспомни, где можно легко заменить реализацию.

Ответ:

- **Через конструктор**
- Через приватный метод внутри UserNotifier
- Жестко создавать в каждом методе new EmailSender()
- Хранить EmailSender в статическом свойстве

Шаг 04:

Миссия 4

Программист Анна использует фреймворк с контейнером зависимостей. Какую задачу решает контейнер?

Подсказка: Он управляет объектами вместо нас.

Ответ:

- **Автоматически создает и передает зависимости в классы**
- Запускает PHP-код в отдельном процессе
- Следит за логами ошибок
- Заменяет PHP-интерпретатор

Шаг 05:

Миссия 5

В проекте добавили десятки сервисов. Как лучше всего упростить управление их зависимостями?

Подсказка: Подумай про единый центр создания объектов.

Ответ:

- **Использовать контейнер внедрения зависимостей**
- Создавать сервисы через `new` в каждом контроллере
- Хранить сервисы в глобальном массиве
- Писать отдельный `include` для каждой зависимости

🔗 Шаг 06:

Миссия 6

Сопоставь типы внедрения зависимостей с их характеристиками.

Подсказка: Разные способы внедрения дают разный уровень контроля.

Ответ:

- **Конструкторное внедрение** — Зависимости передаются при создании объекта
- **Сеттерное внедрение** — Зависимости передаются через методы после создания
- **Интерфейсное внедрение** — Класс реализует метод для получения зависимостей
- **Локальное создание** — Объект сам создает свои зависимости через `new`

🔗 Шаг 07:

Миссия 7

Сопоставь плюсы и минусы разных видов внедрения зависимостей.

Подсказка: Вспомни, что гибкость и тестируемость часто идут рука об руку.

Ответ:

- **Конструкторное внедрение** — Гарантирует наличие зависимости, удобно для тестов
- **Сеттерное внедрение** — Позволяет менять зависимость в любой момент
- **Интерфейсное внедрение** — Требуется реализации доп. методов, увеличивает связность
- **Локальное создание** — Затрудняет тестирование, жесткая привязка

🔗 Шаг 08:

Миссия 8

Программист Саша разбирает код и видит разные подходы. Сопоставь стиль с примером.

Подсказка: Смотри на ключевые слова `new` и передачу через параметры.

Ответ:

- **Конструкторное внедрение** — `class Service { public function __construct(Logger $logger) {...} }`
- **Сеттерное внедрение** — `$service->setLogger(new Logger());`
- **Интерфейсное внедрение** — `interface Injectable { public function inject($dep); }`
- **Локальное создание** — `$this->logger = new Logger();`

🔗 Шаг 09:

Миссия 9

Внедрение зависимостей связано с принципами SOLID. Сопоставь принцип с его описанием.

Подсказка: Особенно обрати внимание на букву D.

Ответ:

- **Single Responsibility** — Класс выполняет только одну задачу
- **Open/Closed** — Классы открыты для расширения, но закрыты для модификации
- **Dependency Inversion** — Модули верхнего уровня не зависят от модулей нижнего уровня
- **Liskov Substitution** — Наследники должны полностью заменять родителей

🔗 Шаг 10:

Миссия 10

Ты смотришь код коллеги. Сопоставь плохую практику с ее последствием.

Подсказка: Вспомни про тесты и гибкость архитектуры.

Ответ:

- **Создание зависимостей через new внутри класса** — Сложно тестировать и подменять
- **Использование глобальных переменных** — Непрозрачность и скрытые зависимости
- **Жесткая привязка к конкретной реализации** — Сложность расширения
- **Отсутствие интерфейсов** — Невозможно подменить реализацию

🔗 Шаг 11:

Миссия 11

Сопоставь фреймворк с его способом работы с зависимостями.

Подсказка: Некоторые используют автосборку, другие требуют конфигурацию.

Ответ:

- **Symfony** — Использует сервис-контейнер с конфигурацией YAML/XML/PHP
- **Laravel** — Автоматическая инъекция через контейнер и рефлексия
- **Zend Framework** — Массивная конфигурация зависимостей
- **Yii2** — DI-контейнер с настройками компонентов

🔗 Шаг 12:

Миссия 12

Сопоставь ситуацию с преимуществом Dependency Injection.

Подсказка: Вспомни про тесты, гибкость и повторное использование.

Ответ:

- **Нужно протестировать класс без подключения реальной базы** — Подмена зависимостей на моки
- **Проект растет и появляются новые реализации логгера** — Гибкость и расширяемость
- **Нужно легко менять конфигурацию сервисов** — Централизованное управление через контейнер
- **Команда работает над большим приложением** — Уменьшение связности

🔗 Шаг 13:

Миссия 13

Расставь шаги настройки DI-контейнера в правильном порядке.

Подсказка: Сначала определяем зависимости, потом регистрируем, потом используем.

Ответ:

- Определение зависимостей классов

- Регистрация сервисов в контейнере
- Получение объектов из контейнера
- Использование сервисов в приложении

Шаг 14:

Миссия 14

Расставь шаги при тестировании класса с DI.

Подсказка: Помни, сначала создаем заглушки, потом передаем, потом проверяем.

Ответ:

- Создание мок-объектов
- Передача моков через конструктор
- Вызов метода тестируемого класса
- Проверка результатов

Шаг 15:

Миссия 15

Расставь шаги при добавлении нового сервиса в контейнер.

Подсказка: Логика схожа с регистрацией зависимостей.

Ответ:

- Создание класса сервиса
- Определение зависимостей сервиса
- Регистрация сервиса в контейнере
- Использование сервиса

Шаг 16:

Миссия 16

Расположи шаги работы Dependency Injection в приложении.

Подсказка: Вспомни жизненный цикл объекта.

Ответ:

- Определение интерфейсов зависимостей
- Реализация зависимостей
- Регистрация зависимостей в контейнере
- Создание и использование объектов

Модуль 4 - «Темные подземелья» — Ошибки, тесты и исключения

Урок 1 - Исключения Искусство ловить ошибки

Шаг 01:

Миссия 1

Программист Артём написал скрипт для обработки заказов, но при неверных данных программа просто падает. Как правильно в PHP перехватывать ошибку?

Подсказка: Вспомни про конструкции try и catch — они как сеть для ловли ошибок.

Ответ:

- **Обернуть код в try...catch**
- Использовать echo внутри функции
- Добавить больше комментариев
- Спрятать ошибку в переменную

🔗 **Шаг 02:**

Миссия 2

Марина тестирует API. В коде выбрасывается `InvalidArgumentException`. Что это за механизм? Подсказка: Такие объекты сигнализируют, что что-то пошло не так в работе программы.

Ответ:

- **Исключение (Exception)**
- Функция обратного вызова
- Класс модели
- Строка с ошибкой

🔗 **Шаг 03:**

Миссия 3

Вася написал код, который ловит исключение, но внутри catch он просто оставил пустой блок. К чему это приведёт?

Подсказка: Подумай, что будет, если ошибка произойдёт, но программист никак на неё не отреагирует.

Ответ:

- **Ошибка будет скрыта, что затруднит отладку**
- Программа автоматически исправит ошибку
- Исключение исчезнет навсегда
- Система завершит работу корректно всегда

🔗 **Шаг 04:**

Миссия 4

Алина запустила сложный импорт данных. Её код может выбросить разные исключения. Как можно обработать их по-разному?

Подсказка: У каждой проблемы свой catch.

Ответ:

- **Использовать несколько блоков catch для разных классов исключений**
- Добавить один общий catch на всё
- Переписать try без catch
- Сохранять все ошибки в массив

🔗 **Шаг 05:**

Миссия 5

Степан решил перехватить ошибку подключения к БД. Какое исключение чаще всего связано с PDO?

Подсказка: Подумай о классе, который бросает ошибки при работе с базой данных.

Ответ:

- **PDOException**
- RuntimeException
- DatabaseError
- SqlCrash

Шаг 06:

Миссия 6

Никита ловит исключение, но хочет в любом случае закрыть файл. Какой блок ему нужен?

Подсказка: Этот блок выполняется всегда, независимо от ошибки.

Ответ:

- **finally**
- end
- always
- return

Шаг 07:

Миссия 7

Разработчик Петя решил создать собственный тип ошибки. Как правильно определить пользовательское исключение?

Подсказка: Нужно унаследоваться от базового класса.

Ответ:

- **Создать класс, наследующий Exception**
- Создать функцию с названием Exception
- Объявить глобальную переменную error
- Написать echo при ошибке

Шаг 08:

Миссия 8

 Задача: сопоставьте типы встроенных исключений с их описанием.

Подсказка: Вспомните, какие ошибки связаны с логикой, а какие с работой окружения.

Ответ:

- **InvalidArgumentException** — Выбрасывается при передаче некорректного аргумента
- **RuntimeException** — Ошибка, возникающая во время выполнения программы
- **OutOfRangeException** — Выход индекса или значения за допустимые пределы
- **DomainException** — Ошибка в логике программы, неверный домен значений

Шаг 09:

Миссия 9

Сопоставьте элементы try/catch с их назначением.

Подсказка: Каждый блок выполняет свою роль: один ловит, другой выбрасывает.

Ответ:

- **try** — Код, в котором может произойти ошибка
- **catch** — Блок, обрабатывающий исключение

- **throw** — Ключевое слово для генерации исключения
- **finally** — Код, который выполняется всегда

Шаг 10:

Миссия 10

Программист Саша отлаживает код и хочет различать виды ошибок. Сопоставьте исключения с реальными примерами.

Подсказка: Представьте себе ситуации: неправильный аргумент, неверный индекс, ошибка окружения.

Ответ:

- **InvalidArgumentException** — Функция ожидает число, а пришла строка
- **OutOfBoundsException** — Попытка обратиться к элементу массива по несуществующему индексу
- **RuntimeException** — Ошибка подключения к внешнему сервису
- **LengthException** — Передана строка недопустимой длины

Шаг 11:

Миссия 11

Разработчик создал собственный класс исключения. Сопоставьте элементы определения с их назначением.

Подсказка: Вспомните структуру класса в PHP.

Ответ:

- **class MyException extends Exception** — Определение нового класса исключения
- **__construct** — Метод для инициализации объекта исключения
- **parent::__construct** — Вызов конструктора родительского класса
- **throw new MyException** — Генерация пользовательского исключения

Шаг 12:

Миссия 12

Сопоставьте исключения с типичными библиотеками/областями их возникновения.

Подсказка: Обрати внимание: PDO связано с БД, а логические — с проверкой условий.

Ответ:

- **PDOException** — Ошибки при работе с базой данных
- **LogicException** — Ошибки в логике приложения
- **UnexpectedValueException** — Получено значение, которое не ожидалось
- **BadMethodCallException** — Вызов метода, которого не существует

Шаг 13:

Миссия 13

Расположите шаги обработки исключения в правильном порядке.

Подсказка: Сначала нужно подготовить код, затем бросить исключение, поймать его и обработать.

Ответ:

- Написать код в блоке try
- Сгенерировать исключение throw

- Перехватить его в `catch`
- Выполнить код в `finally`

Шаг 14:

Миссия 14

Разработчик Антон пишет unit-тесты. Расставьте действия при тестировании исключений в верном порядке.

Подсказка: Подумайте, как PHPUnit проверяет, что исключение действительно выбрасывается.

Ответ:

- Написать тестовый метод
- Указать ожидаемое исключение
- Вызвать тестируемую функцию
- Убедиться, что исключение поймано

Шаг 15:

Миссия 15

Саша проектирует систему логирования ошибок. Расставьте шаги правильной обработки.

Подсказка: Важно не только поймать, но и записать ошибку для анализа.

Ответ:

- Поймать исключение
- Записать информацию в лог
- Сообщить пользователю дружелюбное сообщение
- Продолжить выполнение программы или завершить её безопасно

Шаг 16:

Миссия 16

Вася отлаживает код. Как правильно организовать иерархию пользовательских исключений?

Подсказка: Сначала делается общий класс, потом более узкие.

Ответ:

- Создать базовый класс `AppException`
- Создать наследников для отдельных модулей
- Добавить обработку общего класса
- Добавить обработку конкретных ошибок

Шаг 17:

Миссия 17

Финальная битва: организуйте шаги обработки критической ошибки на продакшене.

Подсказка: Подумайте: поймать, зафиксировать, уведомить и только потом остановить.

Ответ:

- Перехватить исключение
- Записать ошибку в лог
- Отправить уведомление команде
- Безопасно остановить приложение

Урок 2 - Стратегии обработки ошибок — путь без боли

Шаг 01:

Миссия 1

Артём пишет сервис оплаты. При падении транзакции он думает: «Что выбрать: молча завершить или бросить исключение?». Что считается хорошей практикой?

Подсказка: Подумай: замалчивание ошибок всегда ведёт к хаосу.

Ответ:

- **Бросить исключение и обработать его**
- Скрыть ошибку от всех
- Завершить скрипт без объяснений
- Записать ошибку в переменную и забыть

Шаг 02:

Миссия 2

Марина проектирует API. Какой способ лучше при ошибке — возвращать null или выбрасывать исключение?

Подсказка: Подумай: null не объяснит причину, а исключение — да.

Ответ:

- **Выбрасывать исключение**
- Всегда возвращать null
- Печатать текст ошибки в консоль
- Игнорировать ошибку

Шаг 03:

Миссия 3

Вася пишет библиотеку. Он сомневается: обрабатывать исключение прямо в классе или передать его выше? Что гибче?

Подсказка: Локальная обработка ограничивает пользователя библиотеки.

Ответ:

- **Пробросить исключение выше**
- Всегда гасить исключение внутри
- Сохранять ошибку в echo
- Заменить исключение на return 0

Шаг 04:

Миссия 4

Алина пишет микросервис и выбирает стратегию обработки ошибок. Что важно для прозрачности?

Подсказка: Логирование помогает команде понять, что пошло не так.

Ответ:

- **Логировать ошибки и исключения**
- Удалять ошибки из памяти
- Молча завершать сервис
- Игнорировать ошибки при тестировании

Шаг 05:

Миссия 5

Степан делает e-commerce. Что лучше сообщать пользователю при ошибке оплаты?

Подсказка: Пользователю нужно дружелюбное сообщение, а детали — только в логах.

Ответ:

- **Показать общее сообщение и записать детали в лог**
- Вывести stack trace на экран
- Молча завершить заказ
- Отправить SQL-запрос пользователю

Шаг 06:

Миссия 6

Никита делает библиотеку. Как обеспечить единый стиль обработки ошибок в проекте?

Подсказка: Централизованная обработка экономит нервы.

Ответ:

- **Создать глобальный обработчик исключений**
- Добавлять echo в каждом catch
- Скрывать ошибки в комментариях
- Игнорировать исключения

Шаг 07:

Миссия 7

Петя проектирует сервис доставки. Что лучше использовать для предсказуемости: коды ошибок или исключения?

Подсказка: Коды ошибок путают, исключения структурируют обработку.

Ответ:

- **Исключения**
- Массив кодов ошибок
- Глобальную переменную error_code
- Возврат -1 при ошибке

Шаг 08:

Миссия 8

 Задача: Сопоставьте стратегию обработки ошибок с её описанием.

Подсказка: Представь разные уровни — от молчания до централизованного контроля.

Ответ:

- **Игнорирование ошибок** — Полное замалчивание, код работает как будто ничего не произошло
- **Локальная обработка** — Исключение ловится в конкретном месте
- **Глобальный обработчик** — Единая точка для всех ошибок
- **Логирование** — Запись ошибок для дальнейшего анализа

Шаг 09:

Миссия 9

Сопоставьте исключение с правильной стратегией обработки.

Подсказка: Подумай: системные ошибки требуют особого подхода.

Ответ:

- **PDOException** — Логирование и показ дружелюбного сообщения
- **InvalidArgumentException** — Исправление кода и тесты
- **RuntimeException** — Обработка в глобальном обработчике
- **DomainException** — Анализ бизнес-логики

Шаг 10:

Миссия 10

Сопоставьте подходы с их преимуществами.

Подсказка: Один даёт гибкость, другой — предсказуемость.

Ответ:

- **Исключения** — Позволяют структурировать обработку ошибок
- **Коды возврата** — Просты, но приводят к дублированию проверок
- **Глобальный обработчик** — Централизованный контроль
- **Логирование** — История проблем для анализа

Шаг 11:

Миссия 11

Сопоставьте ошибки с действиями пользователя.

Подсказка: Представь: если сломалась БД — что делать? А если пользователь ввёл ерунду?

Ответ:

- **Ошибка БД** — Показать пользователю сообщение о технических проблемах
- **Ошибка валидации** — Попросить пользователя исправить данные
- **Системная ошибка** — Сообщить администратору
- **Ошибка API** — Записать в лог и вернуть fallback-ответ

Шаг 12:

Миссия 12

Сопоставьте стратегии с типичными проектами.

Подсказка: Большие системы требуют глобального подхода.

Ответ:

- **Маленький скрипт** — Локальная обработка исключений
- **Большое веб-приложение** — Глобальный обработчик и логирование
- **Микросервис** — Дружелюбные сообщения + централизованный лог
- **Библиотека** — Проброс исключений выше

Шаг 13:

Миссия 13

Расставьте шаги грамотной обработки ошибки.

Подсказка: Сначала поймать, потом зафиксировать, затем уведомить и вернуть управление.

Ответ:

- Перехватить исключение

- Записать в лог
- Отправить уведомление команде
- Сообщить пользователю

🔮 Шаг 14:

Миссия 14

Разработчик тестирует приложение. Расставьте шаги стратегии «Fail Fast».

Подсказка: Ошибки должны всплывать сразу.

Ответ:

- Валидировать входные данные
- Бросить исключение при ошибке
- Завершить выполнение опасного блока
- Логировать проблему

🔮 Шаг 15:

Миссия 15

Саша проектирует централизованный обработчик. Расставьте шаги.

Подсказка: Центр всегда получает исключение первым.

Ответ:

- Поймать все необработанные исключения
- Записать их в лог
- Отправить уведомление
- Вернуть дружелюбный ответ пользователю

🔮 Шаг 16:

Миссия 16

Антон пишет систему мониторинга. Как расположить шаги обработки критической ошибки?

Подсказка: Сначала зафиксировать, потом оповестить, а затем безопасно остановить.

Ответ:

- Записать ошибку в лог
- Отправить уведомление DevOps
- Сделать graceful shutdown
- Показать пользователю техническую страницу

🔮 Шаг 17:

Миссия 17

Финальная битва: организуйте стратегию обработки ошибок в распределённой системе.

Подсказка: Не забудьте про централизованный лог и уведомления.

Ответ:

- Перехват ошибки на уровне сервиса
- Записать в централизованный лог
- Отправить уведомление в Slack
- Вернуть пользователю общее сообщение

Урок 3 - PHPUnit — Клинок автоматизации

Шаг 01:

Миссия 1

Вы работаете над новым проектом и коллега спрашивает: зачем вообще нужны unit-тесты с PHPUnit?

Подсказка: Тесты помогают избежать ручной проверки каждого изменения.

Ответ:

- **Чтобы автоматически проверять корректность работы кода**
- Чтобы заменить документацию
- Чтобы ускорить работу сервера
- Чтобы исключить необходимость писать комментарии

Шаг 02:

Миссия 2

Молодой разработчик написал тест, но не понимает, почему он всегда зелёный. Что чаще всего является причиной?

Подсказка: Иногда тест просто ничего не проверяет.

Ответ:

- **Отсутствуют assert-методы**
- Используется слишком много assert-методов
- Неправильное имя класса теста
- Тест запускается без phpunit.xml

Шаг 03:

Миссия 3

Разработчик спорит с тимлидом: чем отличаются assertEquals и assertEquals?

Подсказка: Один метод проверяет только значения, другой — ещё и тип.

Ответ:

- **assertEquals сравнивает значения, assertEquals сравнивает ещё и типы**
- assertEquals работает только с числами
- assertEquals работает только со строками
- Оба метода полностью одинаковые

Шаг 04:

Миссия 4

Во время тестирования API вы хотите проверить, что метод бросает исключение. Какой assert использовать?

Подсказка: PHPUnit умеет ожидать исключения.

Ответ:

- **\$this->expectException(...)**
- assertEquals()
- catchException()
- throwException()

🔮 Шаг 05:

Миссия 5

Ваш коллега утверждает, что PHPUnit можно интегрировать в CI/CD. Что это даст?

Подсказка: Подумайте про автоматические проверки перед деплоем.

Ответ:

- **Автоматический прогон тестов при каждом коммите**
- Автоматическая генерация UML-диаграмм
- Замена кода на продакшене
- Отправка писем клиентам

🔮 Шаг 06:

Миссия 6

Разработчик заметил, что его тесты очень медленные. Какую стратегию оптимизации выбрать?

Подсказка: Долгие тесты часто связаны с базой данных или внешними API.

Ответ:

- **Мокирование зависимостей**
- Добавление новых assert-методов
- Запуск тестов только ночью
- Использование echo для логов

🔮 Шаг 07:

Миссия 7

📄 Сопоставьте термины PHPUnit с их правильным описанием.

Подсказка: Подумайте о назначении каждого элемента теста.

Ответ:

- **Fixture** — Подготовка окружения для теста
- **Mock** — Поддельный объект, заменяющий реальную зависимость
- **Test Suite** — Набор тестов, запускаемых вместе
- **Assertion** — Проверка условия в тесте

🔮 Шаг 08:

Миссия 8

Сопоставьте команды с их назначением в PHPUnit.

Подсказка: Каждая команда отвечает за конкретное действие при запуске.

Ответ:

- **--filter** — Запуск только определённых тестов
- **--coverage-html** — Генерация отчёта о покрытии кода
- **--colors** — Включение цветного вывода
- **--testdox** — Отображение тестов в виде документации

🔮 Шаг 09:

Миссия 9

Сопоставьте методы PHPUnit с их значением.

Подсказка: У каждого метода своя роль: проверка равенства, пустоты, истины и т.д.

Ответ:

- **assertTrue** — Проверяет, что условие истинно
- **assertEmpty** — Проверяет, что значение пустое
- **assertCount** — Сравнивает количество элементов
- **assertInstanceOf** — Проверяет, что объект принадлежит классу

Шаг 10:

Миссия 10

Сопоставьте уровни тестирования с их описанием.

Подсказка: Вспомните, чем unit, integration и system отличаются друг от друга.

Ответ:

- **Unit-тесты** — Проверяют отдельные методы или классы
- **Интеграционные тесты** — Проверяют взаимодействие нескольких компонентов
- **Системные тесты** — Проверяют приложение целиком
- **Регрессионные тесты** — Проверяют, что старый функционал работает после изменений

Шаг 11:

Миссия 11

Сопоставьте хуки PHPUnit с их назначением.

Подсказка: Эти методы вызываются автоматически перед или после тестов.

Ответ:

- **setUp** — Вызывается перед каждым тестом
- **tearDown** — Вызывается после каждого теста
- **setUpBeforeClass** — Вызывается один раз перед всеми тестами класса
- **tearDownAfterClass** — Вызывается один раз после всех тестов класса

Шаг 12:

Миссия 12

Расположите шаги написания unit-теста в правильном порядке.

Подсказка: Сначала создаём окружение, потом вызываем метод, а затем проверяем результат.

Ответ:

- Подготовка данных
- Вызов метода
- Сравнение результата с ожиданием
- Очистка окружения

Шаг 13:

Миссия 13

Определите порядок работы PHPUnit при запуске тестов.

Подсказка: Вспомните про хуки и порядок их выполнения.

Ответ:

- setUpBeforeClass
- setUp

- Тестовый метод
- tearDown
- tearDownAfterClass

Шаг 14:

Миссия 14

Расположите шаги при использовании мок-объектов в правильном порядке.

Подсказка: Сначала нужно создать поддельный объект, потом описать его поведение.

Ответ:

- Создание мока
- Настройка поведения
- Передача в тестируемый класс
- Вызов метода
- Проверка результатов

Шаг 15:

Миссия 15

Расположите шаги настройки PHPUnit в новом проекте.

Подсказка: Сначала установка пакета, затем конфигурация, потом запуск.

Ответ:

- Установка через Composer
- Создание phpunit.xml
- Написание первого теста
- Запуск PHPUnit

Шаг 16:

Миссия 16

Расставьте шаги TDD (Test-Driven Development) в правильной последовательности.

Подсказка: Вспомните цикл: красный — зелёный — рефакторинг.

Ответ:

- Написание теста
- Запуск теста (красный)
- Реализация минимального кода (зелёный)
- Рефакторинг

Урок 4 - Моки и стабы Призрачные союзники

Шаг 01:

Миссия 1

Ты пишешь юнит-тесты для сервиса оплаты. В тесте нужно подменить внешний API банка. Что ты используешь?

Подсказка: Призрачный союзник помогает, когда реальный объект слишком сложен или дорог в использовании.

Ответ:

- **Mock или Stub**

- Singleton
- Observer
- Trait

🔗 Шаг 02:

Миссия 2

Твоя команда тестирует сервис доставки пиццы. Чтобы ускорить тесты, вместо реального курьера используется Stub. Что делает Stub?

Подсказка: Он не проверяет поведение, а лишь возвращает заранее подготовленные ответы.

Ответ:

- **Подставляет фиксированные ответы на запросы**
- Следит за вызовами методов
- Логирует ошибки в консоль
- Изменяет поведение тестируемого класса

🔗 Шаг 03:

Миссия 3

Ты пишешь тест для почтового сервиса. Нужно проверить, что метод `send()` был вызван. Какой объект использовать?

Подсказка: Здесь важно зафиксировать сам факт вызова, а не вернуть данные.

Ответ:

- **Mock**
- Stub
- Entity
- Repository

🔗 Шаг 04:

Миссия 4

Программистка Катя проверяет метод авторизации. Реальный сервис базы данных слишком тяжёлый для теста. Что ей стоит использовать?

Подсказка: Временный подменный объект облегчит нагрузку на систему.

Ответ:

- **Stub для базы данных**
- Абстрактный класс
- Фасад
- Финальный класс

🔗 Шаг 05:

Миссия 5

Тестировщик Дима использует Mock, чтобы проверить, что метод `notify()` вызван ровно 3 раза. Почему Stub тут не подойдёт?

Подсказка: Один из призрачных союзников умеет хранить информацию о количестве вызовов.

Ответ:

- **Stub не проверяет количество вызовов**
- Stub требует подключения базы

- Stub слишком медленный
- Stub возвращает случайные данные

🔮 Шаг 06:

Миссия 6

Сопоставь тип подменного объекта с его назначением.

Подсказка: Mock проверяет поведение, Stub — возвращает ответы.

Ответ:

- **Stub** — Возвращает заранее подготовленные данные
- **Mock** — Проверяет, что методы были вызваны
- **Fake** — Упрощённая рабочая реализация
- **Dummy** — Передаётся в метод, но не используется

🔮 Шаг 07:

Миссия 7

Программист Иван использует PHPUnit. Сопоставь методы Mock-объекта с их назначением.

Подсказка: Методы Mock-а помогают контролировать количество вызовов и параметры.

Ответ:

- **expects()** — Устанавливает ожидания для вызовов
- **with()** — Проверяет переданные аргументы
- **willReturn()** — Определяет возвращаемое значение
- **once()** — Указывает, что метод вызван ровно один раз

🔮 Шаг 08:

Миссия 8

Сопоставь тип теста с его кратким описанием.

Подсказка: Моки и стабы чаще всего встречаются в юнит-тестах.

Ответ:

- **Unit-тест** — Проверяет работу отдельного метода или класса
- **Integration-тест** — Проверяет взаимодействие нескольких компонентов
- **System-тест** — Проверяет систему целиком
- **Acceptance-тест** — Проверяет требования заказчика

🔮 Шаг 09:

Миссия 9

В лаборатории тестирования есть разные артефакты. Сопоставь объект с правильным примером использования.

Подсказка: Dummy бесполезен сам по себе, Fake делает упрощённую работу.

Ответ:

- **Dummy** — Передаётся только для соответствия сигнатуре конструктора
- **Fake** — Упрощённая реализация БД в памяти
- **Stub** — Возвращает фиксированный email пользователя
- **Mock** — Проверяет, что вызван метод save()

🔗 Шаг 10:

Миссия 10

Сопоставь ситуацию из жизни команды с нужным тестовым двойником.

Подсказка: Думай о том, проверяем ли мы поведение или просто хотим подставить ответ.

Ответ:

- **Проверка вызова метода `log()`** — Mock
- **Возврат фиксированного курса валюты** — Stub
- **Хранение пользователей в массиве вместо базы** — Fake
- **Передача пустого объекта в тест** — Dummy

🔗 Шаг 11:

Миссия 11

Сопоставь инструмент PHPUnit с его назначением.

Подсказка: Разные методы помогают по-разному управлять ожиданиями в тестах.

Ответ:

- **`createMock()`** — Создаёт поддельный объект
- **`expects()`** — Определяет ожидания вызовов
- **`withConsecutive()`** — Проверяет последовательность аргументов
- **`willReturnMap()`** — Задаёт значения для разных аргументов

🔗 Шаг 12:

Миссия 12

Расположи шаги написания теста с использованием Stub в правильном порядке.

Подсказка: Сначала создаём объект, затем настраиваем ответы, после — выполняем тест.

Ответ:

- Создание Stub
- Определение возвращаемого значения
- Вызов метода тестируемого класса
- Проверка результата

🔗 Шаг 13:

Миссия 13

Упорядочи шаги работы с Mock-объектом.

Подсказка: Сначала формируем ожидания, потом выполняем метод, в конце — проверка.

Ответ:

- Создание Mock
- Задание ожиданий вызова
- Вызов метода
- Проверка, что ожидания выполнены

🔗 Шаг 14:

Миссия 14

Размести этапы тестирования микросервиса оплаты с использованием Fake.

Подсказка: Fake заменяет сложную систему простой реализацией.

Ответ:

- Создать Fake-реализацию
- Подключить Fake в сервис
- Выполнить тестовый сценарий
- Сравнить результат с ожидаемым

 **Шаг 15:**

Миссия 15

Ты тестируешь сервис отправки писем. Расположи шаги работы с Dummy-объектом.

Подсказка: Dummy нужен, чтобы заполнить обязательный параметр, но сам он не используется.

Ответ:

- Создать Dummy
- Передать его в конструктор
- Запустить метод сервиса
- Убедиться, что Dummy не участвовал в логике

 **Шаг 16:**

Миссия 16

Команда пишет тесты для сервиса бронирования. Расставь шаги применения разных тестовых двойников.

Подсказка: Помни: Dummy — для параметров, Stub — для данных, Mock — для проверки вызовов.

Ответ:

- Создать Dummy для конструктора
- Подключить Stub для внешнего API
- Задать ожидания Mock
- Запустить тест и проверить результат

Урок 5 - Тестирование архитектуры Проверка крепости

 **Шаг 01:**

Миссия 1

Архитектор Виталий проверяет слой сервисов в проекте. Что должен тестировать архитектурный тест?

Подсказка: В таких тестах важна структура и зависимости, а не конкретная бизнес-логика.

Ответ:

- **Соответствие зависимостей архитектурным правилам**
- Скорость выполнения SQL-запросов
- Количество комментариев в коде
- Уровень покрытия тестами

 **Шаг 02:**

Миссия 2

В проекте микросервисов команда хочет убедиться, что контроллеры не зависят напрямую от базы данных. Какой тест нужен?

Подсказка: Думай о защите крепости: стены не должны опираться напрямую на сокровищницу.

Ответ:

- **Архитектурный тест зависимостей**
- Нагрузочный тест
- Интеграционный тест
- Снапшот-тест

Шаг 03:

Миссия 3

Разработчица Оля проверяет, что классы в пространстве имён Domain не зависят от Infrastructure . Что это за правило?

Подсказка: Слои крепости должны быть выстроены так, чтобы внутренняя часть не зависела от внешней.

Ответ:

- **Принцип зависимостей в архитектуре слоёв**
- Принцип единой ответственности
- Принцип подстановки Барбары Лисков
- Принцип инверсии управления

Шаг 04:

Миссия 4

Тимлид Сергей хочет быть уверенным, что все контроллеры лежат в папке App/Controller . Какой вид проверки он использует?

Подсказка: Иногда проверка архитектуры сводится к соблюдению соглашений о расположении.

Ответ:

- **Архитектурный тест структуры каталогов**
- Функциональный тест UI
- Юнит-тест
- Smoke-тест

Шаг 05:

Миссия 5

Разработчик Петя заметил, что класс в слое Application напрямую вызывает SQL. Что нарушено?

Подсказка: В крепости Application не должен копаться в подземельях Infrastructure.

Ответ:

- **Границы архитектурных слоёв**
- Нормализация базы данных
- Правило DRY
- Dependency Injection

Шаг 06:

Миссия 6

Компания внедрила слой Domain , чтобы тестировать бизнес-логику отдельно. Какой плюс дают такие архитектурные тесты?

Подсказка: Чем крепче стены, тем проще защитить ядро.

Ответ:

- **Можно тестировать бизнес-логику без инфраструктуры**
- Код быстрее компилируется
- Базы данных работают надёжнее
- Уменьшается размер класса

Шаг 07:

Миссия 7

Сопоставь слой архитектуры с его назначением.

Подсказка: Слои крепости выполняют разные роли: ядро хранит правила, внешние — инфраструктуру.

Ответ:

- **Domain** — Содержит бизнес-логику и правила
- **Application** — Орkestрирует использование бизнес-логики
- **Infrastructure** — Содержит внешние сервисы и базы данных
- **Presentation** — Отвечает за пользовательский интерфейс

Шаг 08:

Миссия 8

Сопоставь принцип SOLID с его значением в архитектуре.

Подсказка: Эти принципы укрепляют стены крепости от разрушения.

Ответ:

- **Single Responsibility** — У класса должна быть только одна причина для изменения
- **Open/Closed** — Класс открыт для расширения, закрыт для изменения
- **Liskov Substitution** — Наследники должны быть взаимозаменяемы с родителями
- **Dependency Inversion** — Зависеть нужно от абстракций, а не реализаций

Шаг 09:

Миссия 9

Сопоставь тип теста с его архитектурной целью.

Подсказка: Не каждый тест проверяет поведение классов — некоторые следят за стенами крепости.

Ответ:

- **Архитектурный тест** — Следит за зависимостями между слоями
- **Интеграционный тест** — Проверяет взаимодействие компонентов
- **Unit-тест** — Тестирует отдельный класс или метод
- **E2E-тест** — Проверяет систему целиком

Шаг 10:

Миссия 10

Сопоставь инструмент с его использованием в архитектурных тестах.

Подсказка: Эти инструменты помогают проверять стены крепости автоматически.

Ответ:

- **PHPUnit** — Фреймворк для архитектурных правил
- **PHPUnit** — Фреймворк юнит-тестирования
- **Behat** — Инструмент для BDD-тестов
- **Psalm** — Статический анализатор типов

🔮 **Шаг 11:**

Миссия 11

Сопоставь нарушение архитектуры с примером из практики.

Подсказка: Подумай, какой слой на какой должен или не должен опираться.

Ответ:

- **Контроллер напрямую вызывает SQL** — Нарушение границ слоёв
- **Domain зависит от EmailSender** — Нарушение принципа зависимостей
- **Infrastructure вызывает логику бизнес-правил** — Нарушение потока управления
- **Отсутствие инверсии зависимостей** — Класс жёстко зависит от реализации

🔮 **Шаг 12:**

Миссия 12

Сопоставь архитектурное правило с его выгодой.

Подсказка: Каждое правило укрепляет определённую стену крепости.

Ответ:

- **Domain не зависит от Infrastructure** — Бизнес-логика независима от технических деталей
- **Application не содержит SQL** — Чистота оркестрации
- **Infrastructure реализует интерфейсы** — Лёгкая подмена реализации
- **Controllers не знают о слоях ниже Application** — UI изолирован от логики

🔮 **Шаг 13:**

Миссия 13

Расположи шаги настройки архитектурного теста в правильном порядке.

Подсказка: Сначала выбираем правило, потом настраиваем проверку, затем запускаем тест.

Ответ:

- Определить правило (например, Domain не зависит от Infrastructure)
- Создать тест с использованием PHPUnit
- Запустить тест
- Проверить, что он выявляет нарушения

🔮 **Шаг 14:**

Миссия 14

Расположи шаги проверки зависимостей в проекте.

Подсказка: Начни с построения карты, затем проверь правила, потом исправь.

Ответ:

- Построить граф зависимостей
- Определить архитектурные правила

- Сравнить правила с графом
- Исправить нарушения

Шаг 15:

Миссия 15

Размести этапы внедрения архитектурных тестов в команде.

Подсказка: Сначала определяем цели, потом инструменты, затем пишем тесты.

Ответ:

- Определить цели архитектурных тестов
- Выбрать инструмент (RHPat, Deptrac)
- Написать правила
- Запустить тесты в CI

Шаг 16:

Миссия 16

Расположи шаги анализа архитектурных нарушений.

Подсказка: Найди проблему, обсуди с командой, предложи решение.

Ответ:

- Обнаружить нарушение тестом
- Обсудить с командой
- Разработать план исправления
- Внедрить исправления

Шаг 17:

Миссия 17

Расположи этапы защиты архитектуры крепости проекта.

Подсказка: Вначале строим крепость, затем ставим стражу, а потом чиним проломы.

Ответ:

- Определить архитектурные границы
- Написать тесты на соблюдение границ
- Запустить их в CI/CD
- Рефакторить при нарушениях

Модуль 5 - «Арена мастеров» — Реализация, безопасность, производительность

Урок 1 - SOLID-принципы Код, стоящий столетия

Шаг 01:

Миссия 1

Вы работаете над большим проектом. Коллега спрашивает: какой принцип SOLID отвечает за то, что каждый класс должен иметь только одну причину для изменения?

Подсказка: Подумайте, какой принцип связан с ответственностью класса.

Ответ:

- **Single Responsibility Principle**
- Open/Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle

🔮 **Шаг 02:**

Миссия 2

Во время код-ревью вы заметили: коллега изменяет существующий класс, чтобы добавить новые функции. Какой принцип нарушается?

Подсказка: Думайте о принципе, который защищает от изменения старого кода и предлагает расширять поведение.

Ответ:

- **Open/Closed Principle**
- Single Responsibility Principle
- Dependency Inversion Principle
- Interface Segregation Principle

🔮 **Шаг 03:**

Миссия 3

При проектировании иерархии классов вы хотите, чтобы подклассы могли заменять родительские классы без поломки системы. Какой принцип поможет?

Подсказка: Подумайте о наследовании и взаимозаменяемости объектов.

Ответ:

- **Liskov Substitution Principle**
- Interface Segregation Principle
- Dependency Inversion Principle
- Single Responsibility Principle

🔮 **Шаг 04:**

Миссия 4

Вы проектируете интерфейсы. Один разработчик предлагает сделать огромный интерфейс с десятками методов. Какой принцип он нарушает?

Подсказка: Вспомните про разделение интерфейсов и минимализм.

Ответ:

- **Interface Segregation Principle**
- Dependency Inversion Principle
- Open/Closed Principle
- Liskov Substitution Principle

🔮 **Шаг 05:**

Миссия 5

В команде обсуждается зависимость модулей от конкретных классов. Вы предлагаете абстракции вместо деталей. Какой принцип применим?

Подсказка: Он касается инверсии зависимостей.

Ответ:

- **Dependency Inversion Principle**
- Single Responsibility Principle
- Open/Closed Principle
- Interface Segregation Principle

Шаг 06:

Миссия 6

Сопоставьте принципы SOLID с их кратким описанием.

Подсказка: Вспомните: каждый принцип отвечает за свою часть архитектуры.

Ответ:

- **Single Responsibility Principle** — Класс должен иметь только одну причину для изменения.
- **Open/Closed Principle** — Классы должны быть открыты для расширения, но закрыты для изменения.
- **Liskov Substitution Principle** — Подклассы должны быть взаимозаменяемы с родительскими классами.
- **Interface Segregation Principle** — Много специализированных интерфейсов лучше, чем один универсальный.
- **Dependency Inversion Principle** — Модули должны зависеть от абстракций, а не от деталей.

Шаг 07:

Миссия 7

 Задача: сопоставьте примеры нарушений с нарушаемыми принципами.

Подсказка: Внимательно смотрите, какой принцип ломается в ситуации.

Ответ:

- **Один класс и логика работы с БД, и валидация данных** — Single Responsibility Principle
- **Изменение старого класса для новых функций** — Open/Closed Principle
- **Подкласс выбрасывает исключение там, где родитель работал корректно** — Liskov Substitution Principle
- **Огромный интерфейс с ненужными методами** — Interface Segregation Principle
- **Модуль напрямую зависит от конкретного драйвера базы данных** — Dependency Inversion Principle

Шаг 08:

Миссия 8

 Сопоставьте принцип с типичной пользой для проекта.

Подсказка: Думайте о преимуществах для поддержки и расширяемости.

Ответ:

- **Single Responsibility Principle** — Код легче тестировать и поддерживать.
- **Open/Closed Principle** — Новые функции добавляются без риска поломки старых.
- **Liskov Substitution Principle** — Упрощается использование полиморфизма.
- **Interface Segregation Principle** — Классы не реализуют лишних методов.

- **Dependency Inversion Principle** — Можно легко подменять реализации через абстракции.

🔧 Шаг 09:

Миссия 9

🧑💻 Сопоставьте разработчика с принципом, которому он должен следовать в ситуации.

Подсказка: Каждый герой сталкивается с классической архитектурной проблемой.

Ответ:

- **Алексей вынужден переписывать старый код для новой фичи** — Open/Closed Principle
- **Мария жалуется на класс, который делает всё сразу** — Single Responsibility Principle
- **Игорь видит, что подкласс нарушает работу родителя** — Liskov Substitution Principle
- **Дина хочет разделить слишком большой интерфейс** — Interface Segregation Principle
- **Павел заменяет конкретный драйвер БД на абстракцию** — Dependency Inversion Principle

🔧 Шаг 10:

Миссия 10

📖 Сопоставьте принцип SOLID с его кратким девизом.

Подсказка: Вспомните ключевые фразы из литературы о SOLID.

Ответ:

- **Single Responsibility Principle** — Одна причина для изменения.
- **Open/Closed Principle** — Расширяй, но не изменяй.
- **Liskov Substitution Principle** — Замена без последствий.
- **Interface Segregation Principle** — Много интерфейсов лучше одного.
- **Dependency Inversion Principle** — Абстракции важнее деталей.

🔧 Шаг 11:

Миссия 11

⚙️ Сопоставьте инструмент тестирования с принципом, который он чаще всего проверяет.

Подсказка: Вспомните, какой тест чаще всего указывает на архитектурную проблему.

Ответ:

- **Юнит-тест для проверки одной логики** — Single Responsibility Principle
- **Модульный тест на расширяемость** — Open/Closed Principle
- **Тест с подменой родительского класса** — Liskov Substitution Principle
- **Контрактные тесты на интерфейсы** — Interface Segregation Principle
- **Интеграционные тесты с абстракциями** — Dependency Inversion Principle

🔧 Шаг 12:

Миссия 12

🏗️ Расположите шаги внедрения SRP в правильном порядке.

Подсказка: Сначала ищем проблемы, затем разделяем и тестируем.

Ответ:

- Найти класс с несколькими обязанностями
- Разделить обязанности на отдельные классы

- Обновить зависимости на новые классы
- Проверить работоспособность через тесты

🔗 Шаг 13:

Миссия 13

🔗 Упорядочьте шаги применения принципа OCP.

Подсказка: Главное — не менять старое, а добавлять новое.

Ответ:

- Найти место, где нужно новое поведение
- Создать абстракцию или интерфейс
- Реализовать новую функциональность в отдельном классе
- Подключить новый класс через абстракцию

🔗 Шаг 14:

Миссия 14

🔗 Расположите шаги проверки LSP при проектировании.

Подсказка: Задача — убедиться, что подкласс можно безопасно подставить.

Ответ:

- Определить поведение родительского класса
- Проверить совместимость поведения у подкласса
- Убедиться, что исключения не ломают логику
- Протестировать замену подкласса в реальном коде

🔗 Шаг 15:

Миссия 15

🔗 Упорядочьте действия для применения ISP.

Подсказка: Начните с анализа интерфейсов.

Ответ:

- Найти перегруженный интерфейс
- Разделить его на специализированные интерфейсы
- Назначить реализации для каждого интерфейса
- Проверить код на использование только нужных методов

🔗 Шаг 16:

Миссия 16

🔗 Упорядочьте шаги применения DIP.

Подсказка: Важный момент — заменить детали на абстракции.

Ответ:

- Выявить прямую зависимость от деталей
- Создать абстракцию (интерфейс)
- Переписать код для зависимости от абстракции
- Подключить конкретные реализации через абстракцию

Урок 2 - DRY и KISS Мудрость краткости

Шаг 01:

Миссия 1

Иван заметил, что в проекте постоянно повторяются одни и те же блоки кода. Что стоит сделать в первую очередь?

Подсказка: Дублирование кода усложняет поддержку и исправление ошибок.

Ответ:

- **Вынести повторяющийся код в отдельную функцию**
- Оставить код как есть
- Добавить комментарии к каждому повтору
- Создать длинный условный блок с повторяющимся кодом

Шаг 02:

Миссия 2

Мария замечает, что методы класса слишком длинные и сложные. Что лучше всего использовать для упрощения?

Подсказка: KISS помогает сделать код проще и понятнее.

Ответ:

- **Разделить методы на несколько логически связанных функций**
- Добавить больше условных операторов
- Игнорировать сложность и продолжать кодить
- Удалить комментарии для экономии места

Шаг 03:

Миссия 3

Дмитрий хочет улучшить производительность проекта. Он видит повторяющиеся вычисления внутри цикла. Что правильнее всего сделать?

Подсказка: Избегайте повторных вычислений, если результат не меняется.

Ответ:

- **Вынести вычисление за пределы цикла**
- Увеличить количество итераций цикла
- Добавить больше условий внутри цикла
- Игнорировать проблему, это мелочь

Шаг 04:

Миссия 4

Сопоставьте термины с их значением в контексте PHP и ООП.

Подсказка: Подумайте о том, зачем нужен рефакторинг и упрощение кода.

Ответ:

- **DRY** — Принцип, запрещающий повторение одного и того же кода
- **KISS** — Принцип, призывающий писать код проще
- **Рефакторинг** — Изменение структуры кода без изменения поведения
- **Юнит-тесты** — Автоматические проверки правильности работы функций

🔮 Шаг 05:

Миссия 5

Сопоставьте проблему с её возможным следствием в проекте.

Подсказка: Думайте о том, как ошибки или дублирование влияют на проект.

Ответ:

- **Дублирование кода** — Трудности в поддержке и исправлении ошибок
- **Сложные методы** — Увеличение времени на чтение и понимание
- **Плохие имена переменных** — Непонимание логики другими разработчиками
- **Отсутствие тестов** — Высокий риск появления багов

🔮 Шаг 06:

Миссия 6

Сопоставьте подходы к коду с их преимуществами.

Подсказка: Помните, что удобочитаемость и поддержка важнее краткости любой ценой.

Ответ:

- **Вынос функций** — Снижение дублирования кода
- **Использование интерфейсов** — Гибкость и расширяемость
- **Комментарии** — Помощь в понимании кода другими
- **Тесты** — Повышение надежности и уменьшение багов

🔮 Шаг 07:

Миссия 7

Сопоставьте типы дублирования кода с их примером.

Подсказка: Дублирование бывает не только в коде, но и в логике или данных.

Ответ:

- **Копипаста** — Одинаковые строки кода в нескольких местах
- **Логическое дублирование** — Повторение алгоритма с разными данными
- **Дублирование данных** — Одинаковые константы или настройки в разных местах
- **Дублирование тестов** — Схожие тесты для одного функционала

🔮 Шаг 08:

Миссия 8

Сопоставьте улучшение кода с ожидаемым эффектом.

Подсказка: Подумайте о том, как каждая оптимизация влияет на проект.

Ответ:

- **Оптимизация цикла** — Сокращение времени выполнения
- **Вынос повторяющегося кода** — Снижение ошибок и дублирования
- **Читаемые имена функций** — Упрощение поддержки
- **Комментарии и документация** — Помощь новым разработчикам

🔮 Шаг 09:

Миссия 9

Расположите этапы рефакторинга метода в правильном порядке:

1. Разделение метода на логические части
2. Тестирование существующей функциональности
3. Замена дублирующегося кода вызовами новых функций
4. Проверка, что код работает корректно после изменений

Подсказка: Сначала анализируем, потом изменяем, и в конце проверяем результат.

Ответ:

- Тестирование существующей функциональности
- Разделение метода на логические части
- Замена дублирующегося кода вызовами новых функций
- Проверка, что код работает корректно после изменений

Шаг 10:

Миссия 10

Расположите действия для оптимизации производительности PHP-кода:

1. Профилирование кода
2. Выявление узких мест
3. Оптимизация алгоритмов и циклов
4. Тестирование после изменений

Подсказка: Сначала измеряем, потом анализируем, затем улучшаем и проверяем.

Ответ:

- Профилирование кода
- Выявление узких мест
- Оптимизация алгоритмов и циклов
- Тестирование после изменений

Шаг 11:

Миссия 11

Упорядочите действия по внедрению KISS и DRY в проект:

1. Идентификация повторов
2. Создание вспомогательных функций
3. Разделение сложных функций на более простые
4. Проверка корректности работы после изменений

Подсказка: Сперва ищем проблему, потом исправляем и проверяем результат.

Ответ:

- Идентификация повторов
- Создание вспомогательных функций
- Разделение сложных функций на более простые
- Проверка корректности работы после изменений

Шаг 12:

Миссия 12

Расположите действия по упрощению метода с множеством условий:

1. Выделение повторяющейся логики в отдельные функции
2. Замена длинных условных операторов на полиморфизм
3. Тестирование работы метода после изменений
4. Рефакторинг оставшихся частей для читаемости

Подсказка: Разделяем, заменяем, тестируем и улучшаем читаемость.

Ответ:

- Выделение повторяющейся логики в отдельные функции
- Замена длинных условных операторов на полиморфизм
- Тестирование работы метода после изменений
- Рефакторинг оставшихся частей для читаемости

Шаг 13:

Миссия 13

Упорядочите действия для безопасного удаления дублирующегося кода:

1. Создание резервной копии и тестов
2. Выделение общего кода в функцию
3. Замена повторов на вызов новой функции
4. Проверка корректности работы программы

Подсказка: Безопасность прежде всего: сначала тесты, потом изменения, потом проверка.

Ответ:

- Создание резервной копии и тестов
- Выделение общего кода в функцию
- Замена повторов на вызов новой функции
- Проверка корректности работы программы

Шаг 14:

Миссия 14

Расположите действия для улучшения производительности и читаемости проекта:

1. Проверка повторяющихся запросов к базе
2. Оптимизация SQL-запросов и кеширование
3. Выделение повторяющихся частей в функции
4. Проверка работы и профилирование после изменений

Подсказка: Сначала анализируем, потом оптимизируем, затем проверяем.

Ответ:

- Проверка повторяющихся запросов к базе
- Оптимизация SQL-запросов и кеширование
- Выделение повторяющихся частей в функции
- Проверка работы и профилирование после изменений

Шаг 15:

Миссия 15

Расположите шаги по внедрению единообразного стиля кода:

1. Обсуждение и выбор стандарта кодирования
2. Автоматическая проверка кода линтером
3. Рефакторинг существующего кода под стандарт
4. Проверка соблюдения стандарта при новых изменениях

Подсказка: Сначала выбираем, потом проверяем, затем исправляем и поддерживаем.

Ответ:

- Обсуждение и выбор стандарта кодирования
- Автоматическая проверка кода линтером
- Рефакторинг существующего кода под стандарт
- Проверка соблюдения стандарта при новых изменениях

Урок 3 - Защита классов Укращение внешних сил

Шаг 01:

Миссия 1

Вы пишете библиотеку для работы с платежами. Как лучше защитить внутренние методы класса от прямого вызова извне?

Подсказка: Подумайте, какие модификаторы доступа ограничивают использование методов извне, но позволяют использовать их внутри класса.

Ответ:

- **Сделать методы `private` или `protected`**
- Сделать методы `public`
- Удалить методы
- Сделать методы `static`

Шаг 02:

Миссия 2

Вы создаете класс конфигурации приложения. Как сделать так, чтобы экземпляр класса нельзя было создать больше одного раза?

Подсказка: Подумайте о паттернах проектирования, которые ограничивают количество экземпляров класса.

Ответ:

- **Использовать паттерн `Singleton`**
- Создавать каждый раз новый объект
- Сделать конструктор `public`
- Наследовать класс от другого

Шаг 03:

Миссия 3

В проекте есть класс с чувствительными данными. Что поможет защитить их от изменения извне?

Подсказка: Рассмотрите подходы, где внутренние свойства недоступны напрямую.

Ответ:

- **Сделать свойства `private` и добавить методы доступа (геттеры)**

- Сделать свойства `public`
- Удалить свойства
- Сделать методы `final`

🔮 Шаг 04:

Миссия 4

📖 Задача: Сопоставьте модификатор доступа с его поведением.

Формулировка: Сопоставьте модификатор с тем, как он ограничивает доступ.

Понятия:

Ответ:

- `public`
- `protected`
- `private`

Определения:

1. Доступен отовсюду, внутри и вне класса.
2. Доступен только внутри класса и его наследников.
3. Доступен только внутри класса.

🔮 Подсказка:

Помните, кто может обращаться к методам и свойствам класса — извне, наследникам или только самому классу.

- **`public`** — Доступен отовсюду, внутри и вне класса.
- **`protected`** — Доступен только внутри класса и его наследников.
- **`private`** — Доступен только внутри класса.

🔮 Шаг 05:

Миссия 5

📖 Задача: Сопоставьте ключевое слово с его действием.

Формулировка: Сопоставьте ключевое слово PHP с его назначением.

Понятия:

Ответ:

- `final class`
- `abstract class`
- `static method`

Определения:

1. Класс, который нельзя наследовать.
2. Метод, который можно вызвать без создания объекта.
3. Класс, который нельзя создать напрямую, только наследовать.

🔮 Подсказка:

Обратите внимание на ограничения: что нельзя изменить или что можно использовать напрямую.

- **`final class`** — Класс, который нельзя наследовать.
- **`abstract class`** — Класс, который нельзя создать напрямую, только наследовать.
- **`static method`** — Метод, который можно вызвать без создания объекта.

🔗 Шаг 06:

Миссия 6

📖 Задача: Сопоставьте понятие инкапсуляции с его эффектом.

Формулировка: Сопоставьте принцип инкапсуляции с тем, как он влияет на код.

Понятия:

Ответ:

- Соккрытие данных
- Контроль доступа
- Изоляция изменений

Определения:

1. Внутренние детали класса скрыты от внешнего кода.
2. Можно управлять, кто может изменять или читать данные.
3. Изменения внутри класса минимально влияют на остальной код.

🔗 Подсказка:

Подумайте, какие преимущества дает ограничение прямого доступа к свойствам и методам.

- **Соккрытие данных** — Внутренние детали класса скрыты от внешнего кода.
- **Контроль доступа** — Можно управлять, кто может изменять или читать данные.
- **Изоляция изменений** — Изменения внутри класса минимально влияют на остальной код.

🔗 Шаг 07:

Миссия 7

📖 Задача: Сопоставьте способ защиты методов с его результатом.

Формулировка: Что даст объявление метода как `private`, `protected` или `final`?

Понятия:

Ответ:

- `private method`
- `protected method`
- `final method`

Определения:

1. Можно использовать только внутри класса.
2. Можно использовать внутри класса и наследников.
3. Нельзя переопределить в дочернем классе.

🔗 Подсказка:

Подумайте, как модификаторы влияют на доступ и возможность переопределения методов.

- **`private method`** — Можно использовать только внутри класса.
- **`protected method`** — Можно использовать внутри класса и наследников.
- **`final method`** — Нельзя переопределить в дочернем классе.

🔗 Шаг 08:

Миссия 8

📖 Задача: Сопоставьте способ защиты свойств с его последствием.

Формулировка: Что даст объявление свойства как public, protected или private?

Понятия:

Ответ:

- public property
- protected property
- private property

Определения:

1. Можно изменять отовсюду.
2. Можно изменять только внутри класса и наследников.
3. Можно изменять только внутри класса.

🔮 **Подсказка:**

Сравните свойства с методами и их модификаторы доступа.

- **public property** — Можно изменять отовсюду.
- **protected property** — Можно изменять только внутри класса и наследников.
- **private property** — Можно изменять только внутри класса.

🔮 **Шаг 09:**

Миссия 9

Расположите шаги создания защищенного класса в правильном порядке:

1. Определение чувствительных данных и методов
2. Выбор модификаторов доступа для методов и свойств
3. Создание методов доступа (геттеры/сеттеры) при необходимости
4. Проверка работы класса и тестирование безопасности

🔮 **Подсказка:**

Сначала определяем, что защищать, потом настраиваем доступ, добавляем безопасный доступ и в конце тестируем.

Ответ:

- Определение чувствительных данных и методов
- Выбор модификаторов доступа для методов и свойств
- Создание методов доступа (геттеры/сеттеры) при необходимости
- Проверка работы класса и тестирование безопасности

🔮 **Шаг 10:**

Миссия 10

Расположите этапы внедрения Singleton для конфигурационного класса:

1. Создание приватного конструктора
2. Создание статического метода для получения единственного экземпляра
3. Хранение экземпляра в приватном статическом свойстве
4. Использование метода для доступа к экземпляру

🔮 **Подсказка:**

Сначала запрещаем создание новых экземпляров, потом организуем хранение и доступ к единственному объекту.

Ответ:

- Создание приватного конструктора
- Создание статического метода для получения единственного экземпляра
- Хранение экземпляра в приватном статическом свойстве
- Использование метода для доступа к экземпляру

 Шаг 11:

Миссия 11

Расположите шаги защиты методов от переопределения и неправильного использования:

1. Определение методов, которые нельзя изменять
2. Объявление их как final
3. Тестирование невозможности переопределения
4. Документирование правил использования

 Подсказка:

Сначала выбираем методы, потом запрещаем изменения, затем проверяем и объясняем другим разработчикам.

Ответ:

- Определение методов, которые нельзя изменять
- Объявление их как final
- Тестирование невозможности переопределения
- Документирование правил использования

 Шаг 12:

Миссия 12

Расположите шаги реализации безопасного доступа к приватным свойствам:

1. Определение приватного свойства
2. Создание метода-геттера для чтения значения
3. Создание метода-сеттера для изменения значения с проверкой
4. Тестирование методов доступа

 Подсказка:

Сначала скрываем данные, потом создаем безопасный доступ, и только в конце проверяем работу.

Ответ:

- Определение приватного свойства
- Создание метода-геттера для чтения значения
- Создание метода-сеттера для изменения значения с проверкой
- Тестирование методов доступа

 Шаг 13:

Миссия 13

Расположите шаги реализации безопасного наследования класса с приватными и protected методами:

1. Создание базового класса с приватными и protected методами
2. Создание дочернего класса, использующего protected методы базового класса

3. Проверка, что приватные методы базового класса недоступны в дочернем
4. Документирование ограничений для наследников

 Подсказка:

Сначала определяем базу, потом наследуем, проверяем ограничения и объясняем другим разработчикам.

Ответ:

- Создание базового класса с приватными и protected методами
- Создание дочернего класса, использующего protected методы базового класса
- Проверка, что приватные методы базового класса недоступны в дочернем
- Документирование ограничений для наследников

 Шаг 14:

Миссия 14

Расположите шаги создания безопасного API-класса:

1. Определение всех публичных методов и их задач
2. Скрытие внутренних вспомогательных методов как private
3. Создание проверок и валидации внутри методов
4. Тестирование взаимодействия с внешним кодом

 Подсказка:

Сначала определяем интерфейс, затем скрываем детали и проверяем безопасность работы.

Ответ:

- Определение всех публичных методов и их задач
- Скрытие внутренних вспомогательных методов как private
- Создание проверок и валидации внутри методов
- Тестирование взаимодействия с внешним кодом

 Шаг 15:

Миссия 15

Расположите шаги по обеспечению неизменности объекта после его создания:

1. Определение свойств как private
2. Удаление или запрет всех сеттеров
3. Инициализация всех значений через конструктор
4. Проверка, что объект не изменяется после создания

 Подсказка:

Сначала скрываем данные, потом запрещаем изменения, затем инициализируем и тестируем объект.

Ответ:

- Определение свойств как private
- Удаление или запрет всех сеттеров
- Инициализация всех значений через конструктор
- Проверка, что объект не изменяется после создания

 Шаг 16:

Миссия 16

Расположите этапы защиты конфиденциальных данных в классе:

1. Сделать свойства `private`
2. Добавить методы-геттеры с проверкой прав доступа
3. Добавить методы-сеттеры с валидацией
4. Тестировать работу и безопасность всех методов

🔗 Подсказка:

Сначала скрываем данные, затем создаем безопасный доступ и проверяем корректность работы.

Ответ:

- Сделать свойства `private`
- Добавить методы-геттеры с проверкой прав доступа
- Добавить методы-сеттеры с валидацией
- Тестировать работу и безопасность всех методов

📖 Урок 4 - Оптимизация ООП Быстрее, проще, сильнее

🔗 Шаг 01:

Миссия 1

Вы создаёте класс для хранения конфиденциальной информации пользователя. Как лучше защитить данные от прямого доступа извне?

Подсказка: Подумайте, какие модификаторы доступа ограничивают видимость свойств класса.

Ответ:

- **Сделать свойства `private` и создать методы-геттеры**
- Сделать свойства `public`
- Удалить свойства
- Сделать методы `static`

🔗 Шаг 02:

Миссия 2

В библиотеке есть класс конфигурации приложения. Как сделать так, чтобы можно было создать только один экземпляр класса?

Подсказка: Существуют паттерны проектирования, ограничивающие количество экземпляров класса.

Ответ:

- **Использовать паттерн `Singleton`**
- Создавать новый объект при каждом вызове
- Сделать конструктор `public`
- Наследовать класс от другого

🔗 Шаг 03:

Миссия 3

Вы разрабатываете класс с методами, которые не должны изменяться в наследниках. Что нужно сделать?

Подсказка: Подумайте, как запретить переопределение методов.

Ответ:

- **Объявить методы как final**
- Сделать методы private
- Сделать методы public
- Удалить методы

Шаг 04:

Миссия 4

 Сопоставьте модификатор доступа с его поведением:

Ответ:

- public
 - protected
 - private
1. Доступен везде, внутри и вне класса
 2. Доступен только в классе и наследниках
 3. Доступен только в классе

Подсказка: Кто может обращаться к методам и свойствам класса?

- **public** — Доступен везде, внутри и вне класса
- **protected** — Доступен только в классе и наследниках
- **private** — Доступен только в классе

Шаг 05:

Миссия 5

 Сопоставьте ключевое слово с его действием:

Ответ:

- final class
 - abstract class
 - static method
1. Класс, который нельзя наследовать
 2. Класс, который нельзя создать напрямую
 3. Метод, который можно вызвать без создания объекта

Подсказка: Обратите внимание на ограничения создания и использования.

- **final class** — Класс, который нельзя наследовать
- **abstract class** — Класс, который нельзя создать напрямую
- **static method** — Метод, который можно вызвать без создания объекта

Шаг 06:

Миссия 6

 Сопоставьте принцип инкапсуляции с его эффектом:

Ответ:

- Скрытие данных
- Контроль доступа

- Изоляция изменений

1. Внутренние детали скрыты от внешнего кода
2. Можно управлять, кто может изменять данные
3. Изменения внутри класса минимально влияют на остальной код

Подсказка: Какие преимущества даёт ограничение прямого доступа к свойствам и методам?

- **Соккрытие данных** — Внутренние детали скрыты от внешнего кода
- **Контроль доступа** — Можно управлять, кто может изменять данные
- **Изоляция изменений** — Изменения внутри класса минимально влияют на остальной код

Шаг 07:

Миссия 7

 Сопоставьте способ защиты методов с его результатом:

Ответ:

- private method
 - protected method
 - final method
1. Можно использовать только внутри класса
 2. Можно использовать внутри класса и наследников
 3. Нельзя переопределить в дочернем классе

Подсказка: Как модификаторы влияют на доступ и переопределение методов?

- **private method** — Можно использовать только внутри класса
- **protected method** — Можно использовать внутри класса и наследников
- **final method** — Нельзя переопределить в дочернем классе

Шаг 08:

Миссия 8

 Сопоставьте свойства с результатом использования модификатора:

Ответ:

- public property
 - protected property
 - private property
1. Можно изменять отовсюду
 2. Можно изменять только внутри класса и наследников
 3. Можно изменять только внутри класса

Подсказка: Сравните свойства с методами и их модификаторы доступа.

- **public property** — Можно изменять отовсюду
- **protected property** — Можно изменять только внутри класса и наследников
- **private property** — Можно изменять только внутри класса

Шаг 09:

Миссия 9

Расположите шаги создания защищённого класса в правильном порядке:

1. Определение чувствительных данных и методов
2. Выбор модификаторов доступа для методов и свойств
3. Создание методов доступа (геттеры/сеттеры) при необходимости
4. Проверка работы класса и тестирование безопасности

Подсказка: Сначала определяем, что защищать, потом настраиваем доступ, добавляем безопасный доступ и проверяем.

Ответ:

- Определение чувствительных данных и методов
- Выбор модификаторов доступа для методов и свойств
- Создание методов доступа (геттеры/сеттеры) при необходимости
- Проверка работы класса и тестирование безопасности

Шаг 10:

Миссия 10

Расположите этапы внедрения Singleton:

1. Создание приватного конструктора
2. Создание статического метода для получения экземпляра
3. Хранение экземпляра в приватном статическом свойстве
4. Использование метода для доступа к экземпляру

Подсказка: Сначала запрещаем создание новых объектов, потом организуем доступ к единственному экземпляру.

Ответ:

- Создание приватного конструктора
- Создание статического метода для получения экземпляра
- Хранение экземпляра в приватном статическом свойстве
- Использование метода для доступа к экземпляру

Шаг 11:

Миссия 11

Расположите шаги защиты конфиденциальных данных в классе:

1. Сделать свойства private
2. Создать методы-геттеры с проверкой прав доступа
3. Создать методы-сеттеры с валидацией
4. Тестировать работу и безопасность методов

Подсказка: Сначала скрываем данные, потом создаём безопасный доступ, затем проверяем корректность работы.

Ответ:

- Сделать свойства private
- Создать методы-геттеры с проверкой прав доступа
- Создать методы-сеттеры с валидацией
- Тестировать работу и безопасность методов

Шаг 12:

Миссия 12

Расположите этапы защиты методов от переопределения:

1. Выбор методов, которые не должны изменяться
2. Объявление этих методов как final
3. Документирование ограничений для наследников
4. Проверка работы методов в наследниках

Подсказка: Сначала выбираем, что защищать, потом запрещаем переопределение, документируем и проверяем.

Ответ:

- Выбор методов, которые не должны изменяться
- Объявление этих методов как final
- Документирование ограничений для наследников
- Проверка работы методов в наследниках

Шаг 13:

Миссия 13

Расположите шаги организации защищённого наследования:

1. Создание базового класса с приватными и protected методами
2. Создание дочернего класса, использующего protected методы базового класса
3. Проверка, что приватные методы недоступны в наследниках
4. Документирование правил использования методов

Подсказка: Сначала определяем базу, затем наследуем и проверяем ограничения.

Ответ:

- Создание базового класса с приватными и protected методами
- Создание дочернего класса, использующего protected методы базового класса
- Проверка, что приватные методы недоступны в наследниках
- Документирование правил использования методов

Шаг 14:

Миссия 14

Расположите шаги обеспечения неизменности объекта после его создания:

1. Определение свойств как private
2. Удаление или запрет всех сеттеров
3. Инициализация значений через конструктор
4. Проверка, что объект не изменяется после создания

Подсказка: Сначала скрываем данные, затем запрещаем изменение, инициализируем и проверяем объект.

Ответ:

- Определение свойств как private
- Удаление или запрет всех сеттеров
- Инициализация значений через конструктор
- Проверка, что объект не изменяется после создания

Шаг 15:

Миссия 15

Расположите шаги защиты API-класса:

1. Определение публичных методов и их задач
2. Скрытие внутренних методов как `private`
3. Добавление проверок и валидации внутри методов
4. Тестирование взаимодействия с внешним кодом

Подсказка: Сначала определяем интерфейс, затем скрываем детали и проверяем безопасность работы.

Ответ:

- Определение публичных методов и их задач
- Скрытие внутренних методов как `private`
- Добавление проверок и валидации внутри методов
- Тестирование взаимодействия с внешним кодом

Шаг 16:

Миссия 16

Расположите шаги безопасного наследования с контролем доступа:

1. Создание базового класса с методами разного уровня доступа
2. Создание дочернего класса
3. Использование `protected` методов базового класса
4. Проверка, что `private` методы недоступны

Подсказка: Сначала создаём базу, потом наследуем и проверяем ограничения доступа.

Ответ:

- Создание базового класса с методами разного уровня доступа
- Создание дочернего класса
- Использование `protected` методов базового класса
- Проверка, что `private` методы недоступны

Шаг 17:

Миссия 17

Расположите этапы создания полностью защищённого класса:

1. Сделать свойства `private`
2. Сделать методы с контролем доступа
3. Документировать правила использования
4. Протестировать работу и безопасность всех методов

Подсказка: Сначала скрываем данные, потом защищаем методы, документируем и проверяем класс.

Ответ:

- Сделать свойства `private`
- Сделать методы с контролем доступа
- Документировать правила использования
- Протестировать работу и безопасность всех методов

Урок 5 - Код-ревью Испытание соратников

Шаг 01:

Миссия 1

Что является главным при проведении код-ревью?

Подсказка: Обратите внимание на то, что помогает улучшить код, а не на стиль общения.

Ответ:

- **Выявление ошибок и улучшение читаемости кода**
- Личная критика автора кода
- Проверка орфографии в комментариях
- Изменение бизнес-логики без согласования

Шаг 02:

Миссия 2

Какая практика делает код-ревью эффективным?

Подсказка: Подумайте, как быстро и понятно передать замечания коллегам.

Ответ:

- **Комментирование конкретных мест кода с предложениями улучшений**
- Оставлять общие комментарии без конкретики
- Использовать длинные отчёты о стиле кода
- Игнорировать тесты и документацию

Шаг 03:

Миссия 3

Что важно учитывать при принятии замечаний в код-ревью?

Подсказка: Важно воспринимать критику конструктивно.

Ответ:

- **Фокусироваться на улучшении кода, а не на личности**
- Игнорировать рекомендации коллег
- Менять код без обсуждения
- Спорить с коллегой о стиле кода без причины

Шаг 04:

Миссия 4

 Сопоставьте термин код-ревью с его описанием.

Ответ:

- Peer Review
 - Pull Request
 - Inline Comment
 - Approval
1. Процесс, когда коллеги проверяют ваш код
 2. Форма запроса на внесение изменений в общий репозиторий
 3. Комментарии, вставленные прямо в строки кода
 4. Подтверждение, что изменения готовы к слиянию

Подсказка: Подумайте, какой процесс связан с проверкой кода, а что с документацией и разрешением изменений.

- **Peer Review** — Процесс, когда коллеги проверяют ваш код
- **Pull Request** — Форма запроса на внесение изменений в общий репозиторий
- **Inline Comment** — Комментарии, вставленные прямо в строки кода
- **Approval** — Подтверждение, что изменения готовы к слиянию

Шаг 05:

Миссия 5

 Сопоставьте тип замечания с его примером.

Ответ:

- Bug
 - Code Style
 - Optimization
 - Security
1. Исправление ошибки, из-за которой падает программа
 2. Приведение форматирования и имен переменных к стандарту
 3. Улучшение производительности медленной функции
 4. Исправление уязвимости SQL-инъекции

Подсказка: Подумайте, какие замечания направлены на функциональность, а какие на качество кода.

- **Bug** — Исправление ошибки, из-за которой падает программа
- **Code Style** — Приведение форматирования и имен переменных к стандарту
- **Optimization** — Улучшение производительности медленной функции
- **Security** — Исправление уязвимости SQL-инъекции

Шаг 06:

Миссия 6

 Сопоставьте роль в код-ревью с её описанием.

Ответ:

- Author
 - Reviewer
 - Observer
 - Maintainer
1. Человек, написавший изменения в коде
 2. Человек, который проверяет код и даёт комментарии
 3. Сотрудник, который следит за процессом, но не участвует активно
 4. Человек, ответственный за слияние изменений в основной репозиторий

Подсказка: Подумайте, кто создаёт код, кто проверяет и кто управляет репозиторием.

- **Author** — Человек, написавший изменения в коде
- **Reviewer** — Человек, который проверяет код и даёт комментарии
- **Observer** — Сотрудник, который следит за процессом, но не участвует активно
- **Maintainer** — Человек, ответственный за слияние изменений в основной репозиторий

Шаг 07:

Миссия 7

 Сопоставьте результат код-ревью с правильным описанием.

Ответ:

- Merge
 - Request Changes
 - Comment Only
 - Rejected
1. Изменения были объединены в основной репозиторий
 2. Требуется доработка перед слиянием
 3. Комментарии добавлены без блокировки слияния
 4. Изменения не приняты и отклонены

Подсказка: Подумайте, какие статусы означают успех, доработку или отказ.

- **Merge** — Изменения были объединены в основной репозиторий
- **Request Changes** — Требуется доработка перед слиянием
- **Comment Only** — Комментарии добавлены без блокировки слияния
- **Rejected** — Изменения не приняты и отклонены

Шаг 08:

Миссия 8

 Сопоставьте этап код-ревью с правильной последовательностью действий.

Ответ:

- Preparation
 - Review
 - Discussion
 - Integration
1. Автор готовит код и документацию для проверки
 2. Ревьюеры проверяют код и добавляют комментарии
 3. Обсуждаются замечания и согласуются изменения
 4. Изменения сливаются в основной репозиторий

Подсказка: Подумайте, какой этап происходит первым, а какой последним.

- **Preparation** — Автор готовит код и документацию для проверки
- **Review** — Ревьюеры проверяют код и добавляют комментарии
- **Discussion** — Обсуждаются замечания и согласуются изменения
- **Integration** — Изменения сливаются в основной репозиторий

Шаг 09:

Миссия 9

 Сопоставьте практику код-ревью с её преимуществом.

Ответ:

- Pair Programming
- Pre-Commit Review
- Post-Commit Review

- Automated Review Tools

1. Улучшение совместного понимания кода и знаний
2. Обнаружение ошибок до коммита
3. Обнаружение проблем после интеграции изменений
4. Быстрое выявление стандартных проблем и нарушений стиля

Подсказка: Подумайте, когда и как применяются разные методы проверки.

- **Pair Programming** — Улучшение совместного понимания кода и знаний
- **Pre-Commit Review** — Обнаружение ошибок до коммита
- **Post-Commit Review** — Обнаружение проблем после интеграции изменений
- **Automated Review Tools** — Быстрое выявление стандартных проблем и нарушений стиля

Шаг 10:

Миссия 10

Расположите шаги эффективного код-ревью:

1. Автор создаёт Pull Request
2. Ревьюеры проверяют код и добавляют комментарии
3. Обсуждаются замечания и вносятся исправления
4. Изменения сливаются в основной репозиторий

Подсказка: Сначала создаём запрос, затем проверяем, обсуждаем и интегрируем изменения.

Ответ:

- Автор создаёт Pull Request
- Ревьюеры проверяют код и добавляют комментарии
- Обсуждаются замечания и вносятся исправления
- Изменения сливаются в основной репозиторий

Шаг 11:

Миссия 11

Расположите этапы подготовки к код-ревью:

1. Проверка тестов и документации
2. Форматирование кода и соблюдение стиля
3. Создание Pull Request с описанием изменений
4. Отправка коллегам для проверки

Подсказка: Сначала проверяем свой код, потом оформляем и отправляем на проверку.

Ответ:

- Проверка тестов и документации
- Форматирование кода и соблюдение стиля
- Создание Pull Request с описанием изменений
- Отправка коллегам для проверки

Шаг 12:

Миссия 12

Расположите шаги обработки замечаний в код-ревью:

1. Чтение комментариев ревьюеров

2. Обсуждение спорных моментов
3. Внесение изменений в код
4. Отправка обновлённого Pull Request

Подсказка: Сначала читаем, потом обсуждаем, исправляем и отправляем снова.

Ответ:

- Чтение комментариев ревьюеров
- Обсуждение спорных моментов
- Внесение изменений в код
- Отправка обновлённого Pull Request

 Шаг 13:

Миссия 13

Расположите шаги внедрения стандартов код-ревью в команде:

1. Определение критериев качества кода
2. Документирование правил и процессов
3. Обучение команды и внедрение практик
4. Регулярная проверка соблюдения стандартов

Подсказка: Сначала устанавливаем критерии, затем учим и проверяем их соблюдение.

Ответ:

- Определение критериев качества кода
- Документирование правил и процессов
- Обучение команды и внедрение практик
- Регулярная проверка соблюдения стандартов

 Шаг 14:

Миссия 14

Расположите шаги внедрения стандартов код-ревью в команде:

1. Определение критериев качества кода
2. Документирование правил и процессов
3. Обучение команды и внедрение практик
4. Регулярная проверка соблюдения стандартов

Подсказка: Сначала устанавливаем критерии, затем обучаем команду, внедряем практики и проверяем соблюдение стандартов.

Ответ:

- Определение критериев качества кода
- Документирование правил и процессов
- Обучение команды и внедрение практик
- Регулярная проверка соблюдения стандартов

 Шаг 15:

Миссия 15

Расположите шаги оптимизации процесса код-ревью:

1. Автоматизация проверки стиля и тестов

2. Определение ответственных ревьюеров
3. Регулярные сессии обсуждения проблем
4. Сбор обратной связи и корректировка процесса

Подсказка: Подумайте, сначала автоматизируем и назначаем, затем обсуждаем и улучшаем процесс.

Ответ:

- Автоматизация проверки стиля и тестов
- Определение ответственных ревьюеров
- Регулярные сессии обсуждения проблем
- Сбор обратной связи и корректировка процесса

Шаг 16:

Миссия 16

Расположите шаги эффективной коммуникации при код-ревью:

1. Чтение и понимание комментариев коллег
2. Формулировка собственных вопросов и замечаний
3. Обсуждение спорных моментов конструктивно
4. Подтверждение согласованных изменений и действий

Подсказка: Сначала читаем, затем формулируем, обсуждаем и подтверждаем изменения.

Ответ:

- Чтение и понимание комментариев коллег
- Формулировка собственных вопросов и замечаний
- Обсуждение спорных моментов конструктивно
- Подтверждение согласованных изменений и действий

Модуль 6 - «Битва с Архитектурным Драконом» — Финальный проект

Урок 1 - Разработка ТЗ героя

Шаг 01:

Миссия 1

Вы проектируете ТЗ для игрового героя. Какой из пунктов важнее всего указать первым?

Подсказка: Начинайте с основ — того, что определяет суть героя.

Ответ:

- **Основные характеристики и способности героя**
- Цветовая схема интерфейса игры
- Список используемых шрифтов
- План маркетинговой кампании

Шаг 02:

Миссия 2

 Сопоставьте элементы ТЗ с их назначением.

Понятия:

Ответ:

- Сила героя
- Скорость передвижения
- Особые способности
- Слабости и уязвимости

Назначения:

1. Определяет эффективность в бою
2. Влияет на маневренность персонажа
3. Дает уникальные преимущества
4. Определяет уязвимые точки и риски

Подсказка: Подумайте, какие характеристики влияют на бой, движение, уникальность и слабости.

- **Сила героя** — Определяет эффективность в бою
- **Скорость передвижения** — Влияет на маневренность персонажа
- **Особые способности** — Дает уникальные преимущества
- **Слабости и уязвимости** — Определяет уязвимые точки и риски

 Шаг 03:

Миссия 3

Расположите этапы создания ТЗ героя в правильном порядке:

1. Сбор требований и пожеланий команды
2. Определение основных характеристик героя
3. Добавление особых способностей и слабостей
4. Финальная проверка и согласование ТЗ

Подсказка: Сначала собираем информацию, затем описываем базу, добавляем уникальные детали и проверяем финальный вариант.

Ответ:

- Сбор требований и пожеланий команды
- Определение основных характеристик героя
- Добавление особых способностей и слабостей
- Финальная проверка и согласование ТЗ

 Шаг 04:

Миссия 4

При проектировании героя вы решаете, какой тип атак выбрать. Что важнее учитывать первым?

Подсказка: Начните с того, что определяет стиль боя героя.

Ответ:

- **Способности и сильные стороны героя**
- Цвет оружия
- Вид гардероба героя
- Список локаций в игре

Шаг 05:

Миссия 5

 Сопоставьте элементы боевых характеристик героя с их эффектами.

Понятия:

Ответ:

- Сила атаки
- Защита
- Ловкость
- Энергия

Назначения:

1. Уменьшает получаемый урон
2. Определяет скорость и точность действий
3. Влияет на наносимый урон
4. Определяет количество действий за раунд

Подсказка: Подумайте, что влияет на удар, защиту, скорость и действия.

- **Сила атаки** — Влияет на наносимый урон
- **Защита** — Уменьшает получаемый урон
- **Ловкость** — Определяет скорость и точность действий
- **Энергия** — Определяет количество действий за раунд

Шаг 06:

Миссия 6

Вы определяете базовые параметры героя. Что следует за характеристиками и способностями?

Подсказка: Подумайте, что помогает сделать героя уникальным и интересным для игрока.

Ответ:

- **Слабости и особенности поведения героя**
- Список музыки в игре
- Эскиз интерфейса
- Цветовая палитра уровней

Шаг 07:

Миссия 7

 Сопоставьте элементы ТЗ героя с игровыми механиками.

Понятия:

Ответ:

- Атака ближнего боя
- Магические способности
- Исследование
- Социальные взаимодействия

Назначения:

1. Взаимодействие с NPC и квестами
2. Наносит урон врагам на близком расстоянии

3. Использование заклинаний и магических эффектов

4. Исследование локаций и поиск предметов

Подсказка: Подумайте, какая способность отвечает за бой, магию, исследование и взаимодействие.

- **Атака ближнего боя** — Наносит урон врагам на близком расстоянии
- **Магические способности** — Использование заклинаний и магических эффектов
- **Исследование** — Исследование локаций и поиск предметов
- **Социальные взаимодействия** — Взаимодействие с NPC и квестами

Шаг 08:

Миссия 8

Вы хотите, чтобы герой имел уникальную черту. Какую опцию выбрать?

Подсказка: Подумайте о том, что сделает героя запоминающимся в игре.

Ответ:

- **Особая способность, недоступная другим героям**
- Обычный набор умений
- Стандартные атрибуты для всех персонажей
- Просто декоративные элементы

Шаг 09:

Миссия 9

 Сопоставьте элементы ТЗ с типами тестирования персонажа.

Понятия:

Ответ:

- Баланс силы
- Скорость реагирования
- Использование особых способностей
- Проверка слабостей

Назначения:

1. Убедиться, что персонаж не слишком силен или слаб
2. Проверка скорости и точности действий
3. Тестирование уникальных возможностей героя
4. Проверка уязвимых мест и логических ошибок

Подсказка: Подумайте, какие тесты нужны для силы, скорости, уникальных способностей и слабостей.

- **Баланс силы** — Убедиться, что персонаж не слишком силен или слаб
- **Скорость реагирования** — Проверка скорости и точности действий
- **Использование особых способностей** — Тестирование уникальных возможностей героя
- **Проверка слабостей** — Проверка уязвимых мест и логических ошибок

Шаг 10:

Миссия 10

Расположите шаги финальной разработки ТЗ героя в правильном порядке:

1. Сбор обратной связи команды
2. Доработка характеристик и способностей
3. Определение уникальных черт и слабостей
4. Финальное согласование и утверждение ТЗ

Подсказка: Сначала собираем отзывы, затем корректируем характеристики, добавляем уникальные черты и в конце утверждаем документ.

Ответ:

- Сбор обратной связи команды
- Доработка характеристик и способностей
- Определение уникальных черт и слабостей
- Финальное согласование и утверждение ТЗ

Шаг 11:

Миссия 11

Вы выбираете основной стиль боя героя. Какой критерий важен первым?

Подсказка: Сначала подумайте о том, что определяет стиль игры за героя.

Ответ:

- **Соответствие способностей героя выбранному стилю боя**
- Декоративные элементы одежды
- Выбор музыки для битвы
- Цвета интерфейса

Шаг 12:

Миссия 12

Вы решаете, какую роль в команде будет играть герой. Что нужно учесть в первую очередь?

Подсказка: Сначала подумайте о том, какую функцию герой будет выполнять в бою и квестах.

Ответ:

- **Навыки и сильные стороны героя**
- Цвет брони
- Размер инвентаря
- Название локации

Шаг 13:

Миссия 13

 Сопоставьте элементы ТЗ героя с видами взаимодействия в игре.

Понятия:

Ответ:

- Боевые навыки
- Исследовательские способности
- Социальные умения
- Магические способности

Назначения:

1. Помогают находить предметы и скрытые локации
2. Участвуют в боях и наносят урон врагам
3. Взаимодействие с NPC и выполнение квестов
4. Использование заклинаний и специальных эффектов

Подсказка: Подумайте, какая способность отвечает за бой, исследование, магию и социальное взаимодействие.

- **Боевые навыки** — Участвуют в боях и наносят урон врагам
- **Исследовательские способности** — Помогают находить предметы и скрытые локации
- **Социальные умения** — Взаимодействие с NPC и выполнение квестов
- **Магические способности** — Использование заклинаний и специальных эффектов

Шаг 14:

Миссия 14

При выборе экипировки героя, что важно учитывать в первую очередь?

Подсказка: Сосредоточьтесь на том, что влияет на боевые и защитные характеристики.

Ответ:

- **Силу, защиту и бонусы экипировки**
- Декоративные элементы
- Цветовое оформление
- Название предмета

Шаг 15:

Миссия 15

Расположите этапы тестирования героя перед релизом ТЗ в правильном порядке:

1. Тестирование боевых навыков
2. Проверка магических способностей
3. Тестирование социальных взаимодействий и квестов
4. Проверка уникальных черт и слабостей

Подсказка: Сначала проверяем основные навыки, затем магию, социальные функции, и в конце — уникальные черты и слабости.

Ответ:

- Тестирование боевых навыков
- Проверка магических способностей
- Тестирование социальных взаимодействий и квестов
- Проверка уникальных черт и слабостей

Шаг 16:

Миссия 16

 Сопоставьте типы врагов с их особенностями:

Понятия:

Ответ:

- Огненный дракон
- Ледяной голем

- Темный маг
- Разбойник

Назначения:

1. Использует заклинания для атаки и защиты
2. Высокая физическая защита, замедляет героя
3. Атакует быстро и неожиданно, наносит критический урон
4. Наносит урон огнем, слаб к воде

Подсказка: Подумайте, какой враг использует магию, кто силен физически и кто действует быстро.

- **Огненный дракон** — Наносит урон огнем, слаб к воде
- **Ледяной голем** — Высокая физическая защита, замедляет героя
- **Темный маг** — Использует заклинания для атаки и защиты
- **Разбойник** — Атакует быстро и неожиданно, наносит критический урон

Шаг 17:

Миссия 17

Расположите шаги подготовки героя к финальной битве в правильном порядке:

1. Проверка и улучшение экипировки
2. Тренировка боевых навыков
3. Изучение карты локации и слабостей врага
4. Сбор команды и распределение ролей

Подсказка: Сначала укрепляем экипировку, затем навыки, потом планируем стратегию и формируем команду.

Ответ:

- Проверка и улучшение экипировки
- Тренировка боевых навыков
- Изучение карты локации и слабостей врага
- Сбор команды и распределение ролей

Урок 2 - Архитектура под ключ — выбор пути

Шаг 01:

Миссия 1

Какой принцип важно соблюдать при проектировании архитектуры классов?

Подсказка: Думайте о том, чтобы код был легко расширяемым и поддерживаемым.

Ответ:

- **Принцип единственной ответственности**
- Смешение логики и интерфейса
- Игнорирование тестирования
- Дублирование кода для удобства

Шаг 02:

Миссия 2

Что из перечисленного помогает защитить класс от внешнего вмешательства?

Подсказка: Подумайте о ключевых словах доступа в PHP.

Ответ:

- **Использование `private` и `protected` свойств и методов**
- Использование глобальных переменных
- Игнорирование инкапсуляции
- Повсеместное наследование без контроля

Шаг 03:

Миссия 3

Какая стратегия проектирования помогает облегчить замену компонентов в будущем?

Подсказка: Думайте о зависимостях и интерфейсах.

Ответ:

- **Программирование через интерфейсы, а не через конкретные классы**
- Жесткое связывание классов
- Использование глобальных функций вместо методов классов
- Дублирование кода вместо наследования

Шаг 04:

Миссия 4

 Сопоставьте архитектурные паттерны с их назначением:

Ответ:

- Singleton
 - Factory
 - Observer
 - Decorator
1. Обеспечивает наличие только одного экземпляра класса
 2. Создает объекты без указания конкретного класса
 3. Позволяет объектам уведомлять других об изменениях
 4. Динамически расширяет функциональность объектов

Подсказка: Вспомните, какой паттерн контролирует единственный объект, а какой добавляет новые возможности.

- **Singleton** — Обеспечивает наличие только одного экземпляра класса
- **Factory** — Создает объекты без указания конкретного класса
- **Observer** — Позволяет объектам уведомлять других об изменениях
- **Decorator** — Динамически расширяет функциональность объектов

Шаг 05:

Миссия 5

 Сопоставьте принципы SOLID с их описанием:

Ответ:

- SRP
- OCP
- LSP

- ISP

1. Класс должен иметь только одну причину для изменения
2. Классы должны быть открыты для расширения, но закрыты для модификации
3. Объекты подкласса должны заменять объекты суперкласса
4. Клиенты не должны зависеть от методов, которые они не используют

Подсказка: Подумайте о каждой букве SOLID как о короткой подсказке для принципа.

- **SRP** — Класс должен иметь только одну причину для изменения
- **OCP** — Классы должны быть открыты для расширения, но закрыты для модификации
- **LSP** — Объекты подкласса должны заменять объекты суперкласса
- **ISP** — Клиенты не должны зависеть от методов, которые они не используют

Шаг 06:

Миссия 6

Расположите этапы проектирования архитектуры в правильном порядке:

1. Сбор требований
2. Проектирование модулей и интерфейсов
3. Реализация кода
4. Тестирование и рефакторинг

Подсказка: Начните с изучения задач, потом проектируйте, затем пишите и проверяйте код.

Ответ:

- Сбор требований
- Проектирование модулей и интерфейсов
- Реализация кода
- Тестирование и рефакторинг

Шаг 07:

Миссия 7

Какой принцип важно соблюдать при организации зависимостей между классами?

Подсказка: Думайте о слабой связности и удобстве замены компонентов.

Ответ:

- **Инверсия зависимостей (Dependency Inversion Principle)**
- Жесткая зависимость между конкретными классами
- Использование глобальных переменных для связи классов
- Игнорирование интерфейсов

Шаг 08:

Миссия 8

Что является ключевым при разделении ответственности между модулями?

Подсказка: Модули должны быть максимально независимыми и понятными.

Ответ:

- **Каждый модуль выполняет одну задачу**
- Модули делают всё вместе
- Модули имеют дублирующую логику

- Модули зависят от глобальных переменных

Шаг 09:

Миссия 9

Какой подход помогает снизить количество ошибок при интеграции компонентов?

Подсказка: Думайте о тестировании каждого блока отдельно.

Ответ:

- **Модульное тестирование**
- Тестирование только на финальном этапе
- Игнорирование тестов для ускорения работы
- Ручная проверка всех функций одновременно

Шаг 10:

Миссия 10

 Сопоставьте типы зависимостей с их описанием:

Ответ:

- Сильная зависимость
 - Слабая зависимость
 - Внедрение через интерфейс
 - Глобальные зависимости
1. Модуль полностью зависит от конкретной реализации другого модуля
 2. Модуль зависит от абстракций, а не от конкретной реализации
 3. Зависимость передается через интерфейсы или конструктор
 4. Использование глобальных переменных или синглтонов для связи

Подсказка: Слабая зависимость облегчает замену компонентов.

- **Сильная зависимость** — Модуль полностью зависит от конкретной реализации другого модуля
- **Слабая зависимость** — Модуль зависит от абстракций, а не от конкретной реализации
- **Внедрение через интерфейс** — Зависимость передается через интерфейсы или конструктор
- **Глобальные зависимости** — Использование глобальных переменных или синглтонов для связи

Шаг 11:

Миссия 11

 Сопоставьте типы тестирования с их целью:

Ответ:

- Unit-тест
 - Integration-тест
 - Functional-тест
 - Acceptance-тест
1. Проверка отдельного метода или класса
 2. Проверка взаимодействия нескольких модулей
 3. Проверка функционала приложения с точки зрения пользователя

4. Проверка соответствия требований заказчика

Подсказка: Начинаем с маленьких частей, заканчиваем проверкой бизнес-требований.

- **Unit-тест** — Проверка отдельного метода или класса
- **Integration-тест** — Проверка взаимодействия нескольких модулей
- **Functional-тест** — Проверка функционала приложения с точки зрения пользователя
- **Acceptance-тест** — Проверка соответствия требований заказчика

Шаг 12:

Миссия 12

Расположите шаги внедрения новой архитектуры в правильном порядке:

1. Планирование изменений
2. Рефакторинг существующего кода
3. Тестирование новых модулей
4. Внедрение в продакшн

Подсказка: Сначала планируем, затем улучшаем код, проверяем тестами и только потом запускаем.

Ответ:

- Планирование изменений
- Рефакторинг существующего кода
- Тестирование новых модулей
- Внедрение в продакшн

Шаг 13:

Миссия 13

Расположите шаги code review для финального проекта:

1. Проверка безопасности кода
2. Проверка архитектуры и модульной структуры
3. Проверка стиля и читаемости
4. Обсуждение с командой и исправления

Подсказка: Начинаем с критичных моментов, потом структуру, стиль и финальная интеграция.

Ответ:

- Проверка безопасности кода
- Проверка архитектуры и модульной структуры
- Проверка стиля и читаемости
- Обсуждение с командой и исправления

Шаг 14:

Миссия 14

Какой принцип важен при расширении функционала существующих модулей?

Подсказка: Думайте о минимальном изменении существующего кода.

Ответ:

- **Следовать принципу OCP (Open/Closed Principle)**
- Вносить изменения в существующие классы без ограничений

- Удалять тесты для ускорения работы
- Игнорировать зависимые модули

Шаг 15:

Миссия 15

 Сопоставьте виды документации с их назначением:

Ответ:

- Техническая документация
 - Пользовательская документация
 - Архитектурная документация
 - Комментарии в коде
1. Описание функций, методов и классов для разработчиков
 2. Инструкции для конечного пользователя
 3. Описание структуры системы и модулей
 4. Объяснение конкретных блоков кода

Подсказка: Думайте, кто будет использовать каждый тип документации.

- **Техническая документация** — Описание функций, методов и классов для разработчиков
- **Пользовательская документация** — Инструкции для конечного пользователя
- **Архитектурная документация** — Описание структуры системы и модулей
- **Комментарии в коде** — Объяснение конкретных блоков кода

Шаг 16:

Миссия 16

Расположите этапы завершения финального проекта в правильном порядке:

1. Тестирование всех модулей
2. Code review и исправления
3. Документирование проекта
4. Релиз и внедрение в продакшн

Подсказка: Последовательность важна: сначала тесты, потом исправления, документация и финальный релиз.

Ответ:

- Тестирование всех модулей
- Code review и исправления
- Документирование проекта
- Релиз и внедрение в продакшн

Урок 3 - Код в бою реализация проекта

Шаг 01:

Миссия 1

Вы начинаете реализацию нового проекта, используя ООП. Какой принцип стоит применять, чтобы классы оставались гибкими?

Подсказка: Подумайте о том, чтобы изменения в одной части кода не ломали другие части.

Ответ:

- **Принцип единственной ответственности (SRP)**
- Хардкодить зависимости прямо в методах
- Создавать один глобальный класс для всего
- Игнорировать интерфейсы и абстракции

Шаг 02:

Миссия 2

При проектировании системы оплаты в интернет-магазине, как лучше управлять разными способами оплаты?

Подсказка: Подумайте о способах добавления новых вариантов без переписывания существующего кода.

Ответ:

- **Использовать паттерн Стратегия для разных методов оплаты**
- Создать один огромный switch для всех способов оплаты
- Хардкодить все способы оплаты в контроллере
- Игнорировать различия между способами оплаты

Шаг 03:

Миссия 3

В процессе разработки вы хотите защитить данные от многократной инициализации. Какой подход лучше использовать?

Подсказка: Это паттерн, обеспечивающий существование только одного экземпляра объекта.

Ответ:

- **Паттерн Синглтон**
- Фабричный метод
- Стратегия
- Декоратор

Шаг 04:

Миссия 4

 Сопоставьте паттерн проектирования с его назначением.

Подсказка: Подумайте, какой паттерн решает задачу создания объекта, а какой — организации поведения.

Ответ:

- **Фабричный метод** — Позволяет создавать объекты без указания точного класса
- **Синглтон** — Обеспечивает существование только одного экземпляра класса
- **Стратегия** — Позволяет выбирать алгоритм во время выполнения
- **Декоратор** — Динамически добавляет новые функции объекту

Шаг 05:

Миссия 5

 Сопоставьте принцип SOLID с его описанием.

Подсказка: Обратите внимание на слова «ответственность», «расширяемость», «замена».

Ответ:

- **SRP** — Класс должен иметь только одну причину для изменения
- **OCP** — Код должен быть открыт для расширения, но закрыт для изменения
- **LSP** — Объекты подкласса должны заменять объекты суперкласса без нарушения работы
- **ISP** — Клиенты не должны зависеть от интерфейсов, которые они не используют

Шаг 06:

Миссия 6

 Сопоставьте термин с примером из реальной разработки PHP.

Подсказка: Вспомните, как классы и объекты применяются в веб-приложениях.

Ответ:

- **Контроллер** — Обрабатывает HTTP-запрос и возвращает ответ
- **Модель** — Работает с данными и бизнес-логикой
- **Вид** — Отвечает за отображение данных пользователю
- **Сервис** — Выполняет отдельную бизнес-операцию, используемую разными классами

Шаг 07:

Миссия 7

 Сопоставьте тип зависимости с подходом внедрения в PHP.

Подсказка: Обратите внимание на то, кто создаёт объект: сам класс или внешний контейнер.

Ответ:

- **Конструктор** — Передача зависимости через конструктор класса
- **Сеттер** — Передача зависимости через метод установки
- **Интерфейс** — Позволяет использовать разные реализации без изменения кода
- **Сервис-контейнер** — Автоматически управляет созданием и передачей зависимостей

Шаг 08:

Миссия 8

Расположите этапы создания проекта на PHP с использованием ООП в правильном порядке:

1. Написание тестов и проверка работоспособности
2. Проектирование классов и их взаимодействия
3. Разработка функциональности, соблюдая принципы SOLID
4. Рефакторинг и оптимизация кода

Подсказка: Сначала нужно продумать архитектуру, затем писать код, а в конце проверять и улучшать.

Ответ:

- Проектирование классов и их взаимодействия
- Разработка функциональности, соблюдая принципы SOLID
- Рефакторинг и оптимизация кода
- Написание тестов и проверка работоспособности

Шаг 09:

Миссия 9

Расположите шаги внедрения паттерна Стратегия:

1. Создание интерфейса для алгоритмов
2. Реализация нескольких классов, реализующих интерфейс
3. Выбор конкретного алгоритма во время выполнения
4. Замена алгоритма без изменения кода клиента

Подсказка: Сначала определяется контракт, потом реализации, потом динамическое использование.

Ответ:

- Создание интерфейса для алгоритмов
- Реализация нескольких классов, реализующих интерфейс
- Выбор конкретного алгоритма во время выполнения
- Замена алгоритма без изменения кода клиента

Шаг 10:

Миссия 10

Вы создаёте систему логирования. Какой подход обеспечит лёгкую замену способа логирования (файл, база, внешняя система)?

Подсказка: Подумайте о возможности динамически выбирать реализацию.

Ответ:

- **Использовать интерфейс ILogger и разные реализации**
- Хардкодить запись в файл
- Использовать глобальную переменную для логирования
- Игнорировать возможность замены способа логирования

Шаг 11:

Миссия 11

Вы работаете с API внешнего сервиса и хотите абстрагировать логику его вызова. Какой паттерн подойдёт?

Подсказка: Думайте о паттерне, который предоставляет унифицированный интерфейс к сложной системе.

Ответ:

- **Фасад**
- Мост
- Строитель
- Прототип

Шаг 12:

Миссия 12

 Сопоставьте термин с примером в PHP-проекте:

Подсказка: Вспомните, какие классы управляют данными, а какие отвечают за взаимодействие с пользователем.

Ответ:

- **Репозиторий** — Класс для доступа к данным из базы
- **Сервис** — Обрабатывает бизнес-логику, вызываемую разными частями приложения
- **Контроллер** — Обрабатывает запросы пользователя

- **Вид** — Отображает данные пользователю

🔮 Шаг 13:

Миссия 13

📄 Сопоставьте тип теста с его целью:

Подсказка: Подумайте о том, что проверяет один класс, а что — взаимодействие нескольких.

Ответ:

- **Unit-тест** — Проверяет отдельные методы или классы
- **Integration-тест** — Проверяет взаимодействие нескольких компонентов
- **Functional-тест** — Проверяет функциональность приложения с точки зрения пользователя
- **Regression-тест** — Проверяет, что исправления не сломали старую функциональность

🔮 Шаг 14:

Миссия 14

Расположите этапы деплоя PHP-проекта с ООП:

1. Тестирование и проверка качества
2. Перенос кода на сервер
3. Настройка окружения и зависимостей
4. Запуск и мониторинг работы приложения

Подсказка: Сначала проверяем, затем переносим и настраиваем, потом запускаем и следим за стабильностью.

Ответ:

- Тестирование и проверка качества
- Настройка окружения и зависимостей
- Перенос кода на сервер
- Запуск и мониторинг работы приложения

🔮 Шаг 15:

Миссия 16

Расположите этапы рефакторинга сложного класса:

1. Выделение методов для повторяющегося кода
2. Разделение больших методов на маленькие
3. Проверка работоспособности после изменений
4. Оптимизация структуры классов и интерфейсов

Подсказка: Сначала устраняем дубли, потом улучшаем читаемость, проверяем, и только после этого оптимизируем архитектуру.

Ответ:

- Выделение методов для повторяющегося кода
- Разделение больших методов на маленькие
- Проверка работоспособности после изменений
- Оптимизация структуры классов и интерфейсов

Шаг 16:

Миссия 17

Расположите шаги завершения проекта на PHP с использованием ООП:

1. Код-ревью и исправление замечаний
2. Написание документации и инструкций для команды
3. Финальное тестирование и деплой на сервер
4. Мониторинг работы приложения и исправление багов

Подсказка: Важно сначала убедиться в качестве кода, затем подготовить документацию, потом деплой и наблюдение за стабильностью.

Ответ:

- Код-ревью и исправление замечаний
- Написание документации и инструкций для команды
- Финальное тестирование и деплой на сервер
- Мониторинг работы приложения и исправление багов

Шаг 17:

Миссия 17

Вы начинаете новый проект. Какой принцип стоит применять, чтобы классы оставались гибкими и понятными?

Подсказка: Подумайте, как изменения в одной части не должны ломать другие.

Ответ:

- **Принцип единственной ответственности (SRP)**
- Создание одного глобального класса для всех задач
- Хардкодить зависимости в методах
- Игнорировать интерфейсы и абстракции

Шаг 18:

Миссия 18

В проекте интернет-магазина необходимо управлять разными способами оплаты. Какой подход лучше всего?

Подсказка: Подумайте о возможности добавления новых вариантов без переписывания старого кода.

Ответ:

- **Использовать паттерн Стратегия для методов оплаты**
- Создать один switch для всех способов оплаты
- Хардкодить все способы оплаты в контроллере
- Игнорировать различия между способами оплаты

Шаг 19:

Миссия 19

Чтобы предотвратить многократную инициализацию объекта, какой паттерн стоит использовать?

Подсказка: Он обеспечивает существование только одного экземпляра класса.

Ответ:

- **Синглтон**
- Фабричный метод
- Стратегия
- Декоратор

Урок 4 - Защита перед Советом Старейшин (ревью финального проекта)

Шаг 01:

Миссия 1

Что поможет сделать код более поддерживаемым при добавлении новых функций?

Подсказка: Подумайте, как избежать изменений в уже работающих частях.

Ответ:

- **Использование интерфейсов и абстрактных классов**
- Хардкодить новые функции в существующих классах
- Игнорировать принципы SOLID
- Создавать длинные методы с множеством условий

Шаг 02:

Миссия 2

Вы хотите добавить новые способы оплаты в интернет-магазине без переписывания существующего кода. Какой паттерн выбрать?

Подсказка: Подумайте о возможности менять алгоритм динамически.

Ответ:

- **Паттерн Стратегия**
- Синглтон
- Фабричный метод
- Декоратор

Шаг 03:

Миссия 3

Чтобы гарантировать единственный экземпляр объекта конфигурации, какой паттерн применить?

Подсказка: Он ограничивает создание объекта одним экземпляром.

Ответ:

- **Синглтон**
- Стратегия
- Фабричный метод
- Декоратор

Шаг 04:

Миссия 4

 Сопоставьте паттерн с его назначением.

Подсказка: Подумайте, какой паттерн создаёт объект, а какой управляет поведением.

Ответ:

- **Фабричный метод** — Создаёт объекты без указания точного класса
- **Синглтон** — Обеспечивает существование только одного экземпляра
- **Стратегия** — Позволяет выбирать алгоритм во время выполнения
- **Декоратор** — Динамически добавляет функциональность объекту

Шаг 05:

Миссия 5

 Сопоставьте принцип SOLID с его объяснением.

Подсказка: Обратите внимание на слова «ответственность», «расширяемость», «замена».

Ответ:

- **SRP** — Класс должен иметь только одну причину для изменения
- **ОСР** — Код открыт для расширения, но закрыт для изменения
- **LSP** — Подклассы могут заменять суперкласс без нарушений
- **ISP** — Клиенты не зависят от интерфейсов, которые они не используют

Шаг 06:

Миссия 6

 Сопоставьте компоненты MVC с их назначением в PHP-проекте.

Подсказка: Кто отвечает за данные, за отображение, а кто за логику?

Ответ:

- **Модель** — Работает с данными и бизнес-логикой
- **Вид** — Отвечает за отображение данных пользователю
- **Контроллер** — Обрабатывает запросы и связывает модель с видом
- **Сервис** — Выполняет отдельную бизнес-операцию

Шаг 07:

Миссия 7

 Сопоставьте способ внедрения зависимости с примером в PHP.

Подсказка: Подумайте, кто создаёт объект: сам класс или внешний контейнер?

Ответ:

- **Конструктор** — Передача зависимости через конструктор
- **Сеттер** — Передача зависимости через метод установки
- **Интерфейс** — Позволяет использовать разные реализации без изменения кода клиента
- **Сервис-контейнер** — Автоматическое управление созданием и передачей зависимостей

Шаг 08:

Миссия 8

Расположите шаги внедрения паттерна Стратегия:

1. Создать интерфейс для алгоритмов
2. Реализовать классы с разными алгоритмами
3. Выбрать алгоритм во время выполнения
4. Заменить алгоритм без изменения кода клиента

Подсказка: Сначала контракт, потом реализации, потом динамическое использование.

Ответ:

- Создать интерфейс для алгоритмов
- Реализовать классы с разными алгоритмами
- Выбрать алгоритм во время выполнения
- Заменить алгоритм без изменения кода клиента

 Шаг 09:

Миссия 9

Расположите этапы создания Синглтона:

1. Приватный конструктор
2. Статический метод для получения экземпляра
3. Проверка существования экземпляра
4. Возврат единственного экземпляра

Подсказка: Сначала ограничиваем создание, потом контролируем доступ.

Ответ:

- Приватный конструктор
- Статический метод для получения экземпляра
- Проверка существования экземпляра
- Возврат единственного экземпляра

 Шаг 10:

Миссия 10

Расположите шаги подготовки к финальному ревью проекта:

1. Собрать все изменения в ветке разработки
2. Запустить автоматические тесты
3. Проверить соответствие стандартам кодирования
4. Подготовить презентацию для Совета Старейшин

Подсказка: Сначала код, затем тесты, потом проверка качества и презентация.

Ответ:

- Собрать все изменения в ветке разработки
- Запустить автоматические тесты
- Проверить соответствие стандартам кодирования
- Подготовить презентацию для Совета Старейшин

 Шаг 11:

Миссия 11

 Сопоставьте ошибки кода с типичными последствиями в финальном проекте:

Подсказка: Думайте, что произойдёт, если нарушить принципы ООП.

Ответ:

- **Нарушение инкапсуляции** — Прямой доступ к внутренним данным может привести к непредвиденным изменениям
- **Дублирование кода** — Трудно поддерживать и исправлять баги
- **Сильная связанность** — Изменения в одном классе ломают другие
- **Отсутствие документации** — Новым разработчикам сложно понять архитектуру

🔮 Шаг 12:

Миссия 12

📖 Сопоставьте тип тестирования с его задачей:

Подсказка: Подумайте, что проверяют unit, integration и end-to-end тесты.

Ответ:

- **Unit-тесты** — Проверяют отдельные методы и классы
- **Integration-тесты** — Проверяют взаимодействие между модулями
- **End-to-end тесты** — Проверяют работу всей системы от начала до конца
- **Smoke-тесты** — Быстрая проверка основных функций приложения

🔮 Шаг 13:

Миссия 13

Расположите шаги рефакторинга перед защитой проекта:

1. Определить участки кода, требующие улучшения
2. Создать тесты для этих участков
3. Внести изменения без нарушения функциональности
4. Прогнать тесты и убедиться, что проект работает

Подсказка: Сначала диагностика, потом тесты, изменения и проверка.

Ответ:

- Определить участки кода, требующие улучшения
- Создать тесты для этих участков
- Внести изменения без нарушения функциональности
- Прогнать тесты и убедиться, что проект работает

🔮 Шаг 14:

Миссия 14

Какой подход помогает защитить проект от непредвиденных багов перед ревью?

Подсказка: Надёжный способ — это проверка поведения каждого модуля отдельно и вместе.

Ответ:

- **Покрытие проекта тестами и код-ревью**
- Игнорирование мелких ошибок
- Добавление новых функций в последний момент
- Удаление старых модулей без проверки

🔮 Шаг 15:

Миссия 15

📖 Сопоставьте типы архитектурных проблем с примерами:

Подсказка: Подумайте, что чаще всего ломает масштабирование, тестируемость и поддержку.

Ответ:

- **Тесная связность** — Изменение одного класса вызывает поломку других
- **Нарушение SRP** — Класс делает слишком много, сложно тестировать
- **Отсутствие интерфейсов** — Трудно заменить реализацию без изменений клиента
- **Плохая документация** — Сложно понять и поддерживать код

Шаг 16:

Миссия 16

Расположите этапы подготовки презентации проекта для Совета Старейшин:

1. Выделение ключевых функций проекта
2. Подготовка демонстрации работы модулей
3. Создание слайдов с пояснениями архитектуры
4. Репетиция выступления

Подсказка: Сначала что показать, затем демонстрация, пояснения и репетиция.

Ответ:

- Выделение ключевых функций проекта
- Подготовка демонстрации работы модулей
- Создание слайдов с пояснениями архитектуры
- Репетиция выступления

Шаг 17:

Миссия 17

Что важнее всего при защите проекта перед Советом Старейшин?

Подсказка: Подумайте, что оценят профессионалы: стабильность, читаемость и демонстрация работы.

Ответ:

- **Показ стабильной и работающей версии с понятной архитектурой**
- Демонстрация всех экспериментов и тестовых функций
- Презентация с минимальным количеством кода
- Использование самых новых и нестабильных библиотек

Урок 5 - Получение чёрного пояса — церемония мастера PHP

Шаг 01:

Миссия 1

Что символизирует получение чёрного пояса в PHP?

Подсказка: Это не просто цвет, а знак мастерства и понимания принципов ООП.

Ответ:

- **Высокий уровень мастерства и понимания PHP 8 ООП**
- Простое знание синтаксиса
- Умение писать длинные скрипты
- Использование любых библиотек без анализа

Шаг 02:

Миссия 2

Какая привычка помогает поддерживать чистоту кода перед церемонией?

Подсказка: Это правило сокращает дублирование и повышает читаемость.

Ответ:

- **Вынос повторяющегося кода в отдельные функции**
- Игнорирование комментариев

- Смешивание логики с отображением
- Применение непонятных сокращений

🔧 Шаг 03:

Миссия 3

Что важно продемонстрировать на церемонии мастера PHP?

Подсказка: Думайте о стабильности и архитектуре проекта.

Ответ:

- **Рабочую и хорошо структурированную систему**
- Проект без тестов, но с новым дизайном
- Случайные эксперименты с кодом
- Минимальный функционал без демонстрации архитектуры

🔧 Шаг 04:

Миссия 4

📖 Сопоставьте термины с их объяснением, важным для церемонии мастера:

Подсказка: Обратите внимание, какие принципы делают проект стабильным.

Ответ:

- **Инкапсуляция** — Скрытие внутренних данных и методов, доступ через публичный интерфейс
- **Наследование** — Создание новых классов на основе существующих для повторного использования кода
- **Полиморфизм** — Возможность использовать объекты разных классов через общий интерфейс
- **SRP** — Каждый класс выполняет только одну ответственность

🔧 Шаг 05:

Миссия 5

📖 Сопоставьте тип ошибки с её возможным последствием:

Подсказка: Подумайте, как ошибки влияют на демонстрацию проекта.

Ответ:

- **Дублирование кода** — Сложнее поддерживать и исправлять баги
- **Плохие названия** — Новым разработчикам трудно понять назначение переменных
- **Сильная связанность** — Изменения одного модуля ломают другие
- **Отсутствие тестов** — Трудно проверить корректность проекта

🔧 Шаг 06:

Миссия 6

📖 Сопоставьте тип теста с его назначением:

Подсказка: Думайте, что проверяют unit, integration и e2e тесты.

Ответ:

- **Unit-тест** — Проверяет отдельный метод или класс
- **Integration-тест** — Проверяет взаимодействие между модулями
- **End-to-end тест** — Проверяет работу всей системы от начала до конца
- **Smoke-тест** — Быстрая проверка основных функций

Шаг 07:

Миссия 7

Расположите шаги подготовки к церемонии мастера:

1. Сбор всех изменений и веток проекта
2. Запуск тестов для проверки стабильности
3. Проверка соответствия стандартам кодирования
4. Подготовка демонстрации работы системы

Подсказка: Сначала код, затем тесты, проверка качества и демонстрация.

Ответ:

- Сбор всех изменений и веток проекта
- Запуск тестов для проверки стабильности
- Проверка соответствия стандартам кодирования
- Подготовка демонстрации работы системы

Шаг 08:

Миссия 8

Расположите шаги защиты проекта перед Советом Старейшин:

1. Выделение ключевых функций проекта
2. Подготовка демонстрации модулей
3. Создание слайдов с пояснением архитектуры
4. Репетиция выступления

Подсказка: Сначала что показать, затем демонстрация, слайды и репетиция.

Ответ:

- Выделение ключевых функций проекта
- Подготовка демонстрации модулей
- Создание слайдов с пояснением архитектуры
- Репетиция выступления

Шаг 09:

Миссия 9

 Сопоставьте тип теста с его назначением:

Подсказка: Думайте, что проверяют unit, integration и e2e тесты.

Ответ:

- **Unit-тест** — Проверяет отдельный метод или класс
- **Integration-тест** — Проверяет взаимодействие между модулями
- **End-to-end тест** — Проверяет работу всей системы от начала до конца
- **Smoke-тест** — Быстрая проверка основных функций

Шаг 10:

Миссия 10

Расположите шаги подготовки к церемонии мастера:

1. Сбор всех изменений и веток проекта
2. Запуск тестов для проверки стабильности

3. Проверка соответствия стандартам кодирования
4. Подготовка демонстрации работы системы

Подсказка: Сначала код, затем тесты, проверка качества и демонстрация.

Ответ:

- Сбор всех изменений и веток проекта
- Запуск тестов для проверки стабильности
- Проверка соответствия стандартам кодирования
- Подготовка демонстрации работы системы

Шаг 11:

Миссия 11

Расположите шаги защиты проекта перед Советом Старейшин:

1. Выделение ключевых функций проекта
2. Подготовка демонстрации модулей
3. Создание слайдов с пояснением архитектуры
4. Репетиция выступления

Подсказка: Сначала что показать, затем демонстрация, слайды и репетиция.

Ответ:

- Выделение ключевых функций проекта
- Подготовка демонстрации модулей
- Создание слайдов с пояснением архитектуры
- Репетиция выступления

Шаг 12:

Миссия 12

Как лучше всего демонстрировать архитектуру проекта на церемонии?

Подсказка: Важна наглядность и ясность для Совета Старейшин.

Ответ:

- **Через схемы классов и диаграммы взаимодействий**
- Просто показывая код без пояснений
- Печатной документацией без примеров
- Упоминанием имён методов без контекста

Шаг 13:

Миссия 13

Что является ключевым в защите проекта перед экспертами?

Подсказка: Это не внешняя красота, а внутренняя стабильность.

Ответ:

- **Стабильность работы и тестовое покрытие**
- Использование модных фреймворков
- Больше комментариев, чем кода
- Превосходный UI без архитектуры

🔗 Шаг 14:

Миссия 14

📖 Сопоставьте принцип ООП с его эффектом на проект:

Подсказка: Подумайте, какой принцип облегчает поддержку и расширение.

Ответ:

- **Инкапсуляция** — Защищает внутренние данные от нежелательного доступа
- **Наследование** — Повторное использование кода без копирования
- **Полиморфизм** — Позволяет использовать разные объекты через единый интерфейс
- **SRP** — Каждый класс выполняет только одну задачу

🔗 Шаг 15:

Миссия 15

📖 Сопоставьте элементы проекта с их проверкой:

Подсказка: Подумайте, какие аспекты важно показать на церемонии.

Ответ:

- **Бизнес-логика** — Unit-тесты
- **Взаимодействие модулей** — Integration-тесты
- **Полная система** — End-to-end тесты
- **Главные функции** — Smoke-тесты

🔗 Шаг 16:

Миссия 16

Расположите шаги по устранению критических ошибок перед демонстрацией:

1. Анализ отчёта об ошибках
2. Исправление ошибок
3. Повторное тестирование исправлений
4. Документирование изменений

Подсказка: Сначала анализ, потом исправление, тесты и документация.

Ответ:

- Анализ отчёта об ошибках
- Исправление ошибок
- Повторное тестирование исправлений
- Документирование изменений